CHAPTER 10 Graphs

10.1 Graphs and Graph Models

10.2 Graph Terminology and Special Types of Graphs

10.3 Representing Graphs and Graph Isomorphism

10.4 Connectivity

10.5 Euler and Hamilton Paths

10.6 Shortest Path Problems

10.7 Planar Graphs

10.8 Graph Coloring



Graphs theory

Graphs theory is an old subject with many modern application.

For example, graphs can be used

- to study the structure of the World Wide Web.

- to determine whether a circuit can be implemented on a planar circuit board.

- to solve problem such as finding the shortest path between two cities in a transportation network.

- to schedules exams, and so on.



10.1 Graphs and Graph Models

【Definition 1】 A *graph* $G=(V,E)$ consists of V, a nonempty set of *vertices* (or *nodes*) and E, a set of *edges*. Each edge has either one or two vertices associated with it, called its *endpoints*. An edge is said to *connect* its endpoints.

Remark:

Infinite Graph, finite Graph

Types of *Undirected Graphs*

- **Simple graph**
- **Multigraph**
- **Pseudograph**

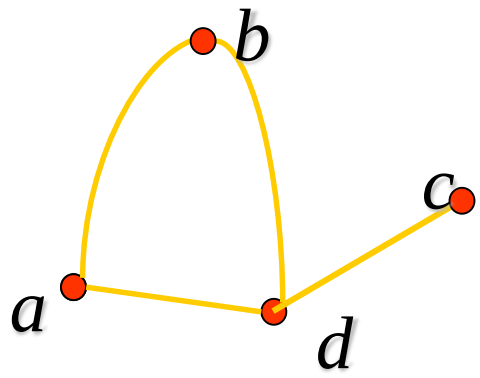


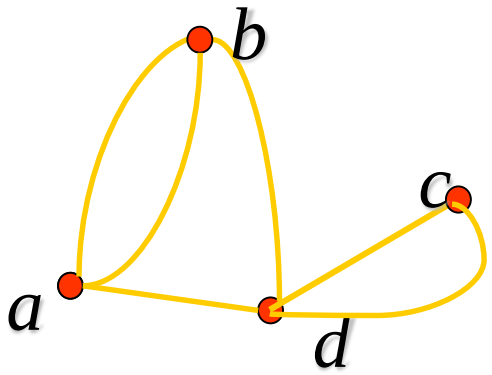
***Simple* graph: A graph in which each edge connects two different vertices and where no two edges connect the same pair of vertices.**

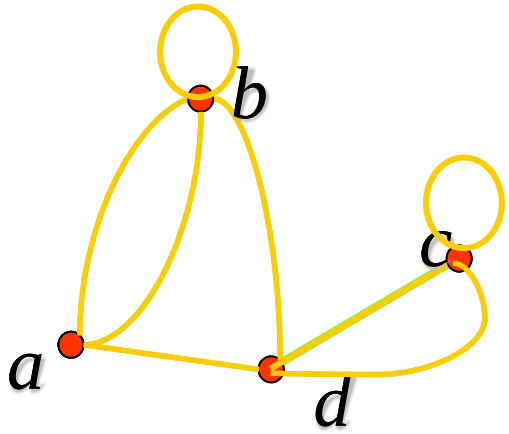
***Multigraph:* Graphs that may have multiple edges connecting the same vertices.**

***Pseudograph:* Graphs that may include loops, and possibly multiple edges connecting the same pair of vertices.**

For example,







A simple graph

SET OF VERTICES

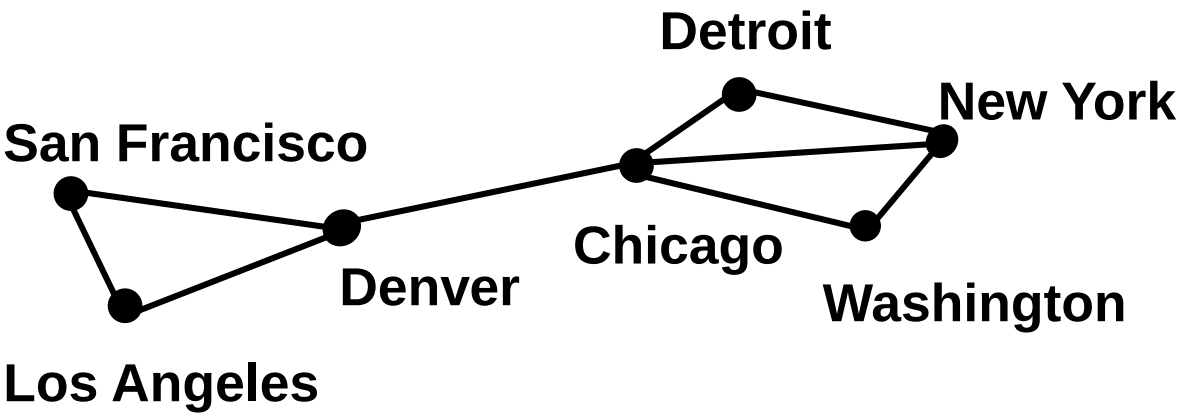
V = { Chicago, Denver, Detroit, Los Angeles,
New York, San Francisco, Washington }

SET OF EDGES

E = { {San Francisco, Los Angeles}, {San Francisco, Denver},
{Los Angeles, Denver}, {Denver, Chicago},
{Chicago, Detroit}, {Detroit, New York},
{New York, Washington}, {Chicago, Washington},
{Chicago, New York} }



A simple graph

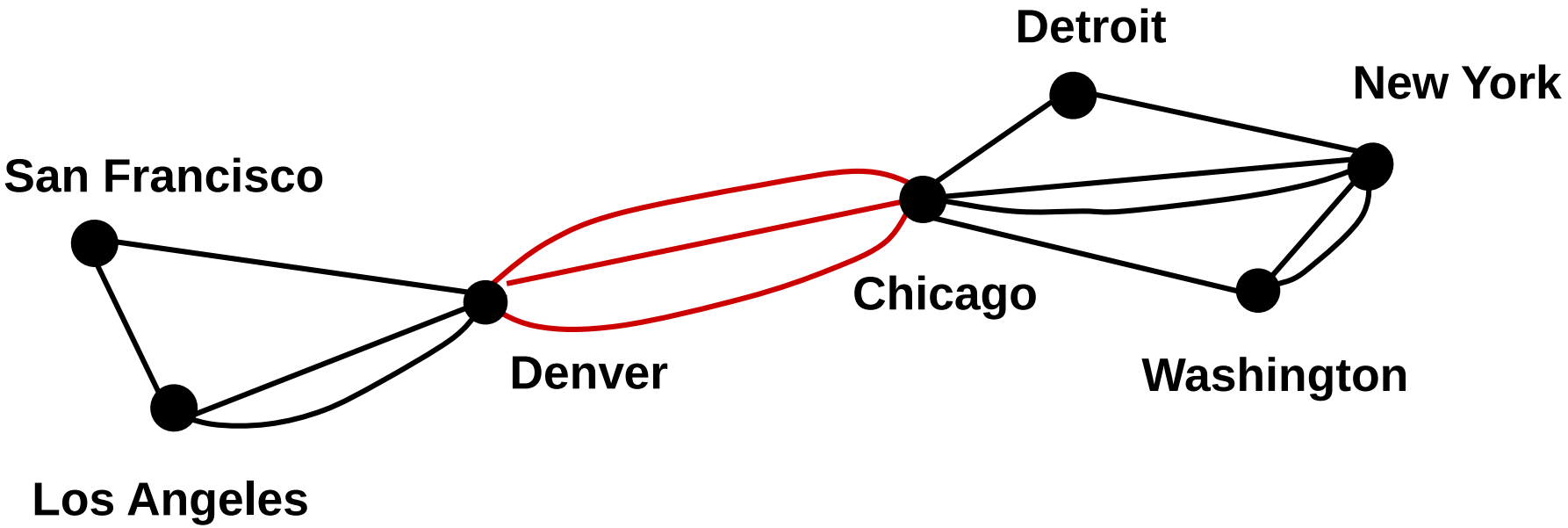


The network is made up of computers and telephone lines between computers. There is at most 1 telephone line between 2 computers in the network. Each line operates in both directions. No computer has a telephone line to itself.

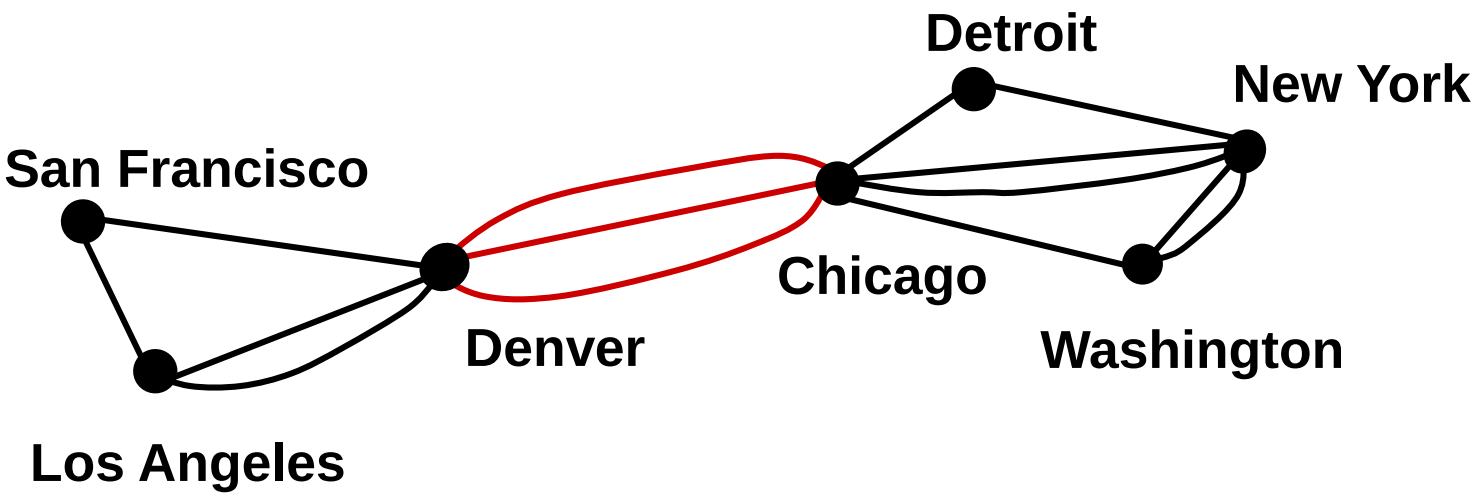
These are undirected edges,
each of which connects two distinct vertices, and
no two edges connect the same pair of vertices.

A Multigraph

THERE CAN BE MULTIPLE TELEPHONE LINES BETWEEN TWO COMPUTERS IN THE NETWORK.



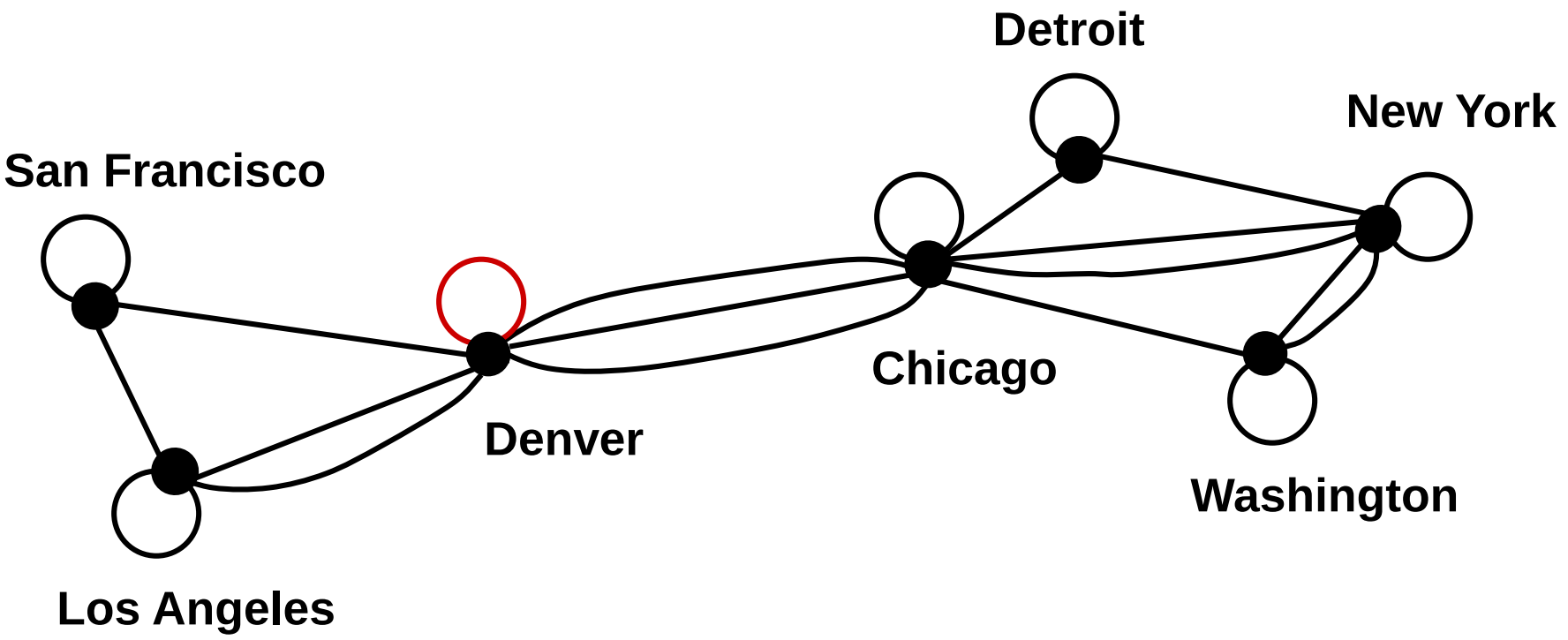
Multiple Edges



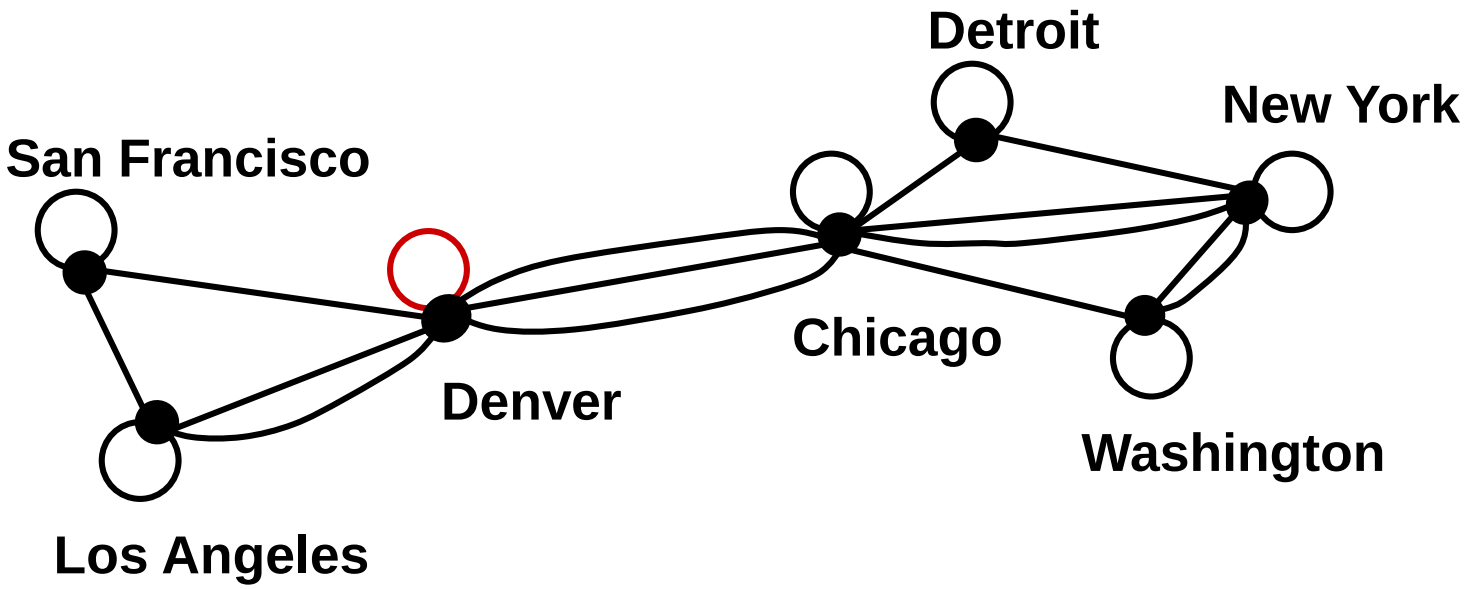
Two edges are called *multiple or parallel edges* if they connect the same two distinct vertices.

A Pseudograph

THERE CAN BE TELEPHONE LINES IN THE NETWORK FROM A COMPUTER TO ITSELF (for diagnostic use).

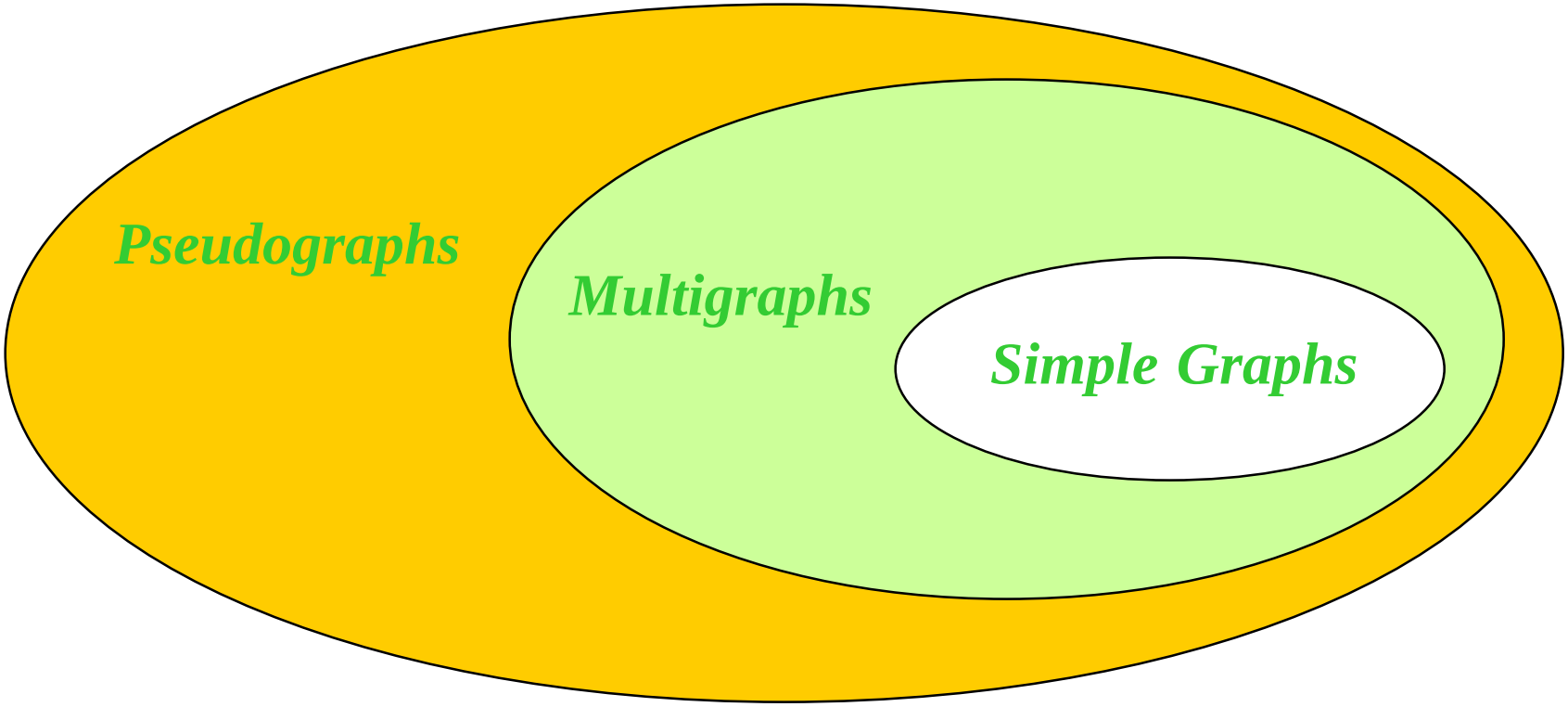


Loops



An edge is called a *loop*
if it connects a vertex to itself.

The relations of different undirected graphs



【Definition 2】A *directed graph* (or *digraph*) *(V, E)* consists of a nonempty set of vertices *V* and a set of *directed edges* (or *arcs*) E. Each directed edge is associated with an ordered pair of vertices. The directed edge associated with the ordered pair (*u*,*v*) is said to *start* at *u* and *end* at *v*.

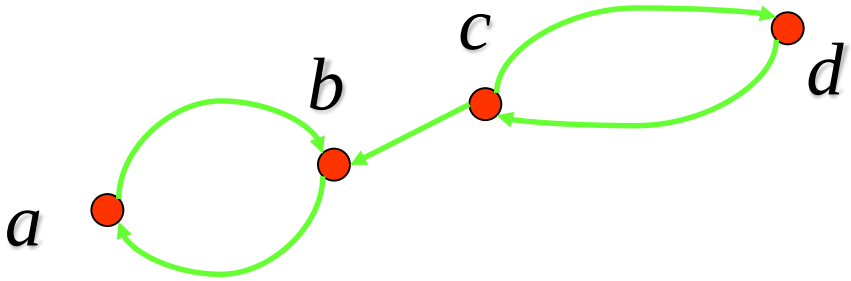


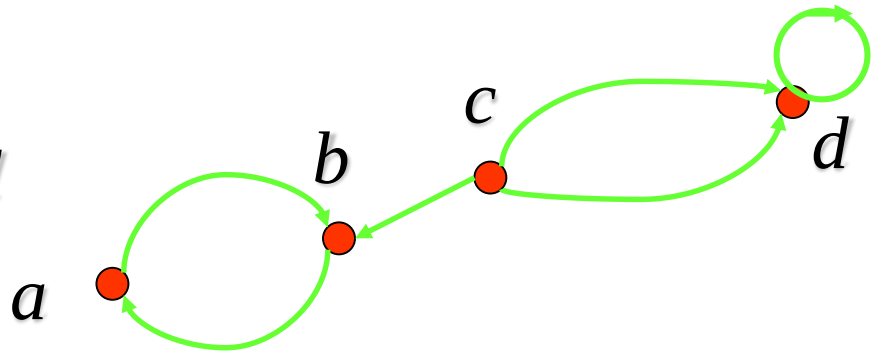
Types of digraphs:

Simple directed graph: **a directed graph has no loops and has no multiple directed edges.**

directed multigraph: **a directed graphs that may have multiple directed edges from a vertex to a second (possibly the same) vertex.**

For example,





Graph Models

Problems in almost every conceivable discipline can be solved using graph models.

For example,

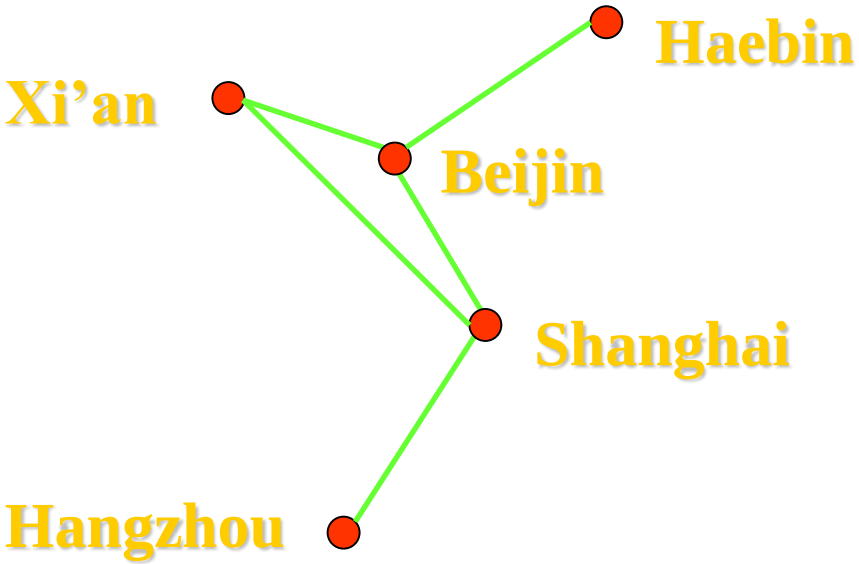
- ✓ **Niche overlap Graphs in Ecology**
- ✓ **Influence Graphs**
- ✓ **The Hollywood Graph**
- ✓ **Round-Robin Tournament**
- ✓ **The Web Graph**
- ✓ **……**



〖Example 1〗 How can we represent a network of (bi-directional) railways connecting a set of cities?

Solution:

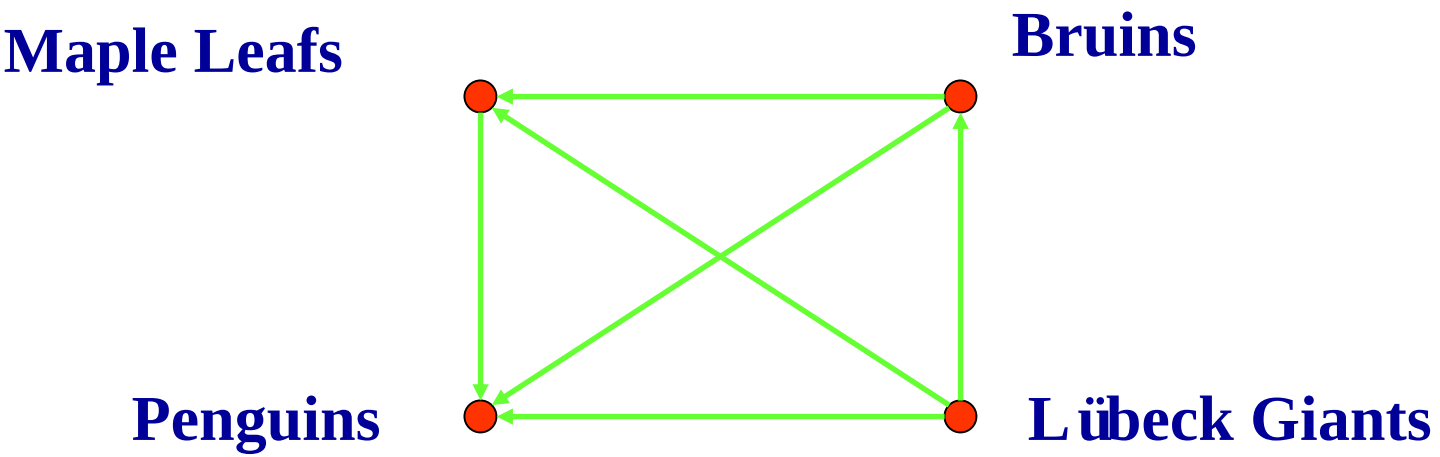
We should use a simple graph with an edge {*a*, *b*} indicating a direct train connection between cities *a* and *b*.



〖Example 2〗 In a round-robin tournament, each team plays against each other team exactly once. How can we represent the results of the tournament (which team beats which other team)?

Solution:

We should use a *directed graph* with an edge (*a*, *b*) indicating that team *a* beats team *b*.



Homework:

P. 682 : 3-9, 21, 23



CHAPTER 10 Graphs

10.1 Graphs and Graph Models

10.2 Graph Terminology and Special Types of Graphs

10.3 Representing Graphs and Graph Isomorphism

10.4 Connectivity

10.5 Euler and Hamilton Paths

10.6 Shortest Path Problems

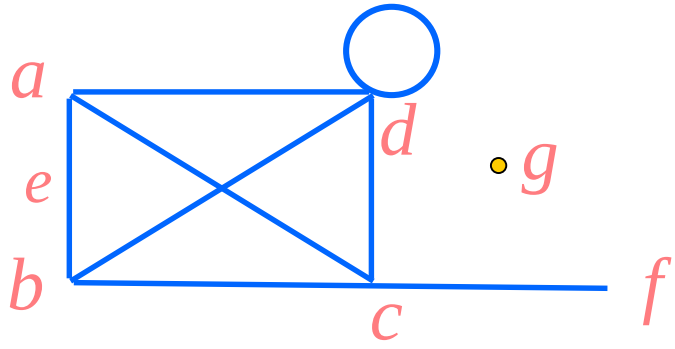
10.7 Planar Graphs

10.8 Graph Coloring



1. Basic Terminology

Undirected Graphs $G=(V, E)$



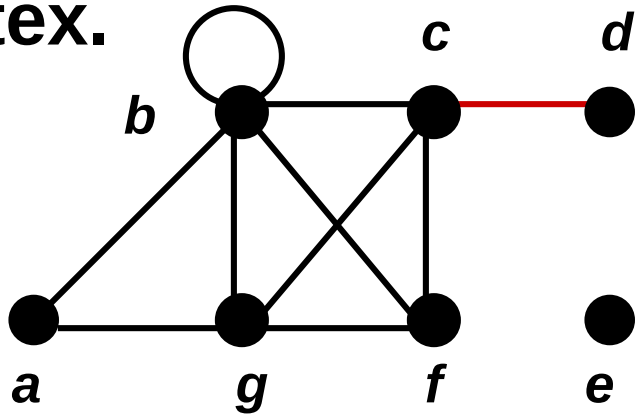
- Two vertices, u and v in an undirected graph G are called *adjacent* (or *neighbors*) in G, if $\{u, v\}$ is an edge of G.
- An edge e connecting u and v is called *incident with vertices u and v*, or is said to connect u and v.
- The vertices u and v are called *endpoints* of edge $\{u, v\}$.
- The *degree of a vertex* in an undirected graph is the number of edges incident with it, except that a loop at a vertex contributes twice to the degree of that vertex

Notation: deg(v)

- If deg(v) = 0, v is called *isolated*.
- If deg(v) = 1, v is called *pendant*.

Degree of a vertex

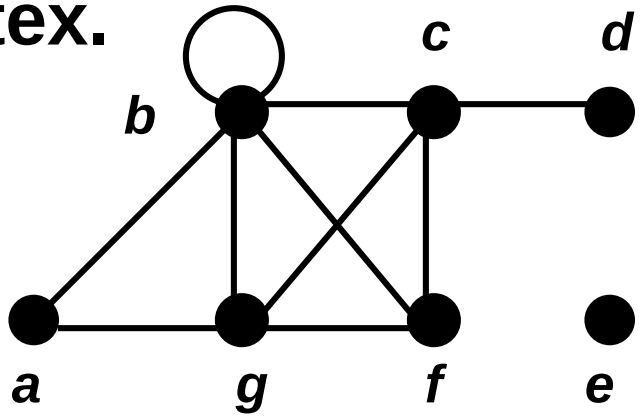
Definition 1. The **degree of a vertex** in an undirected graph is the number of edges incident with it, except that a loop at a vertex contributes twice to the degree of that vertex.



deg(*d*) = 1

Degree of a vertex

Definition 1. The **degree of a vertex** in an undirected graph is the number of edges incident with it, except that a loop at a vertex contributes twice to the degree of that vertex.

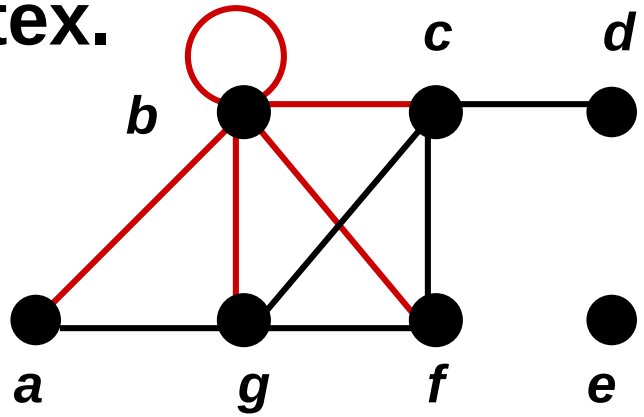


deg(*e*) = 0

Degree of a vertex

Definition 1. The **degree of a vertex** in an undirected graph is the number of edges incident with it, except that a loop at a vertex contributes twice to the degree of that vertex.

deg(*b*) = 6

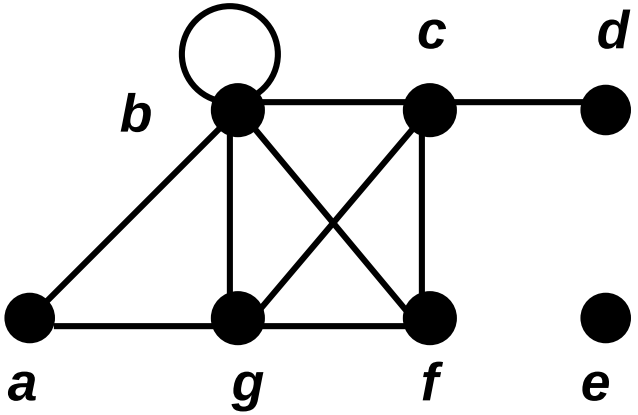


Degree of a vertex

Find the degree of all the other vertices.

deg(*a*) **deg(c)** **deg(f)** **deg(*g*)**

deg(*b*) = 6



deg(*d*) = 1

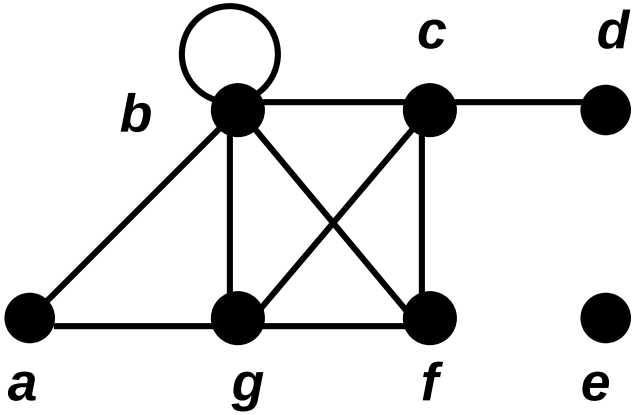
deg(*e*) = 0

Degree of a vertex

Find the degree of all the other vertices.

deg(*a*) = 2 deg(c) = 4 deg(f) = 3 deg(*g*) = 4

deg(*b*) = 6



deg(*d*) = 1

deg(*e*) = 0

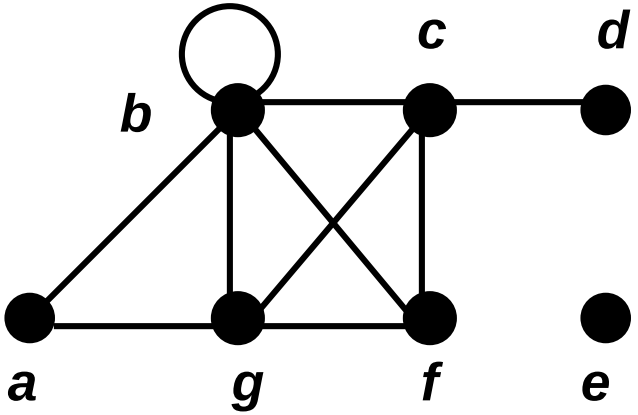
Degree of a vertex

Find the degree of all the other vertices.

deg(*a*) = 2 deg(c) = 4 deg(f) = 3 deg(*g*) = 4

TOTAL of degrees = 2 + 4 + 3 + 4 + 6 + 1 + 0 = 20

deg(*b*) = 6



deg(*d*) = 1

deg(*e*) = 0

Degree of a vertex

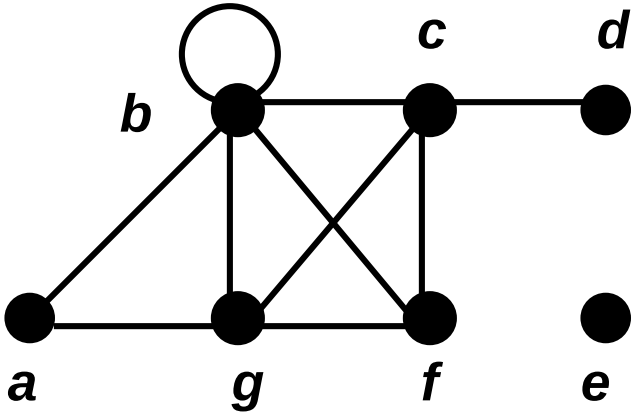
Find the degree of all the other vertices.

deg(*a*) = 2 deg(c) = 4 deg(f) = 3 deg(*g*) = 4

TOTAL of degrees = 2 + 4 + 3 + 4 + 6 + 1 + 0 = 20

TOTAL NUMBER OF EDGES = 10

deg(*b*) = 6



deg(*d*) = 1

deg(*e*) = 0

【Theorem 1】 The Handshaking Theorem

Let $G = (V, E)$ be an undirected graph G with e edges. Then

$$\sum_{v \in V} \deg(v) = 2e$$

The sum of the degrees over all the vertices equals twice the number of edges.

Proof:

Each edge represents contributes twice to the degree count of all vertices.

Note:

This applies even if multiple edges and loops are present.



Application:

The sum, over the set of people at a party, of the number of people a person has shaken hands with, is even.

Qusetion:

If a graph has 5 vertices, can each vertex have degree 3? 4?

- **The sum is 3•5 = 15 which is an odd number.**

  Not possible.

- **The sum is 20 = 2 | *E* | and 20/2 = 10.**

  May be possible.



【Theorem 2】 An undirected graph has an even number of vertices of odd degree.

Proof:

Let V_1, V_2 be the set of vertices of even degree and the set of vertices of odd degree, respectively.

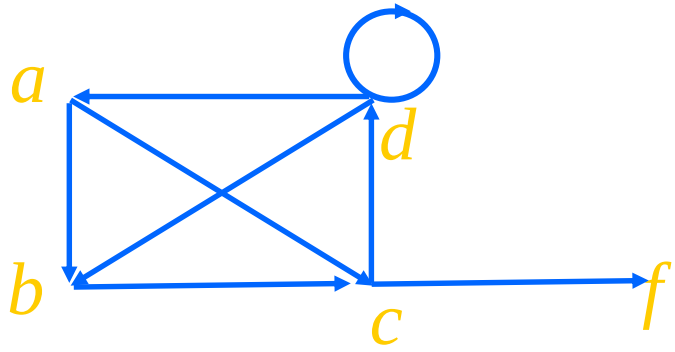
$$\sum_{v \in V_1} d(v) + \sum_{v \in V_2} d(v) = 2m$$

Qusetion:

Is it possible to have a graph with 3 vertices each of which has degree 3?



Directed Graphs $G=(V, E)$



Let (u, v) be an edge in G. Then u is an *initial vertex* and is *adjacent to* v and v is a *terminal vertex* and is *adjacent from* u.

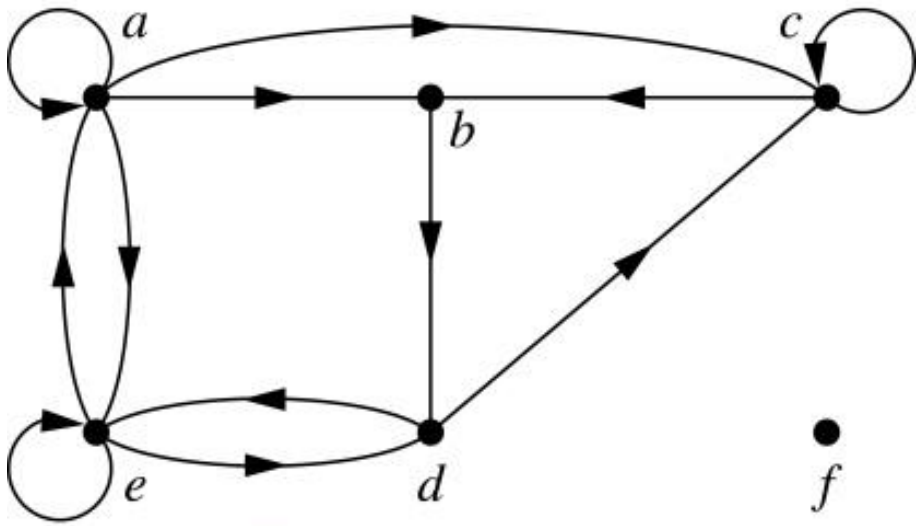
The *in degree* of a vertex v, denoted $\deg^-(v)$ is the number of edges which terminate at v.

Similarly, the *out degree* of v, denoted $\deg^+(v)$, is the number of edges which initiate at v.

underlying undirected graph

【Theorem 3】 Let $G = (V, E)$ be a graph with direct edges. Then

$$\sum_{v\in V} d^+(v) = \sum_{v\in V} d^-(v) = |E|$$



G

$$\deg^-(a) = 2, \deg^-(b) = 2, \deg^-(c) = 3,$$
$$\deg^-(d) = 2, \deg^-(e) = 3, \deg^-(f) = 0.$$

$$\deg^+(a) = 4, \deg^+(b) = 1, \deg^+(c) = 2,$$
$$\deg^+(d) = 2, \deg^+(e) = 3, \deg^+(f) = 0.$$

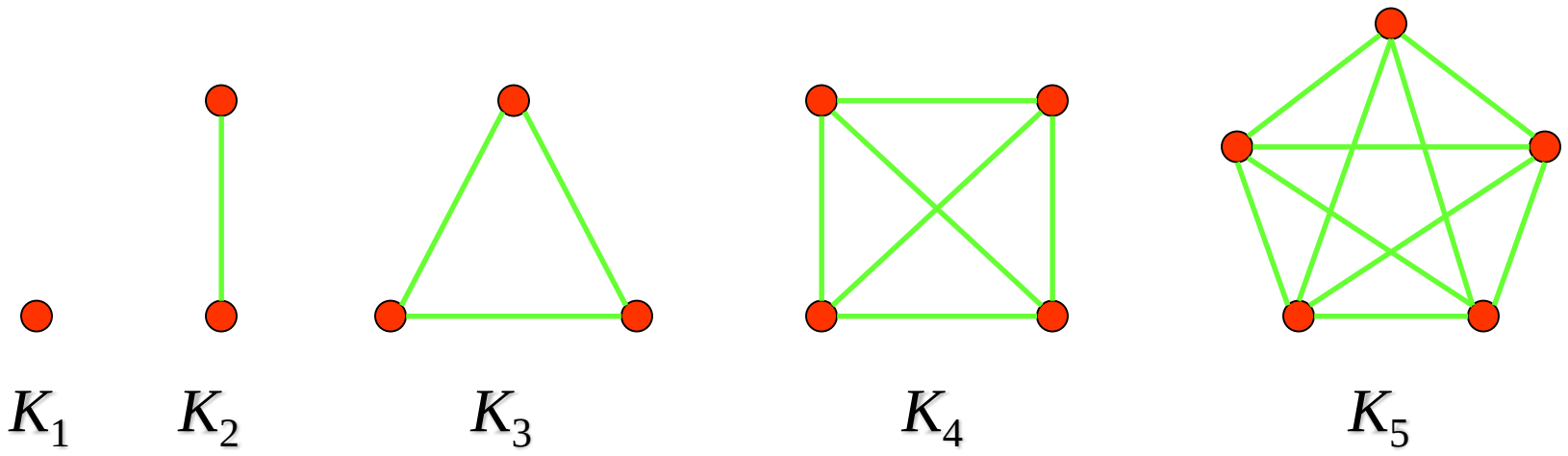
2. Some Special Simple Graphs

(1) Complete Graphs - K_n: **the simple graph with**

- *n* vertices

- exactly one edge between every pair of distinct vertices.

The graphs K_n for n=1,2,3,4,5.

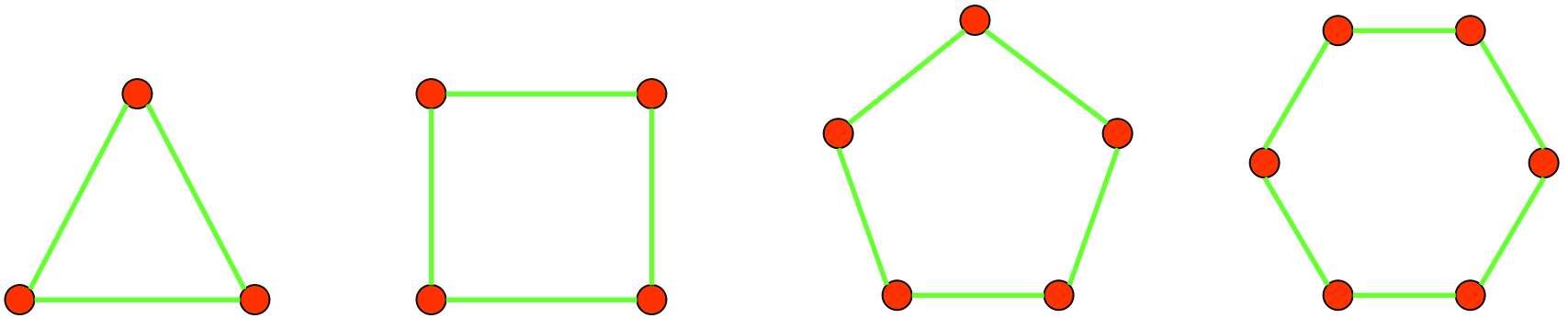


Qusetion:

The number of edges in K_n ?

(2) Cycles C_n (n>2)

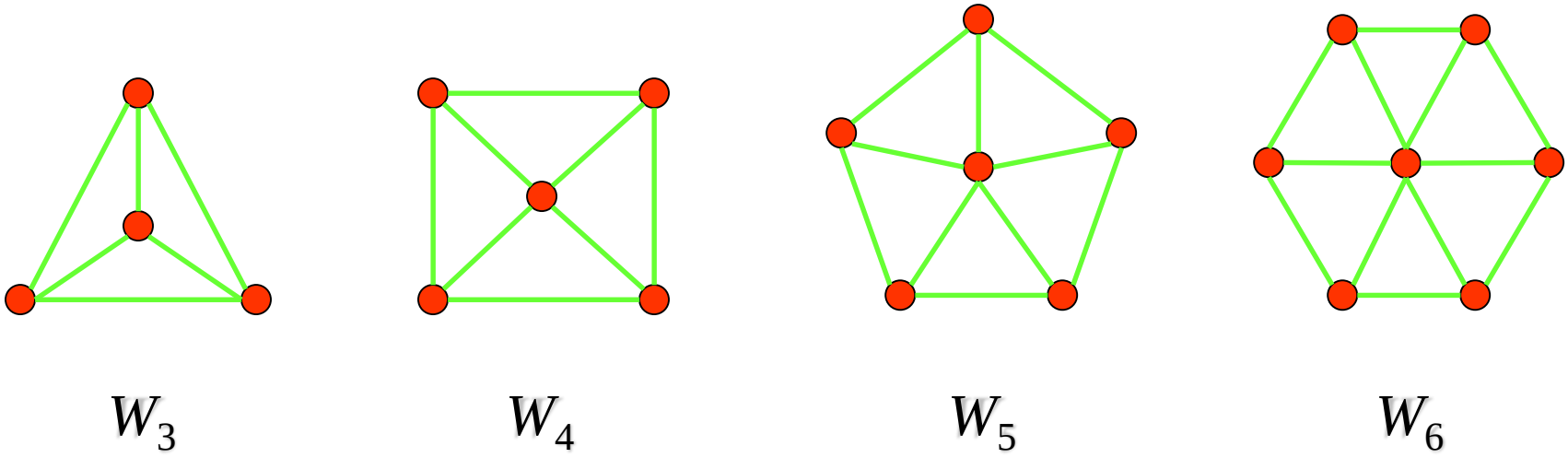
C_n is an n vertex graph which is a cycle.



C_3 C_4 C_5 C_6

(3) Wheels W_n (n>2)

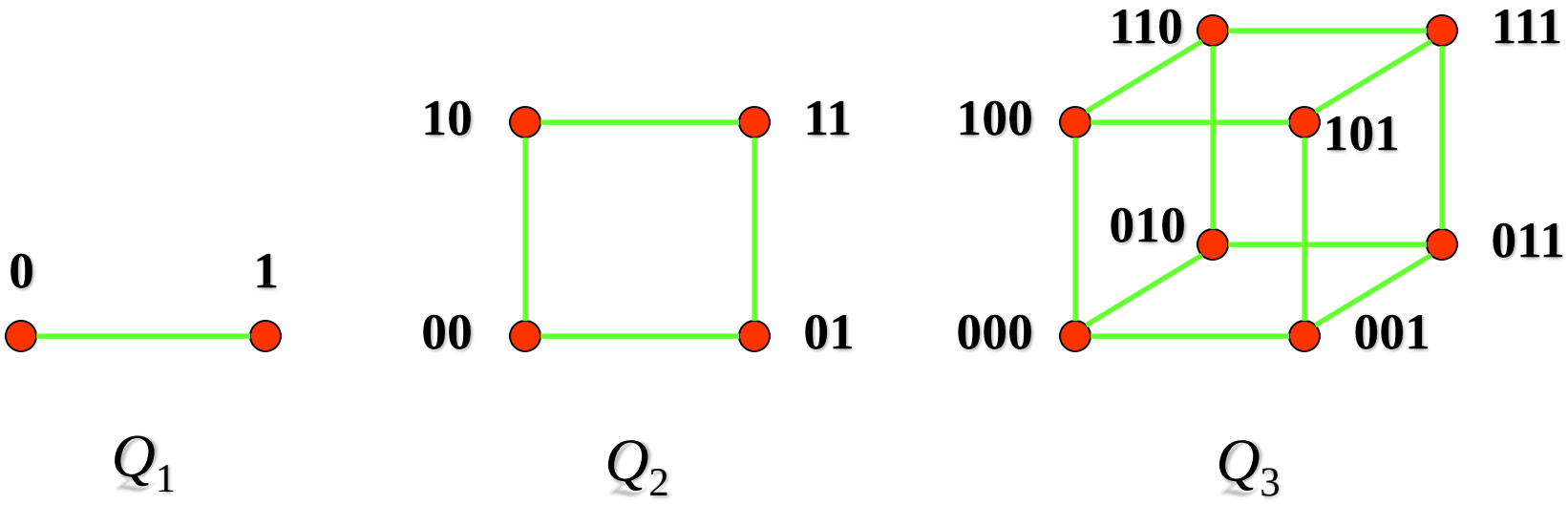
Add one additional vertex to the cycle C_n and add an edge from each vertex to the new vertex to produce W_n.



W_3 W_4 W_5 W_6

(4) *n-Cubes* Q_n (n>0)

Q_n is the graph with 2^n vertices representing bit strings of length n. An edge exists between two vertices that differ by one bit position.



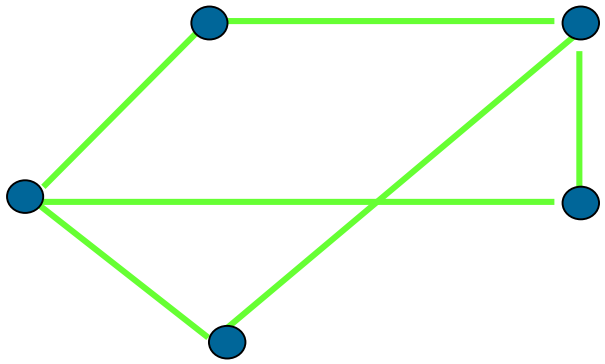
3. Bipartite Graphs

A simple graph G is *bipartite* if V can be partitioned into two disjoint subsets V_1 and V_2 such that every edge connects a vertex in V_1 and a vertex in V_2.

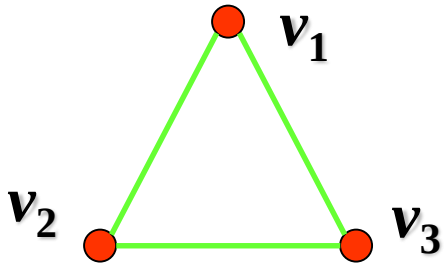
Note:

There are no edges which connect vertices in V_1 or in V_2.

For example,

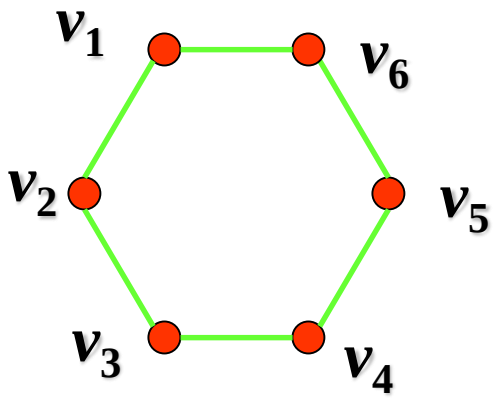


〖Example 1〗 Is C_3 bipartite?

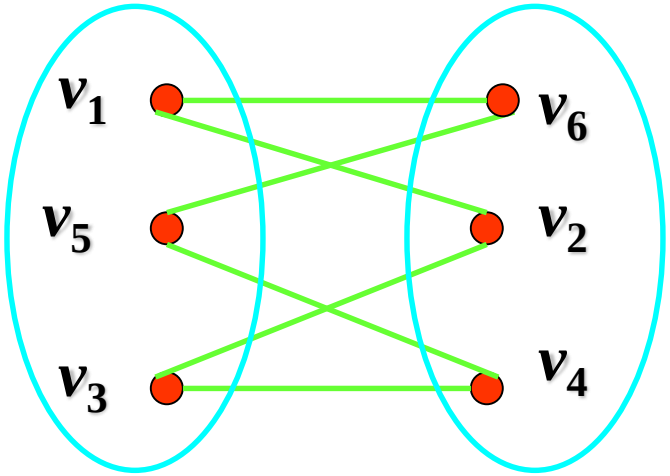


No.

〖Example 2〗 Is C_6 bipartite?



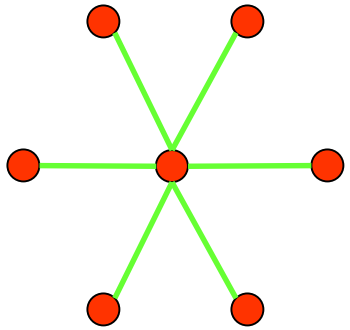
Yes. Because we can display C_6 like this:



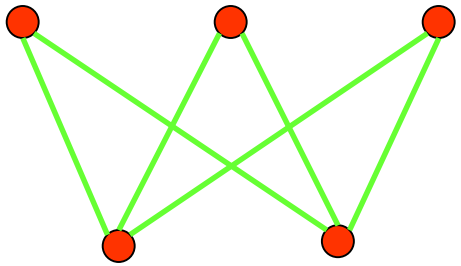
The *complete bipartite graph* is the simple graph that has its vertex set partitioned into two subsets V_1 and V_2 with m and n vertices, respectively, and *every vertex* in V_1 is connected to *every vertex* in V_2, denoted by $K_{m,n}$, where $m = |V_1|$ and $n = |V_2|$.

For example,

(1) A Star network is a $K_{1,n}$



(2) $K_{3,2}$



【Theorem 4】 A simple graph is bipartite if and only if it is possible to assign one of two different colors to each vertex of the graph so that no two adjacent vertices are assigned the same color.

Proof:

(1) Suppose that $G=(V, E)$ is a bipartite simple graph. Then $V=V_1 \cup V_2$, where V_1, V_2 are disjoint sets and every edge in E connects a vertex in V_1 and a vertex in V_2 .

(2) Suppose that it is possible to assign colors to the vertices of the graph using just two colors so that no two adjacent vertices are assigned the same color.



Regular graph

A simply graph is called *regular* if every vertex of this graph has the same degree.

A *regular graph* is called *n*-regular if every vertex in this graph has degree *n*.

For example,

(1) k_n **is a (*n*-1)-regular.**

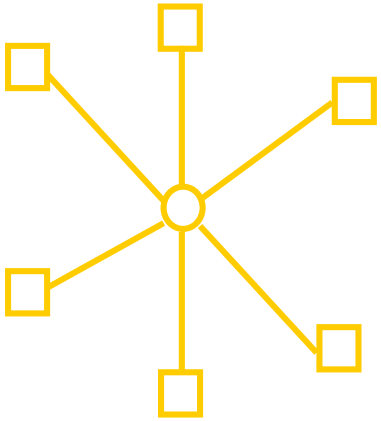
(2) For which values of *m* and *n* is $K_{m,n}$ regular?

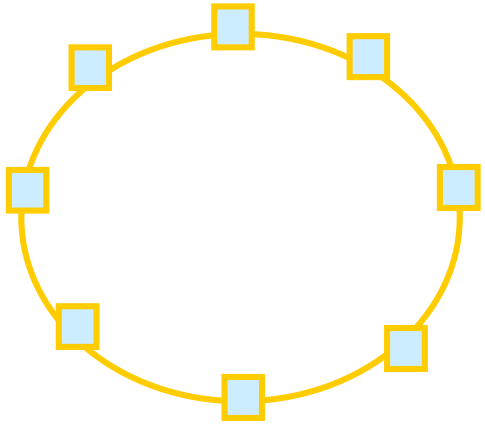


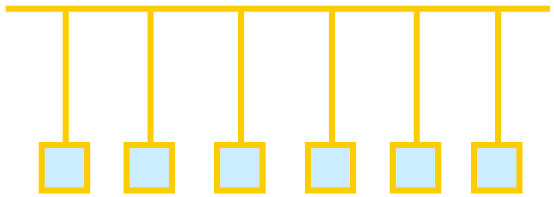
4. Some applications of special types of graphs

〖Example 3〗 Local Area Networks.

1. Star topology
2. Ring topology
3. Bus topology





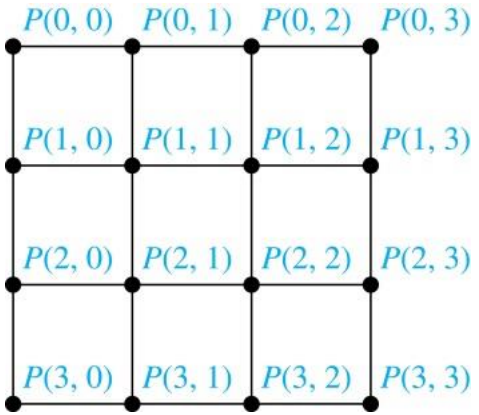


4. Some applications of special types of graphs

〖Example 4〗 Parallel processing .

The processors need to be interconnected as one processor may need the output generated by another.

- ✓ The *n-dimensional hypercube*, or *n-cube*, Q_n, is a common way to connect processors in parallel, e.g., Intel Hypercube.
- ✓ Another common method is the *mesh* network, illustrated here for 16 processors.



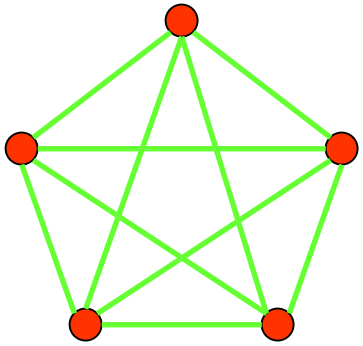


5. Some New Graphs From Old

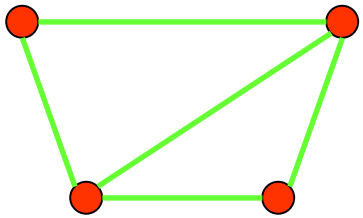
【Definition】 $G=(V, E)$, $H=(W, F)$

- H is a *subgraph* of G if $W \subseteq V, F \subseteq E$.
- subgraph H is a *proper subgraph* of G *if* $H \neq G$.
- H is a *spanning subgraph* of G if $W = V, F \subseteq E$.

For example,



K_5

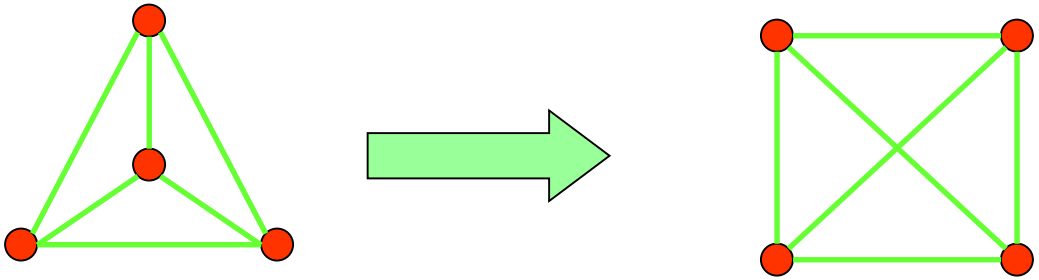


subgraph of K_5



〖Example 4〗 How many subgraphs with at least one vertex does W_3 have?

Solution:



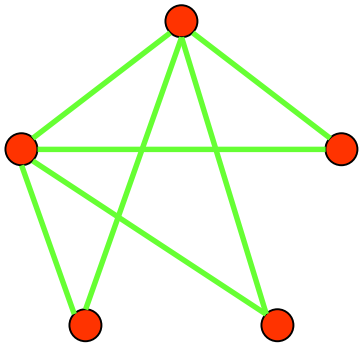
$$C(4,1)+C(4,2)\times 2+C(4,3)\times 2^3+C(4,4)\times 2^6$$

The union of G_1 and G_2

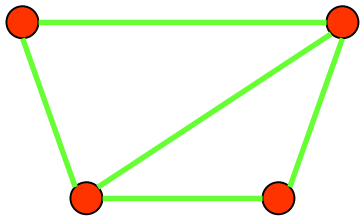
The *union* of two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is the simple graph with vertex set $V = V_1 \cup V_2$ and edge set $E = E_1 \cup E_2$.

Notation: $G_1 \cup G_2$

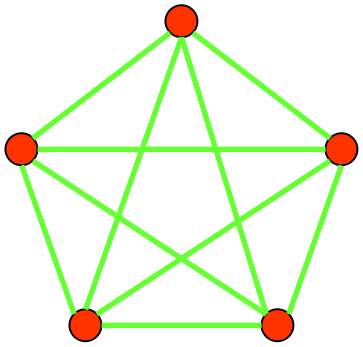
For example,



G_1



G_2



$G_1 \cup G_2 = K_5$

Homework:

P. 699: 5, 21-25, 30, 50-52, 55-57



CHAPTER 10 Graphs

10.1 Graphs and Graph Models

10.2 Graph Terminology and Special Types of Graphs

10.3 Representing Graphs and Graph Isomorphism

10.4 Connectivity

10.5 Euler and Hamilton Paths

10.6 Shortest Path Problems

10.7 Planar Graphs

10.8 Graph Coloring



1. Representing Graphs

Methods for representing graphs:

- **Graphs**
- **Adjacency lists**

  -- lists that specify all the vertices that are adjacent to each vertex

- **Adjacency matrice**
- **Incidence matrices**



2. Adjacency Matrices

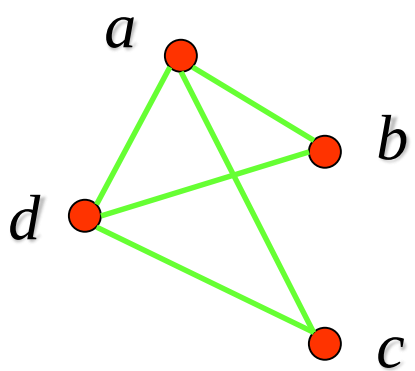
A simple graph $G = (V, E)$ with n vertices ($v_1, v_2, \cdots, v_n$) can be represented by its adjacency matrix, A, where

$a_{ij} = 1$ if $\{v_i, v_j\}$ is an edge of G,

$a_{ij} = 0$ otherwise.

Note: An adjacency matrix of a graph is based on the ordering chosen for the vertices.

〖Example 1〗 What i
following graph G l



Note: Adjacency matrices of undirected graphs are always symmetric.

$$A_G = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

◆ The adjacency matrix of a multigraph or pseudograph

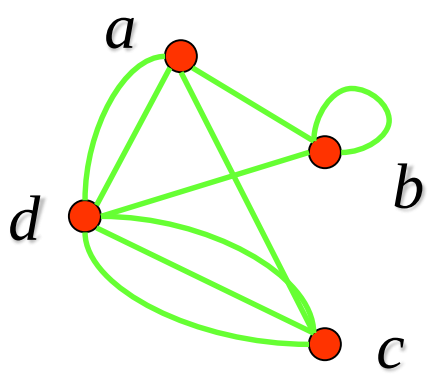
For the representation of graphs with multiple edges, we can no longer use zero-one matrices.

Instead, we use matrices of natural numbers.

The (i, j)th entry of such a matrix equals the number of edges that are associated to $\{v_i, v_j\}$.



〖Example 2〗 What is the adjacency matrix A_G for the following graph G based on the order of vertices a, b, c, d ?



Note: For undirected multigraph or pseudograph , adjacency matrices are symmetric.

Solution:

$$A_G = \begin{bmatrix} 0 & 1 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 3 \\ 2 & 1 & 3 & 0 \end{bmatrix}$$

◆ The adjacency matrix of a directed graph

Let $G = (V, E)$ be a directed graph with $|V| = n$. Suppose that the vertices of G are listed in arbitrary order as $v_1, v_2, \ldots, v_n$.

The adjacency matrix A (or A_G) of G, with respect to this listing of the vertices, is the $n \times n$ zero-one matrix with 1 as its (i, j)th entry when there is an edge from v_i to v_j, and 0 otherwise.

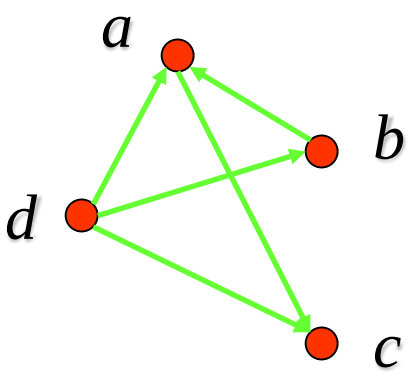
In other words, for an adjacency matrix $A = [a_{ij}]$,

$a_{ij} = 1$ if (v_i, v_j) is an edge of G,

$a_{ij} = 0$ otherwise.



〖Example 3〗 What is the adjacency matrix A_G for the following graph G based on the order of vertices a, b, c, d ?

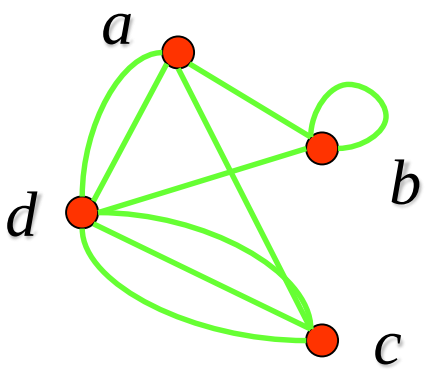


Solution:

$$A_G = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

Question:

1. **What is the sum of the entries in a row of the adjacency matrix for an undirected graph?**



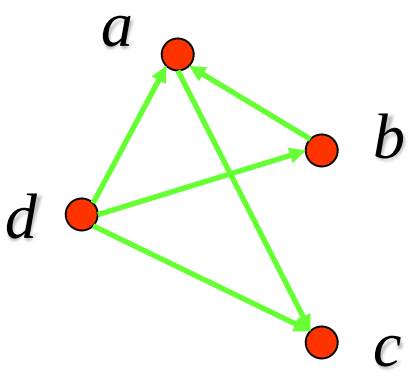
$$A_G = \begin{bmatrix} 0 & 1 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 3 \\ 2 & 1 & 3 & 0 \end{bmatrix}$$

Question:

1. **What is the sum of the entries in a row of the adjacency matrix for an undirected graph?**

The number of edges incident to the vertex *i*, which is the same as degree of *i* minus the number of loops at *i*.

For a directed graph?



$$A_G = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

Question:

1. **What is the sum of the entries in a row of the adjacency matrix for an undirected graph?**

   The number of edges incident to the vertex *i*, which is the same as degree of *i* minus the number of loops at *i*.

   For a directed graph?

   $\deg^+(v_i)$

2. **What is the sum of the entries in a column of the adjacency matrix for an undirected graph?**

   The number of edges incident to the vertex *i*, which is the same as degree of *i* minus the number of loops at *i*.

   For a directed graph?

   $\deg^-(v_i)$



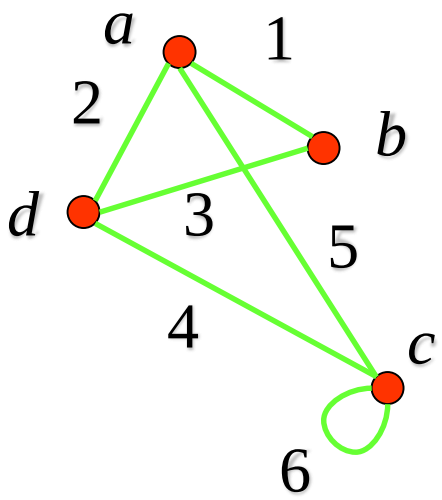
3. Incidence matrices

Let $G = (V, E)$ be an undirected graph. Suppose that $v_1, v_2, \cdots, v_n$ are the vertices and $e_1, e_2, \cdots, e_m$ are the edges of G. Then the *incidence matrix* with respect to this ordering of V and E is $n \times m$ matrix $\mathbf{M} = [m_{ij}]_{n \times m}$, where

$$m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i \\ 0 & \text{otherwise} \end{cases}$$



〖Example 4〗 What is the incidence matrix M for the following graph G based on the order of vertices a, b, c, d and edges 1, 2, 3, 4, 5, 6?



Solution:

$$M = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Note:

Incidence matrices of undirected graphs contain two 1s per column for edges connecting two vertices and one 1 per column for loops.

4. Isomorphism Of Graphs

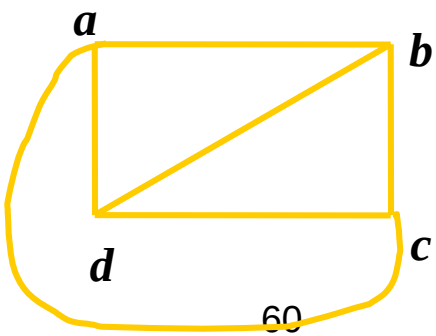
Graphs with the same structure are said to be *isomorphic*.

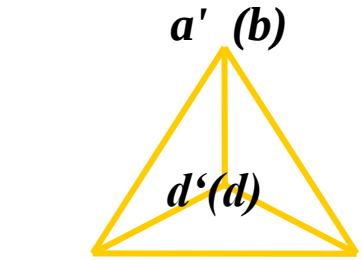
Formally, two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are isomorphic if there is a 1-1 and onto function f from V_1 to V_2 such that for all a and b in V_1, a and b are adjacent in G_1 iff $f(a)$ and $f(b)$ are adjacent in G_2.

Such a function f is called an *isomorphism*.

In other words, when two simple graphs are isomorphic, there is a one-to-one correspondence between vertices of the two graphs that preserves the adjacency relationship.

For example,







Question:

How to determine whether two simple graphs are isomorphic?

It is usually difficult since there are *n*! possible 1-1 correspondence between the two vertex sets with *n* vertices. However, some properties (called *invariants*) in the graphs may be used *to show that they are not isomorphic*.

invariants

-- things that G_1 and G_2 must have in common to be isomorphic.

- ✓ **However, there are algorithms with linear average-case time complexity.**
- ✓ **You can use a public domain program called NAUTY and TRACES to determine in less than a second whether two graphs with as many as 100 vertices are isomoprhic.**



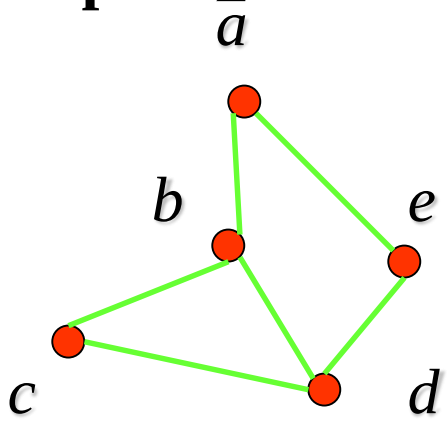
Important *invariants in isomorphic graphs*:

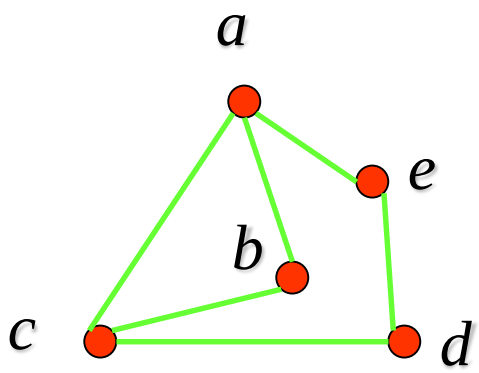
- the number of vertices
- the number of edges
- the degrees of corresponding vertices
- if one is bipartite, the other must be
- if one is complete, the other must be
- if one is a wheel, the other must be

etc.



〖Example 5〗 Are the following two graphs isomorphic?





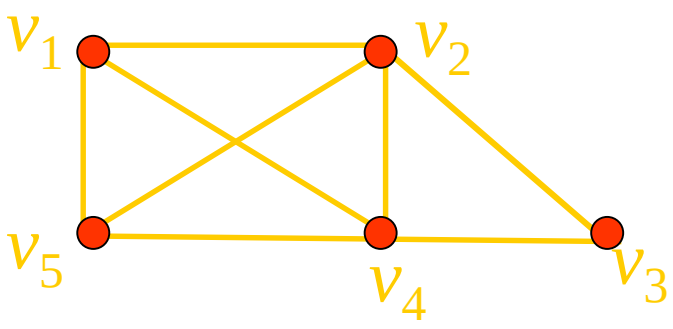
Solution:

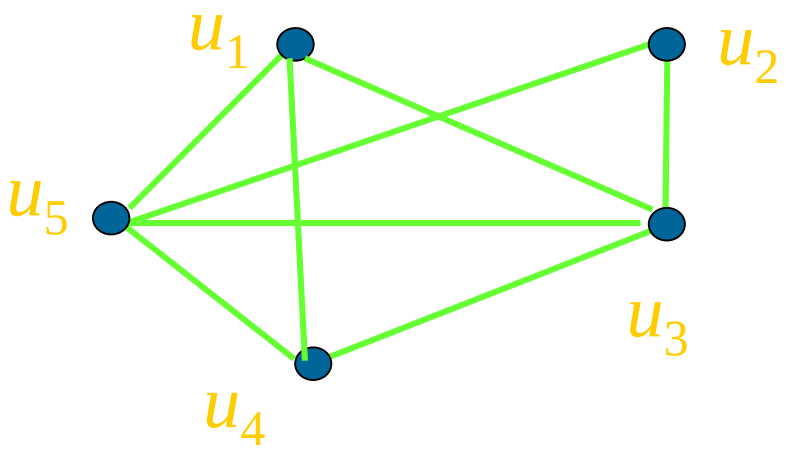
They are isomorphic, because they can be arranged to look identical.

You can see this if in the right graph you move vertex *b* to the left of the edge {*a*, *c*}. Then the isomorphism *f* from the left to the right graph is:

$f(a) = e,\ f(b) = a,$
$f(c) = b,\ f(d) = c,\ f(e) = d.$

〖Example 6〗 Show that the following two graphs are isomorphic.



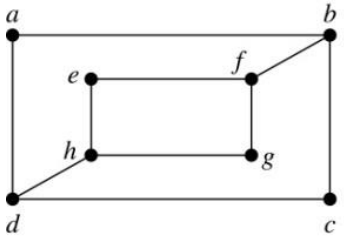


Proof:

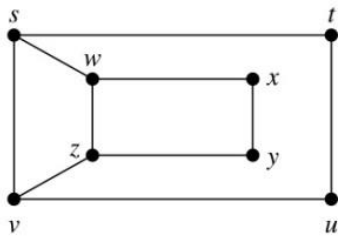
- Try to find an isomorphism *f*
- Show that *f* preserves adjacency relation

- The adjacency matrix of a graph *G* is the same as the adjacency matrix of another graph *H*, when rows and columns are labeled to correspond to the images under *f* of the vertices in *G*

Example: Determine whether these two graphs are isomorphic.



G

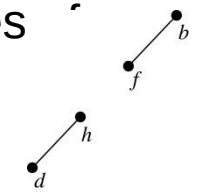


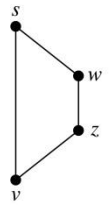
H

Solution: Both graphs have eight vertices and ten edges.
They also both have four vertices of degree two and four of degree three.

However, *G* and *H* are not isomorphic. Note that since $deg(a) = 2$ in *G*, *a* must correspond to *t*, *u*, *x*, or *y* in H, because these are the vertices of degree 2. But each of these vertices is adjacent to another vertex of degree two in *H*, which is not true for *a* in *G*.

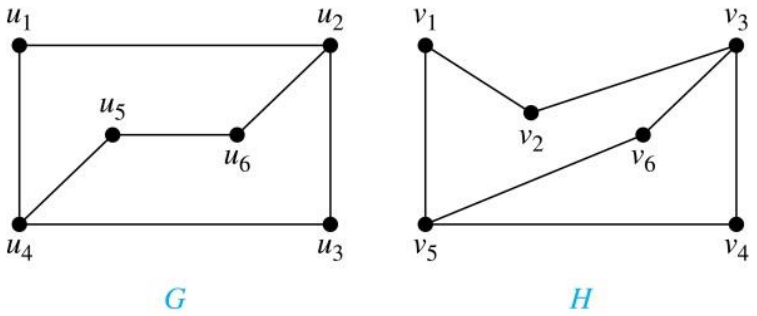
Alternatively, note that the subgraphs of *G* and *H* made up of vertices degree three and the edges connecting them must be isomorphic. But the subgraphs, as shown at the right, are not isomorphic.







Example: Determine whether these two graphs are isomorphic.



Solution: Both graphs have six vertices and seven edges.
They also both have four vertices of degree two and two of degree three.
The subgraphs of *G* and *H* consisting of all the vertices of degree two and the edges connecting them are isomorphic. So, it is reasonable to try to find an isomorphism *f*.

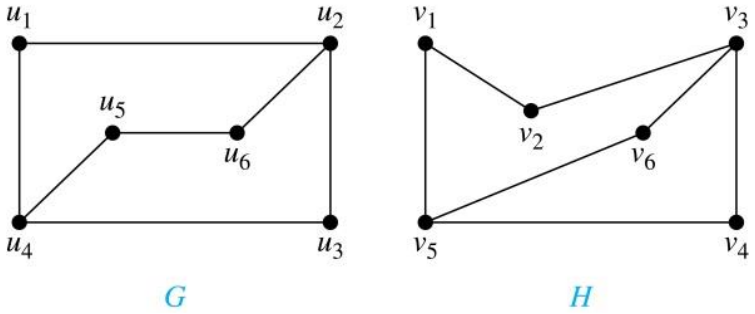
We define an injection *f* from the vertices of *G* to the vertices of *H* that preserves the degree of vertices. We will determine whether it is an isomorphism.

The function *f* with $f(u_1) = v_6$, $f(u_2) = v_3$, $f(u_3) = v_4$, and $f(u_4) = v_5$, $f(u_5) = v_1$, and $f(u_6) = v_2$ is a one-to-one correspondence between *G* and *H*. Showing that this correspondence preserves edges is straightforward, so we will omit the details here. Because *f* is an isomorphism, it follows that *G* and *H* are isomorphic graphs.

See the text for an illustration of how adjacency matrices can be used for this verification.



$$\mathbf{A}_G = \begin{array}{c} \\ u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \end{array} \begin{array}{c} \begin{array}{cccccc} u_1 & u_2 & u_3 & u_4 & u_5 & u_6 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix} \end{array},$$



G H

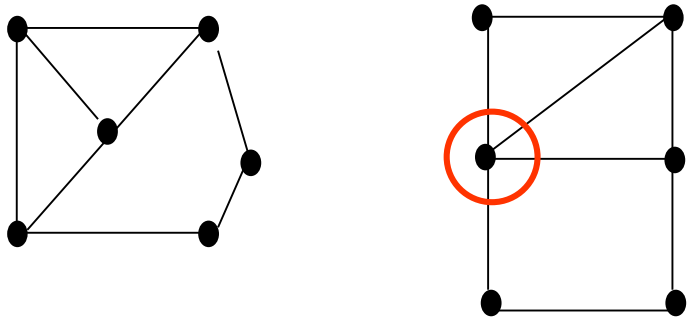
and the adjacency matrix of H with the rows and columns labeled by the images of the corresponding vertices in G,

$$\mathbf{A}_H = \begin{array}{c} \\ v_6 \\ v_3 \\ v_4 \\ v_5 \\ v_1 \\ v_2 \end{array} \begin{array}{c} \begin{array}{cccccc} v_6 & v_3 & v_4 & v_5 & v_1 & v_2 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix} \end{array}.$$



〖Example 7〗 Determine whether the given pair of graphs is isomorphic?

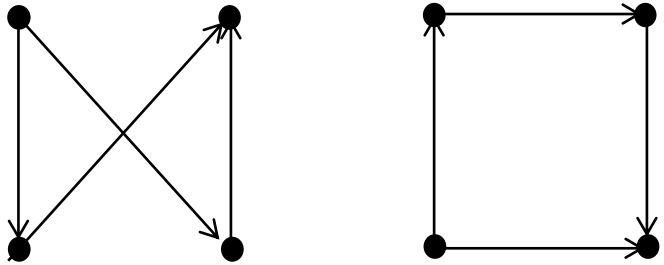
(1)



(2)



(3)



Applications of Graph Isomorphism

- The question whether graphs are isomorphic plays an important role in applications of graph theory. For example,
  - chemists use molecular graphs to model chemical compounds. Vertices represent atoms and edges represent chemical bonds. When a new compound is synthesized, a database of molecular graphs is checked to determine whether the graph representing the new compound is isomorphic to the graph of a compound that this already known.
  - Electronic circuits are modeled as graphs in which the vertices represent components and the edges represent connections between them. Graph isomorphism is the basis for
    - the verification that a particular layout of a circuit corresponds to the design's original schematics.
    - determining whether a chip from one vendor includes the intellectual property of another vendor.



Homework:

P. 710: 9, 18, 21, 32, 33, 45-48, 67-70, 74



CHAPTER 10 Graphs

10.1 Graphs and Graph Models

10.2 Graph Terminology and Special Types of Graphs

10.3 Representing Graphs and Graph Isomorphism

10.4 Connectivity

10.5 Euler and Hamilton Paths

10.6 Shortest Path Problems

10.7 Planar Graphs

10.8 Graph Coloring



1. Paths

A *path of length n* in an undirected graph is a sequence of vertices $v_0, v_1, \ldots, v_n$ such that $\{v_0, v_1\}, \{v_1, v_2\}, \ldots, \{v_{n-1}, v_n\}$ are n edges in the graph.

The path is a *circuit* if the path begins and ends with the same vertex.

A path is *simple* if it does not contain the same edge more than once.

Note:

1. There is nothing to prevent traversing an edge back and forth to produce arbitrarily long paths. This is usually not interesting which is why we define a simple path.
2. The notation of a path: vertex sequence
3. A path of length zero consists of a single vertex.



Path in directed graph

A *path of length n* in a directed graph is a sequence of vertices $v_0, v_1, \ldots v_n$ such that (v_0, v_1) , (v_1, v_2) , ..., (v_{n-1}, v_n) are n directed edges in the graph.

Circuit or cycle : **the path begins and ends with the same vertex.**

Simple path: **the path does not contain the same edge more than once.**



Paths represent useful information in many graph models.

〖Example 1〗Path in Acquaintanceship Graphs

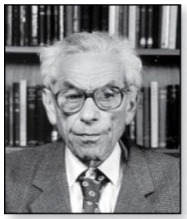
In an acquaintanceship graph there is a path between two people if there is a chain of people linking these people, where two people adjacent in the chain know one other.

Many social scientists have conjectured that almost every pair of people in the world are linked by a small chain of people, perhaps containing just five or fewer people.

The play *Six Degrees of Separation* by John Guare is based on this notion.



Erdős numbers



Example: *Erdős numbers*.
In a collaboration graph, two people *a* and *b* are connected by a path when there is a sequence of people starting with *a* and ending with *b* such that the endpoints of each edge in the path are people who have collaborated.

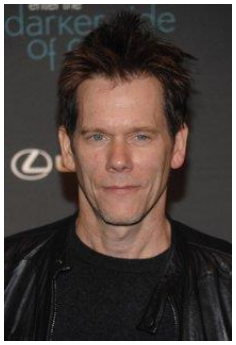
- In the academic collaboration graph of people who have written papers in mathematics, the *Erdős number* of a person *m* is the length of the shortest path between *m* and the prolific mathematician Paul Erdős.

TABLE 1 The Number of Mathematicians with a Given Erdős Number (as of early 2006).

Erdős Number	*Number of People*
0	1
1	504
2	6,593
3	33,605
4	83,642
5	87,760
6	40,014
7	11,591
8	3,146
9	819
10	244
11	68
12	23
13	5



Bacon Numbrers



- In the Hollywood graph, two actors *a* and *b* are linked when there is a chain of actors linking *a* and *b*, where every two actors adjacent in the chain have acted in the same movie.
- The *Bacon number* of an actor *c* is defined to be the length of the shortest path connecting *c* and the well-known actor Kevin Bacon. (Note that we can define a similar number by replacing Kevin Bacon by a different actor.)

TABLE 2 The Number of Actors with a Given Bacon Number (as of early 2011).

Bacon Number	*Number of People*
0	1
1	2,367
2	242,407
3	785,389
4	200,602
5	14,048
6	1,277
7	114
8	16

2. Counting paths between vertices

The number of paths between two vertices in a graph can be determined using its adjacency matrix.



【Theorem 2】 The *number of different paths of length r* from v_i to v_j is equal to the (i, j)th entry of A^r, where A is the adjacency matrix representing the graph consisting of vertices $v_1, v_2, \ldots v_n$.

Note: This is the standard power of A, not the Boolean product.

Proof:

Let $A = (a_{ij})_{n\times n}$

(1) A

$a_{ij} = 1$

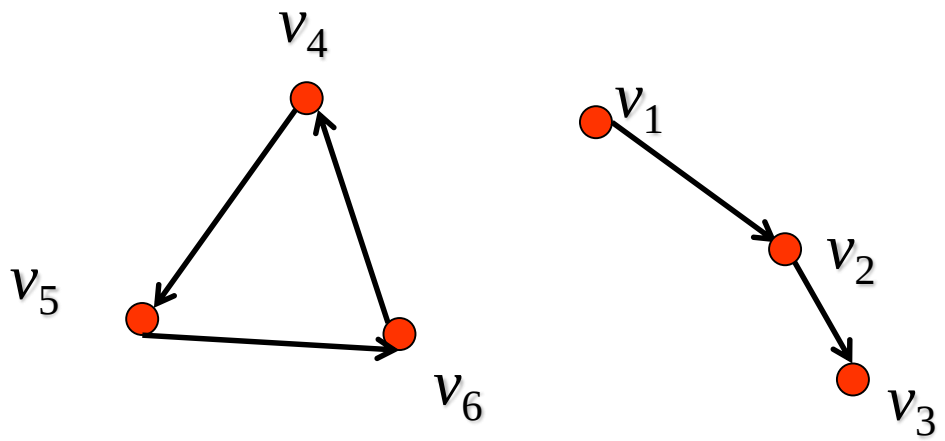
$a_{ij} = 0$

(3) $A^r = (c_{ij})_{n\times n}$

$$A^{r+1} = A^r \cdot A = (d_{ij})_{n\times n}$$

$$d_{ij} = c_{i1}a_{1j} + c_{i2}a_{2j} + \cdots + c_{in}a_{nj} = \sum_{k=1}^{n} c_{ik}a_{kj}$$

〖Example 2〗



(1) How many paths of length 2 are there from v_5 to v_4?

a_{54} in A^2; 1

(2) The number of paths not exceeding 6 are there from v_4 to v_5?

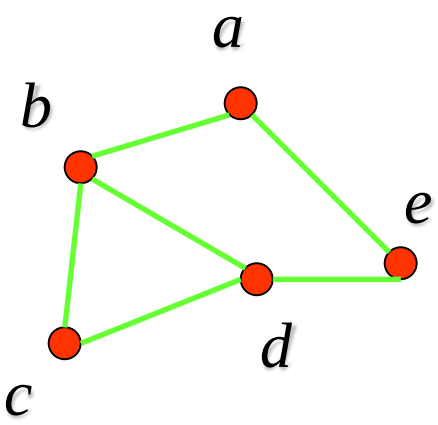
a_{45} in $A+A^2+A^3+A^4+A^5+A^6$; 2

(3) The number of circuits starting at vertex v_5 whose length is not exceeding 6?

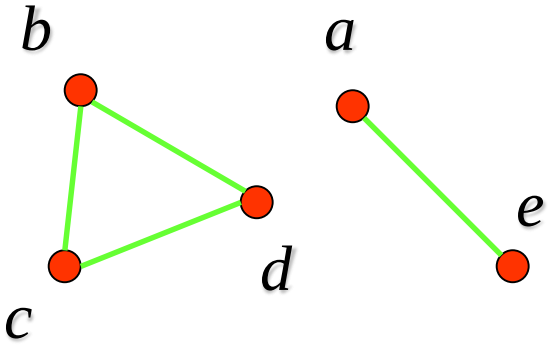
3. Connectedness in undirected graphs

An undirected graph is called *connected* if there is a path between every pair of distinct vertices of the graph.

〖Example 3〗 Are the following graphs connected?



Yes.



No.

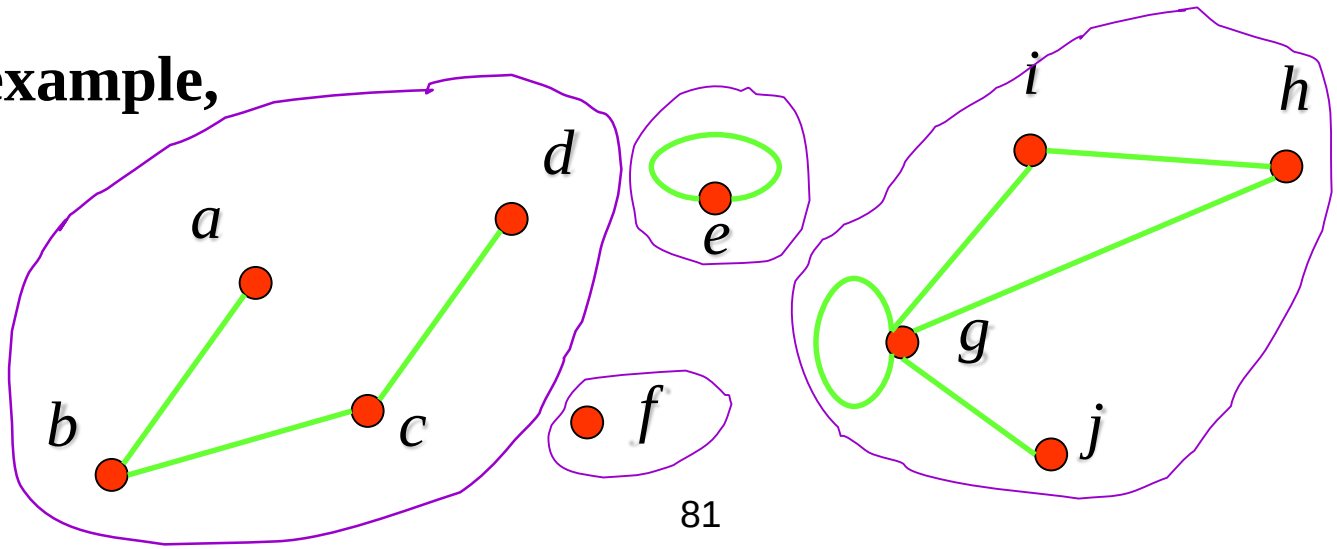
【Theorem 1】 There is a simple path between every pair of distinct vertices of a connected undirected graph.

Proof:

Because the graph is connected there is a path between *u* and *v*. Throw out all redundant circuits to make the path simple.

The maximally connected subgraphs of *G* are called the *connected components* or just the *components*.

For example,

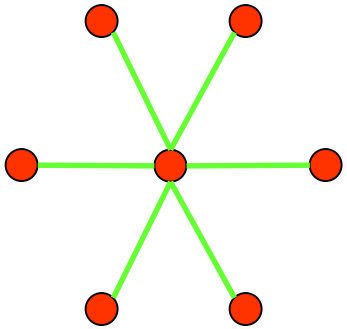


A vertex is a *cut vertex* (or *articulation point*), if removing it and all edges incident with it results in more connected components than in the original graph.

Similarly if removal of an edge creates more components the edge is called a *cut edge* or *bridge*.

For example,

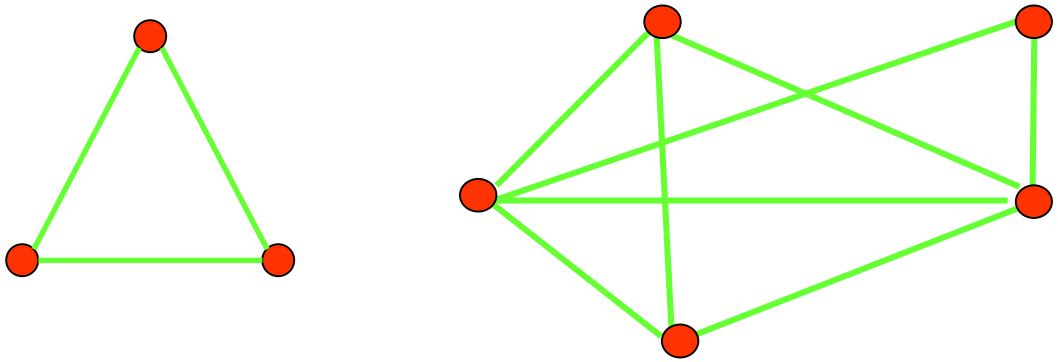
（1）



In the star network the center vertex is a cut vertex.

All edges are cut edges.

（2）



There are no cut edges or vertices in the graph *G* above.

Removal of any vertex or edge does not create additional components.

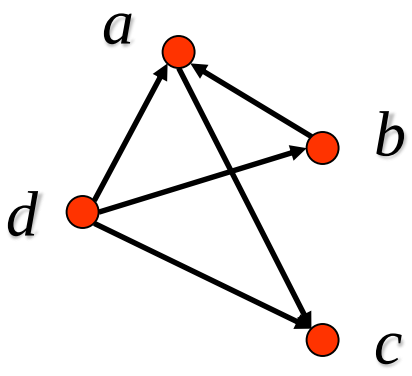
4. Connectedness in directed graphs

A directed graph is *strongly connected* if there is a path from *a* to *b* and from *b* to *a* for all vertices *a* and *b* in the graph.

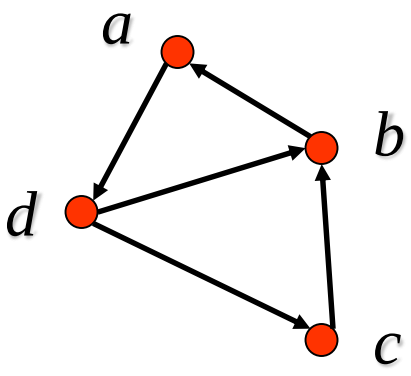
The graph is *weakly connected* if the underlying undirected graph is connected.

Note: By the definition, any strongly connected directed graph is also weakly connected.

For example,



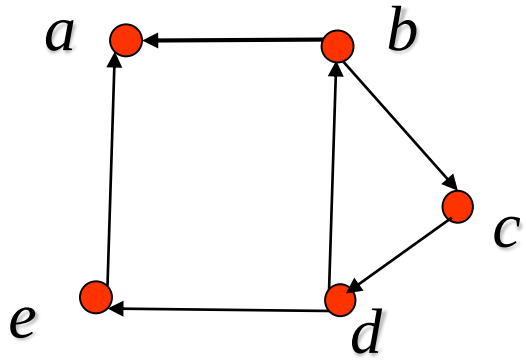
Weakly connected



Strongly connected

For directed graph, the maximal strongly connected subgraphs are called the *strongly connected components* or just the *strong components*.

For example,



Problem:

1. **How to determine whether a given directed graph is strongly connected or weakly connected ?**
2. **How to find the strongly connected components in a directed graph ?**

The Connected Components of the Web Graph

- Recall that at any particular instant the web graph provides a snapshot of the web, where vertices represent web pages and edges represent links. According to a 1999 study, the Web graph at that time had over 200 million vertices and over 1.5 billion edges. (The numbers today are several orders of magnitude larger.)
- The underlying undirected graph of this Web graph has a connected component that includes approximately 90% of the vertices.
- There is a *giant strongly connected component (GSCC)* consisting of more than 53 million vertices. A Web page in this component can be reached by following links starting in any other page of the component. There are three other categories of pages with each having about 44 million vertices:
  - pages that can be reached from a page in the GSCC, but do not link back.
  - pages that link back to the GSCC, but can not be reached by following links from pages in the GSCC.
  - pages that cannot reach pages in the GSCC and can not be reached from pages in the GSCC.



5. Paths and Isomorphism

Idea:

(1) **Some other invariants**

- **The number and size of connected components**
- **Path**
- ✓ **Two graphs are isomorphic only if they have simple circuits of the same length.**
- ✓ **Two graphs are isomorphic only if they contain paths that go through vertices so that the corresponding vertices in the two graphs have the same degree.**

(2) **We can also use paths to find mapping that are potential isomorphisms.**



〖Example 4〗 Are these two graphs isomorphic?



Solution:

No.

Because the right graph contains circuits of length 3, while the left graph does not.

Homework:

P. 724: 11, 12, 15, 20, 26-28, 31, 45, 53



CHAPTER 10 Graphs

10.1 Graphs and Graph Models

10.2 Graph Terminology and Special Types of Graphs

10.3 Representing Graphs and Graph Isomorphism

10.4 Connectivity

10.5 Euler and Hamilton Paths

10.6 Shortest Path Problems

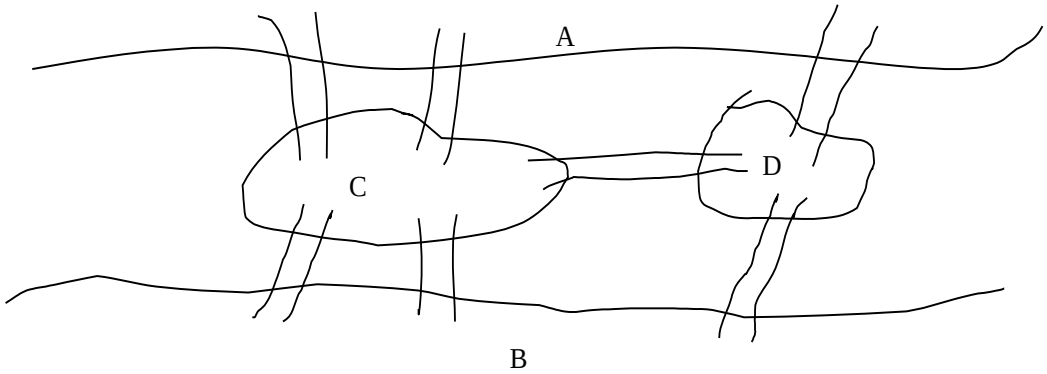
10.7 Planar Graphs

10.8 Graph Coloring

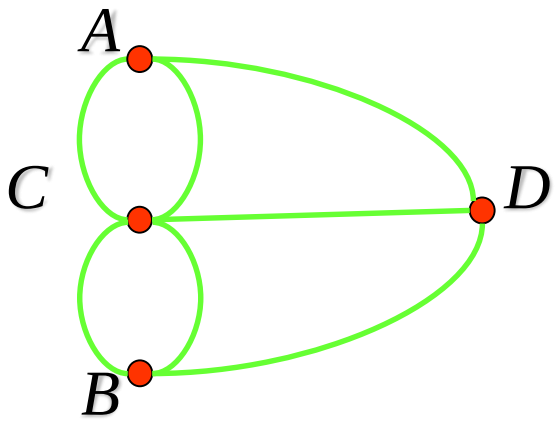


10.5.1 Euler Paths

1. Konigsberg Seven Bridge Problem



Problem: Whether it was possible to start at some location in the town, travel across all the bridges without crossing any bridge twice, and return the starting point.



The question becomes: Is there a simple circuit in this multigraph that contains every edge?

Terminologies:

- ***Euler Path***

  An Euler path is a simple path containing every edge of *G*.

- ***Euler Circuit***

  An Euler circuit is a simple circuit containing every edge of *G*.

- ***Euler Graph***

  A graph contains an Euler circuit.



2. Necessary and sufficient condition for Euler circuit and paths

【Theorem 1】 A connected multigraph has an Euler circuit if and only if each of its vertices has even degree.

Proof:

(1) Necessary condition

G has an Euler circuit ⇒ Every vertex in *V* has even degree

Consider the Euler circuit.

- the vertex *a* which the Euler circuit begins with
- the other vertex



(2) sufficient condition

We will form a simple circuit that begins at an arbitrary vertex *a* of *G*.

- **Build a simple circuit $x_0=a, x_1, x_2,\ldots,x_n=a$.**
- **An Euler circuit has been constructed if all the edges have been used. otherwise,**
- **Consider the subgraph *H* obtained from *G* with the edges of this circuit removed.**

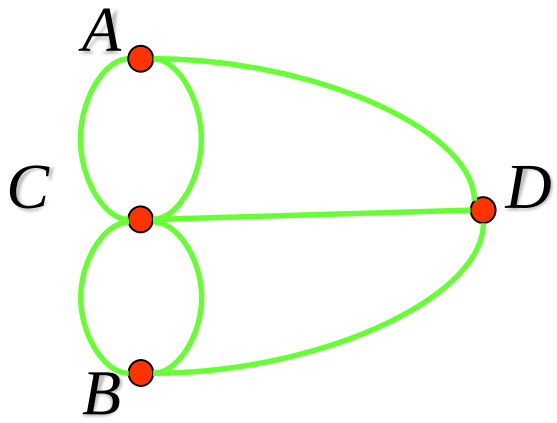
  Let *w* be a vertex which is the common vertex of the circuit and *H*.

  Beginning at *w*, construct a simple path in *H*.
- **Insert a new circuit in *H* to the old circuit at the vertex *w*.**



【 Theorem 2】 A connected multigraph has an Euler path but not an Euler circuit if and only if it has exactly two vertices of odd degree.

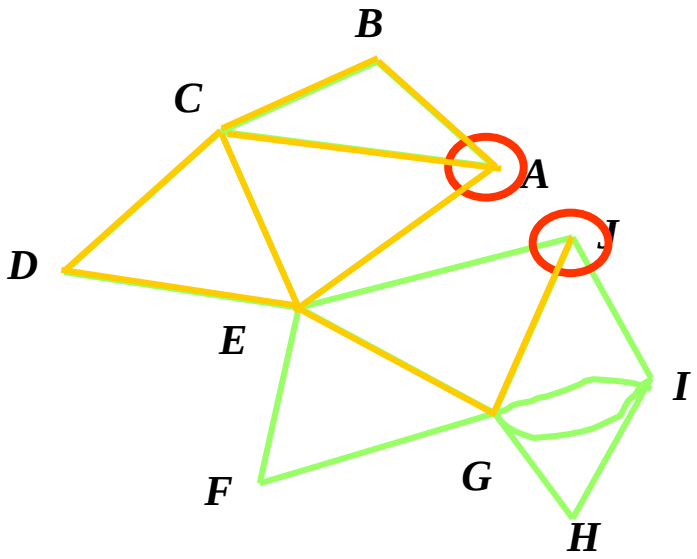
〖Example 1〗 Konigsberg Seven Bridge Problem



1. The graph has four vertices of odd degree. Therefore, it does not have an Euler circuit.

2. It does not have an Euler path either.

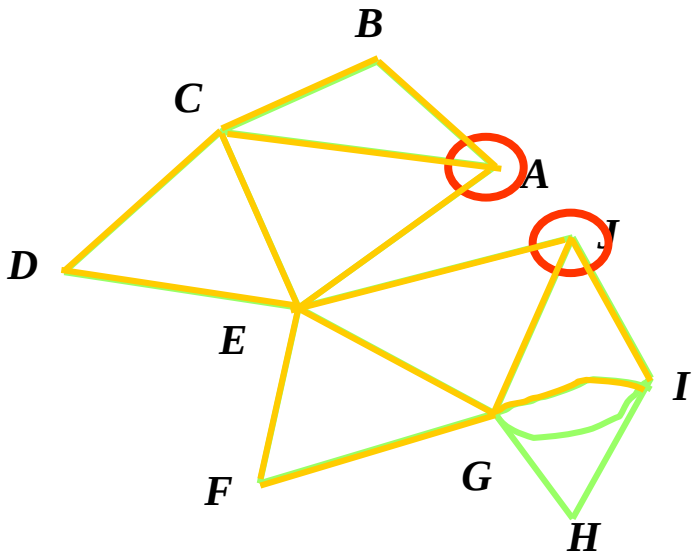
〖Example 2〗 Determine whether the following graph has an Euler path. Construct such a path if it exists.



Solution:

The graph has 2 vertices of odd degree, and all of other vertices have even degree . Therefore, this graph has an Euler path.

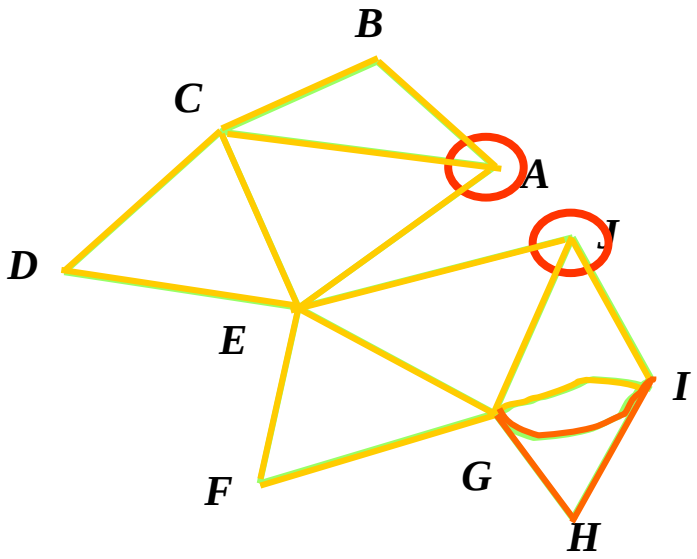
〖Example 2〗 Determine whether the following graph has an Euler path. Construct such a path if it exists.



Solution:

The graph has 2 vertices of odd degree, and all of other vertices have even degree . Therefore, this graph has an Euler path.

〖Example 2〗 Determine whether the following graph has an Euler path. Construct such a path if it exists.



The Euler path:

A,C,E,F,G,I,J,E,A,B, C,D,E,G, H,I,G,J

Solution:

The graph has 2 vertices of odd degree, and all of other vertices have even degree . Therefore, this graph has an Euler path.

3. Euler circuit and paths in directed graphs

A directed multigraph having no isolated vertices has an Euler circuit if and only if

-- the graph is weakly connected

-- the in-degree and out-degree of each vertex are equal.

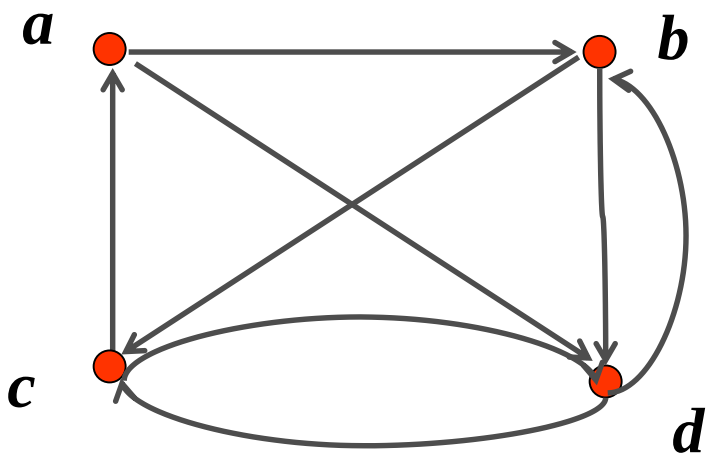
A directed multigraph having no isolated vertices has an Euler path but not an Euler circuit if and only if

-- the graph is weakly connected

-- the in-degree and out-degree of each vertex are equal for all but two vertices, one that has in-degree 1 larger than its out-degree and the other that has out-degree 1 larger than its in-degree.



〖Example 3〗 Determine whether the directed graph has an Euler path. Construct an Euler path if it exists.

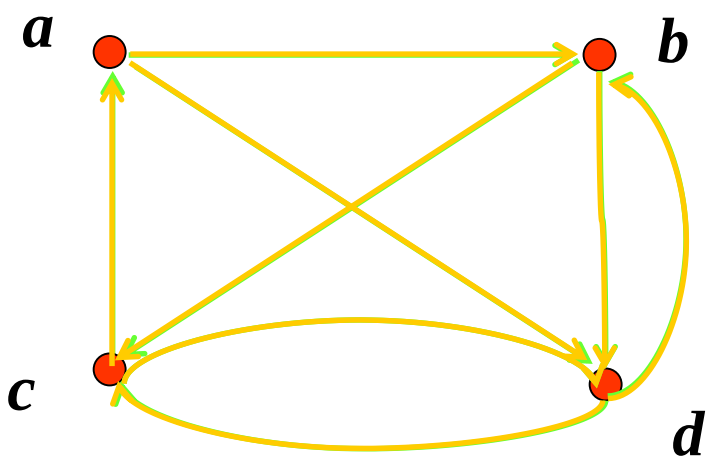


Solution:

	$\deg^-(v)$	$\deg^+(v)$
a	1	2
b	2	2
c	2	2
d	3	2

Hence, the directed graph has an Euler path.

〖Example 3〗 Determine whether the directed graph has an Euler circuit. Construct an Euler circuit if it exists.



Solution:

	$\deg^-(v)$	$\deg^+(v)$
a	1	2
b	2	2
c	2	2
d	3	2

Hence, the directed graph has an Euler path.

4. Application

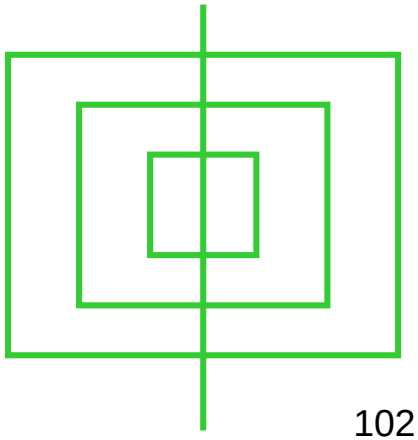
Euler path or circuit can be used to solve many practical problems.

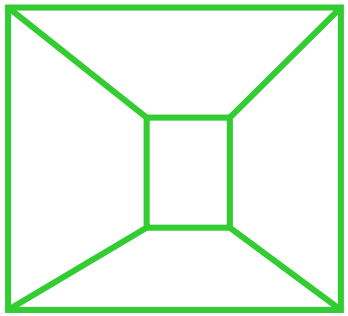
1) A type of puzzle

Draw a picture in a continuous motion without lifting a pencil so that no part of the picture is retraced.

The equivalent problem: Whether the graph exist an Euler path or circuit.

For example,





4. Application

Euler path or circuit can be used to solve many practical problems.

2) The Chinese postman problem

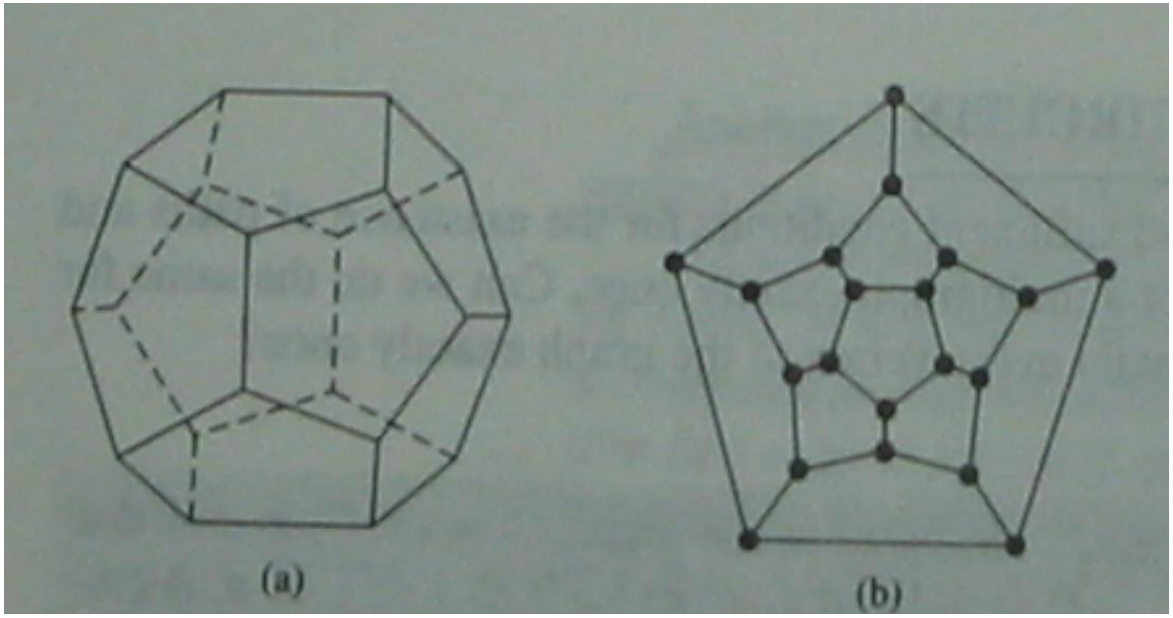
- The problem is named in honor of Guan Meigu, who posed it in 1962.

3) The other area, such as networking, molecular biology etc.



10.5.2 Hamilton paths and circuit

Hamilton's puzzle



The object of the puzzle was to start at a city and travel along the edges of the dodecahedron, visiting each of the other 19 cities exactly once, and end back at the first city.

The equivalent question: Is there a circuit in the graph shown in b that passes through each vertex exactly once?

A *Hamilton path* in a graph *G* is a path which visits every vertex in *G* exactly once.

A *Hamilton circuit* (or *Hamilton cycle*) is a cycle which visits every vertex exactly once, *except for the first vertex*, which is also visited at the end of the cycle.

If a connected graph *G* has a Hamilton circuit, then *G* is called a *Hamilton graph*.

Note:

The definition applies both to undirected as well as directed graphs of all types.

Question:

1) H path is a simple path?

2) H path and Euler path.



1. The sufficient condition for the existence of Hamilton path and Hamilton circuit

【Theorem 3】 DIRAC'THEOREM

If *G* is a simple graph with *n* vertices with *n*>=3 such that the degree of every vertex in *G* is at least *n/2*, then *G* has a Hamilton circuit.

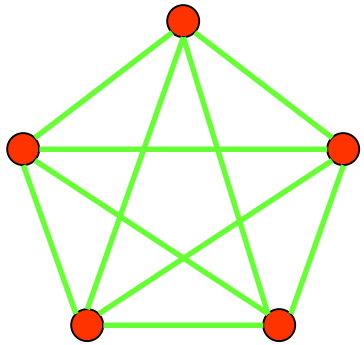
【Theorem 4】ORE'THEOREM

If *G* is a simple graph with *n* vertices with *n*>=3 such that deg(*u*)+deg(*v*) >= *n* for every pair of nonadjacent vertices *u* and *v* in *G* , then *G* has a Hamilton circuit.



〖Example 4〗 Show that K_n has a Hamilton circuit whenever n>=3 ?

For example,



Proof:

We can form a Hamilton circuit in K_n beginning at any vertex. Such a circuit can be built by visiting vertices in any order we choose, as long as the path begins and ends at the same vertex and visits each other vertex exactly once. This is possible since there are edges in K_n between any two vertices.

2. The necessary condition for Hamilton path and Hamilton circuit

For undirected graph:

The necessary condition for the existence of Hamilton path:

- ***G* is connected;**
- **There are at most two vertices which degree are less than 2.**

The necessary condition for the existence of Hamilton circuit:

- **The degree of each vertex is larger than 1.**

Some properties:

- **If a vertex in the graph has degree two, then both edges that are incident with this vertex must be part of any Hamilton circuit.**
- **When a Hamilton circuit is being constructed and this circuit has passed through a vertex, then all remaining edges incident with this vertex, other than the two used in the circuit , can be removed from consideration.**



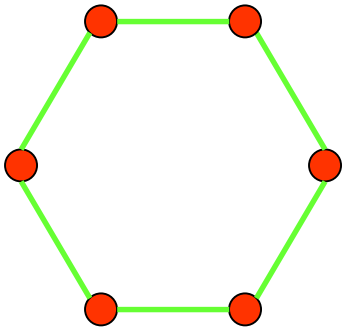
Another important necessary condition

- **For any nonempty subset *S* of set *V*, the number of connected components in *G*-*S* <=|*S*|.**

Note:

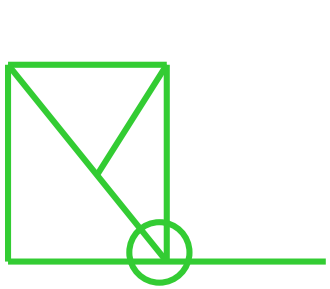
(1)* *G*-*S* is a subgraph of *G

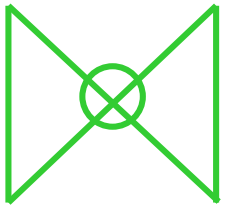
(2) Suppose that *C* is a H circuit of *G*. For any nonempty subset *S* of set *V*, the number of connected components in *C*-*S* <=|*S*|.

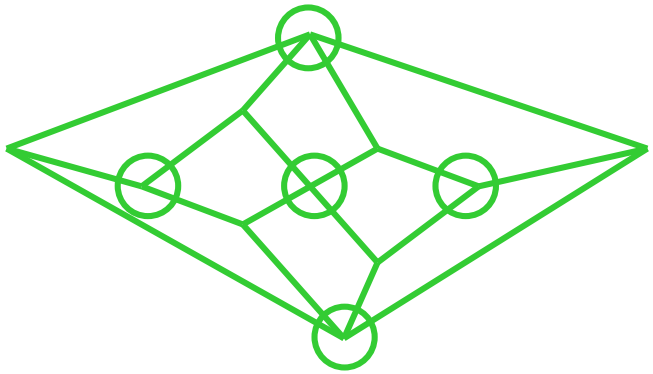


(3) the number of connected components in *G*-*S* <= the number of connected components in *C*-*S*

〖Example 5〗 Does the following graphs have a Hamilton circuit?







3. Application

Hamilton path or circuit can be used to solve many practical problems also.

For example,

1) Find a path or circuit that visits each road intersection in a city, or each node in a communication network exactly once.

2) The famous traveling salesman problem (TSP)

3) ……



〖Example 6〗 There are seven people denoted by A, B, C, D, E, F, G. Suppose that the following facts are known.

A--English (A can speak English.)

B--English, Chinese

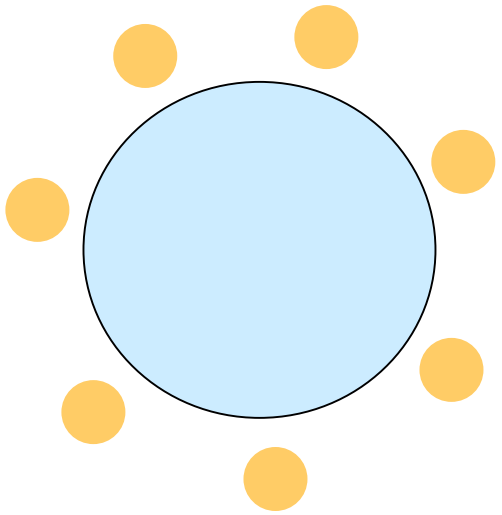
C--English, Italian, Russian

D--Japanese, Chinese

E--German, Italia

F--French, Japanese, Russian

G--French, German

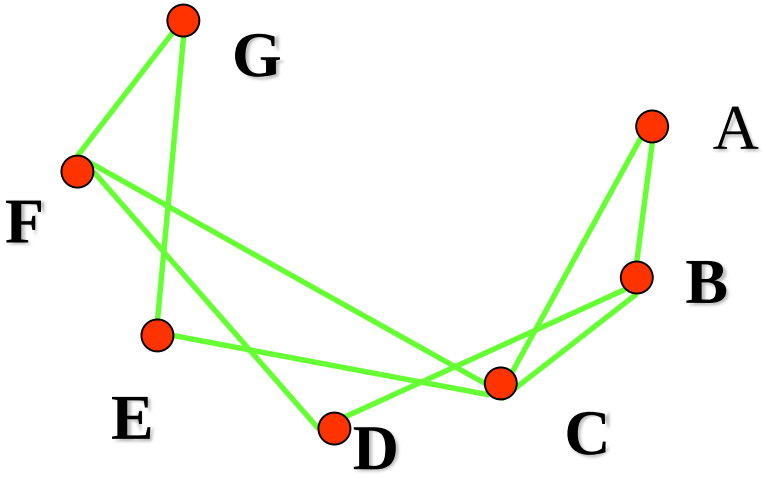


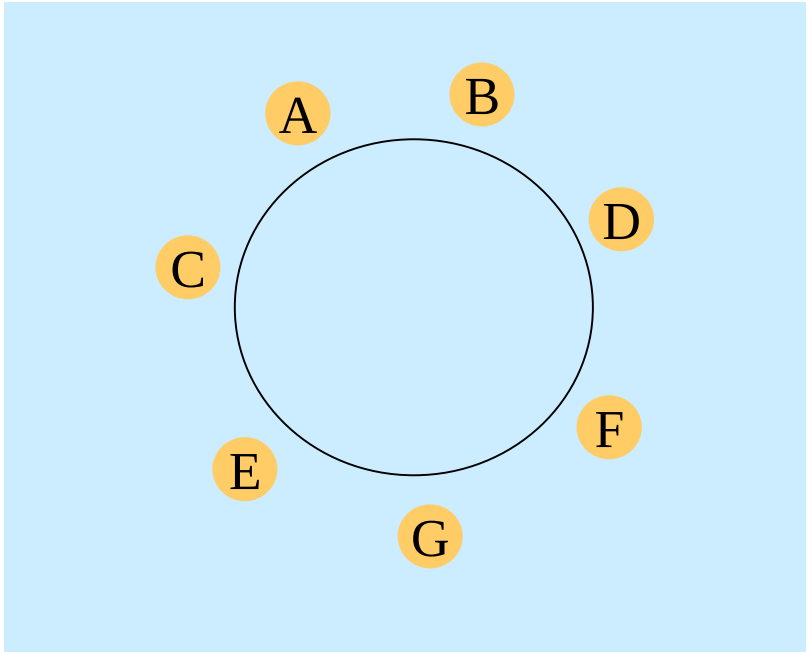
How to arrange seat for the round desk such that the seven people can talk each other?

Solution:

(1) Construct graph

V={A,B,C,D,E,F,G}, E={(u,v)|u,v can speak at least one common language.}





(2) If there is a H circuit, then we can arrange seat for the round desk such that the seven people can talk each other.

H circuit: A,B,D,F,G,E,C,A

〖Example 8〗 Gray codes (page 702)

A Gray code by listing all bit strings of length *n* in such a way that each string differs in exactly one position from the preceding bit string, and the last string differs from the first in exactly one position. We can model this problem using the *n*-cube *Qn*. What is needed to solve this problem is a Hamilton circuit in *Qn*.



Homework:

P. 739: 4, 6, 20, 22, 28, 36, 43, 45



CHAPTER 10 Graphs

10.1 Graphs and Graph Models

10.2 Graph Terminology and Special Types of Graphs

10.3 Representing Graphs and Graph Isomorphism

10.4 Connectivity

10.5 Euler and Hamilton Paths

10.6 Shortest Path Problems

10.7 Planar Graphs

10.8 Graph Coloring

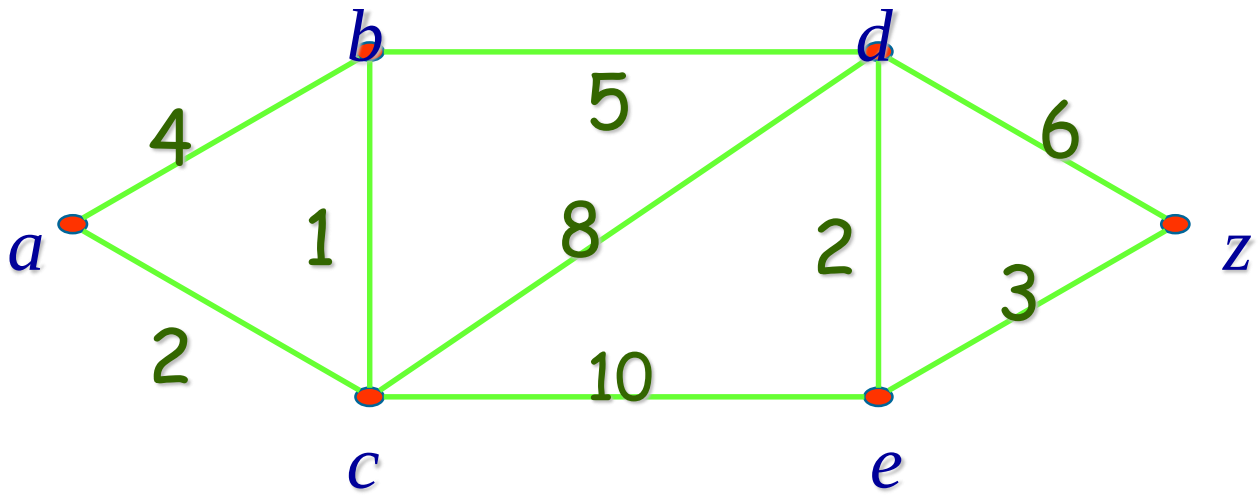


1. Introduction

- *Weighted graph* $G = (V,E,W)$

  We can assign weights to the edges of graphs.

For example,



1. Introduction

- ***Weighted graph*** $G = (V,E,W)$

  We can assign weights to the edges of graphs.

- ***The length of a path in a weighted graph***

  -- the sum of the weights of the edges of this path.

- ***Shortest Path Problems***

  $G = (V, E, W)$ **is a weighted graph, where** $w(x, y)$ **is the weight of edge associated vertices *x* and *y* (if** $(x, y) \notin E, w(x, y) = \infty$ **).** $a, z \in V$**, find the shortest path between *a* and *z*.**



2. A shortest path algorithm

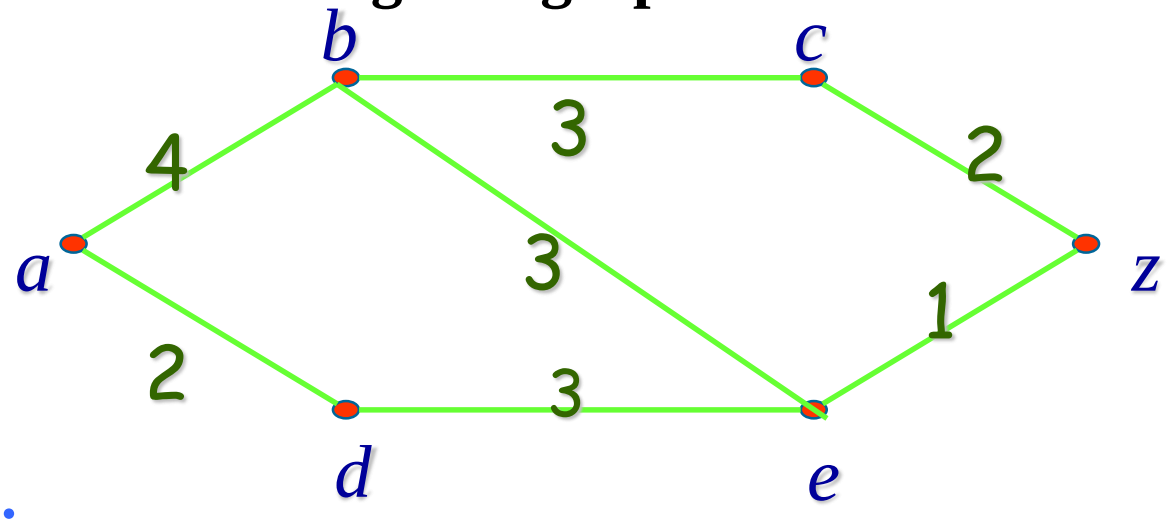
There are several different algorithms that find the shortest path between two vertices in a weighted graph.

Dijkstra's Algorithm

- An algorithm discovered by the Dutch mathematician E. Dijkstra in 1959.
- An iterative procedure.
- Proceed by finding the length of the shortest path from *a* to a first vertex, the length of the shortest path from *a* to a second vertex, and so on, until the length of the shortest path from *a* to *z*.



〖Example 1〗What is the length of the shortest path between *a* and *z* in the weighted graph.



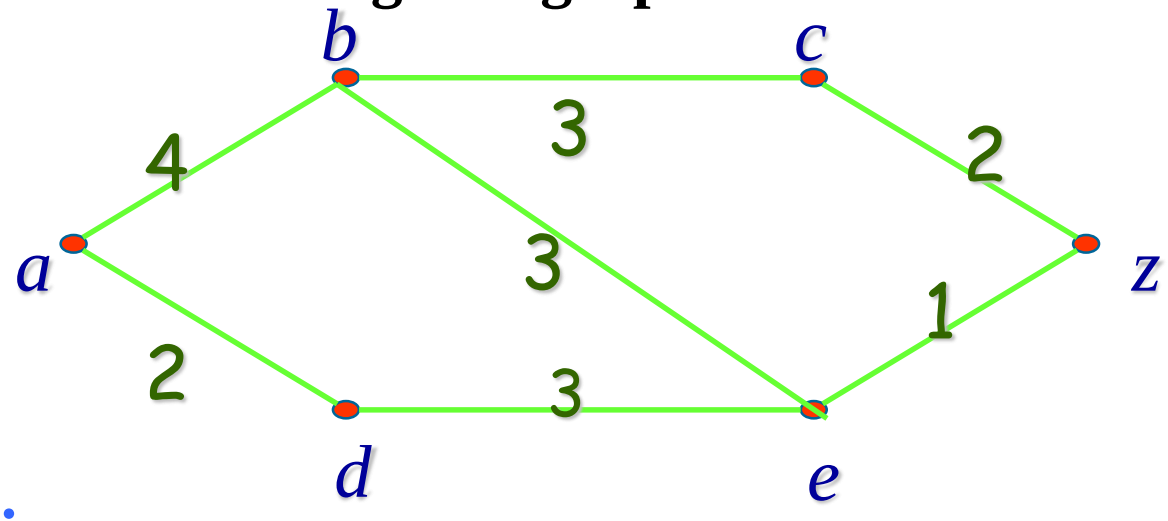
Solution:

We will solve this problem by finding the length of a shortest path from *a* to successive vertices, until *z* is reached.

1) The first closest vertex: *d*

The only paths starting at *a* that contain no vertex other than *a* are *a*, *b* and *a*, *d*.

〖Example 1〗What is the length of the shortest path between *a* and *z* in the weighted graph.



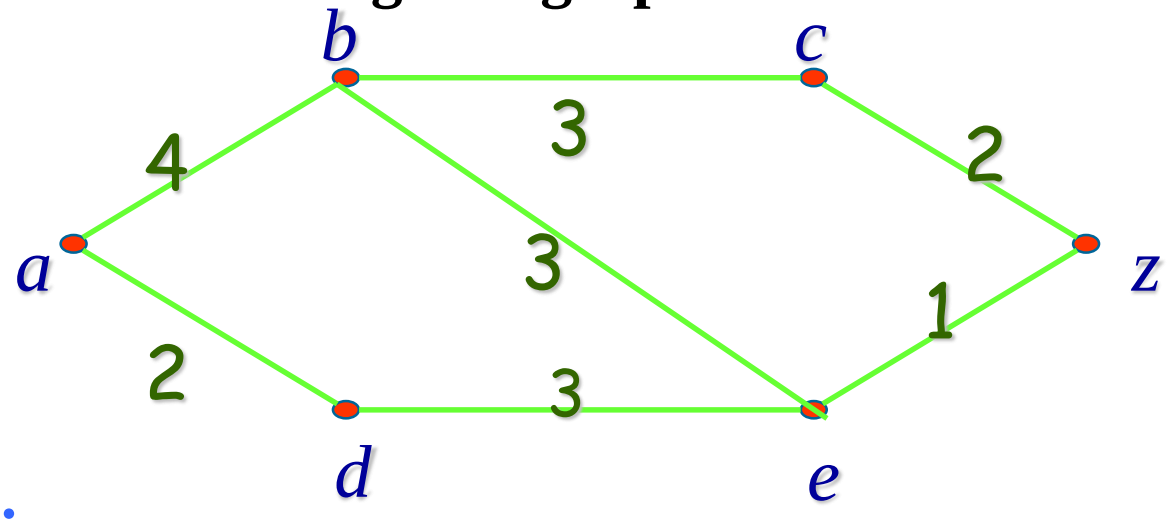
Solution:

1) The first closest vertex: *d*

2) The second closest vertex: *b*

Looking at all paths that go through only *a* and *d*.

〖Example 1〗What is the length of the shortest path between *a* and *z* in the weighted graph.



Solution:

1) The first closest vertex: *d*

2) The second closest vertex: *b*

3) The third closest vertex: *e*

Examine only paths go through only *a*, *d* and *b*.

4) The forth closest vertex: *z*

The details of Dijkstra's algorithm

Dijkstra's algorithm proceeds by forming a distinguished set of vertices. Let S_k denote this set of vertices after k iterations of labeling procedure.

Step1 Label a with 0 and other with ∞, i.e. $L_0(a)=0$, and $L_0(v)=\infty$ and $S_0=\phi$.

Step 2 The set S_k is formed from S_{k-1} by adding a vertex u not in S_{k-1} with the smallest label. Once u is added to S_k , we update the labels of all vertices not in S_k , so that $L_k(v)$, the label of the vertex v at the kth stage, is the length of the shortest path from a to v that containing vertices only in S_k .



Let v be a vertex not in S_k . To update the label of v, note that $L_k(v)$ is the shortest path from a to v containing only vertices in S_k ,

1. the shortest path from a to v containing only elements of S_{k-1}

   Or

2. it is the shortest path from a to u at the (k-1)st stage with the edge (u,v) added.

In other words,

$$L_k(v)=\min\{L_{k-1}(v), L_{k-1}(u)+w(u,v)\}$$



Algorithm 1 Dijkstra's Algorithm.

Procedure Dijkstra(*G*: weighted connected simple graph, with all weights positive)

{*G* has vertices $a = v_0, v_1, \cdots, v_n = z$ and weights $w(v_i, v_j)$

where $w(v_i, v_j) = \infty$ if $\{v_i, v_j\}$ is not an edge in *G*}

For *i*:= 1 to *n*

$L(v_i) := \infty$

$L(a) := 0$

$S := \emptyset$

{the labels are now initialized so that the label of *a* is zero and all other labels are ∞,and *S* is the empty set }



While $z \notin S$

Begin

u :=a vertex not in S with $L(u)$ minimal

S:=$S \cup \{u\}$

for all vertices v not in S

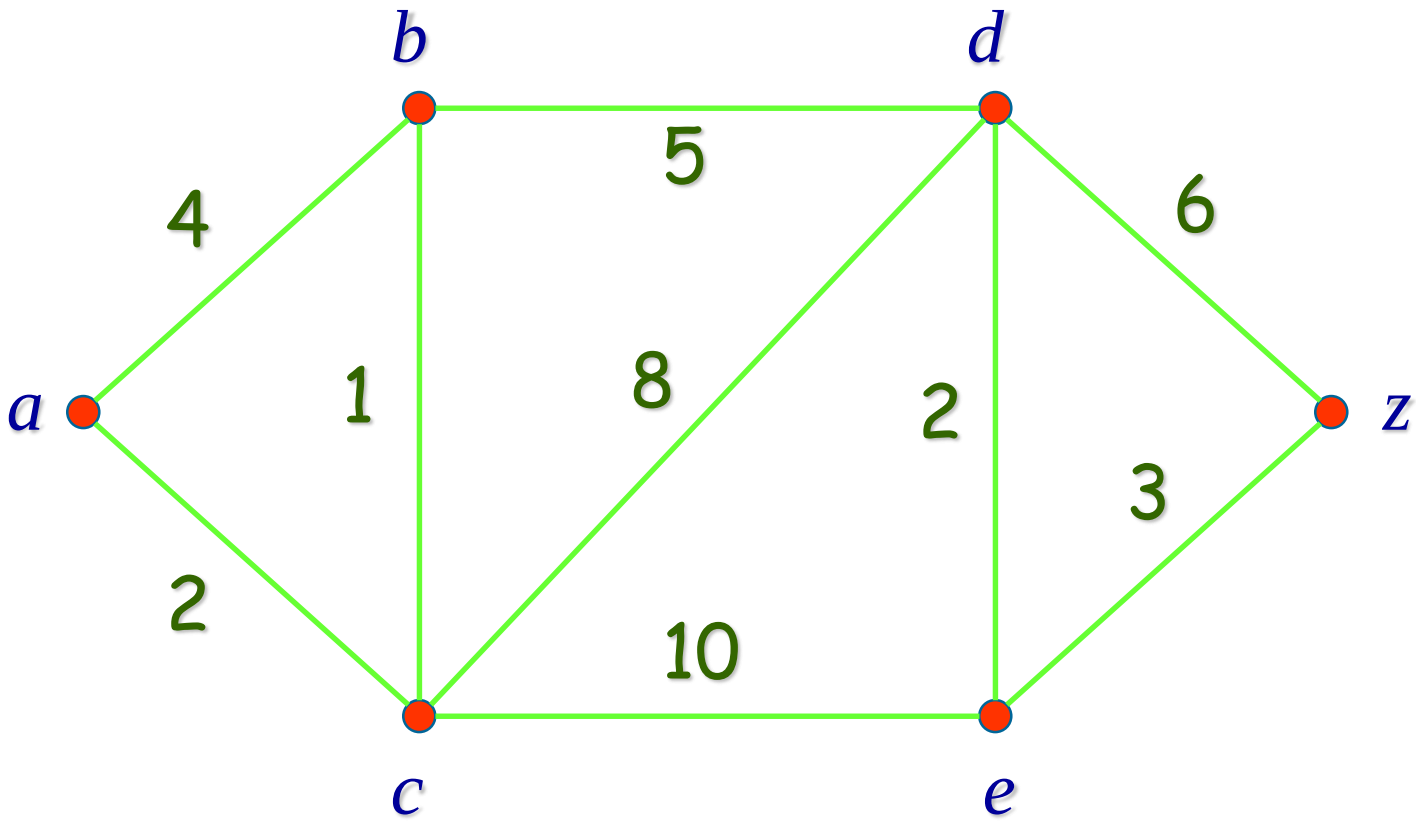
if $L(u) + w(u,v) < L(v)$ $L(v) := L(u) + w(u,v)$

{this adds a vertex to S with minimal label and updates the labels of vertices not in S}

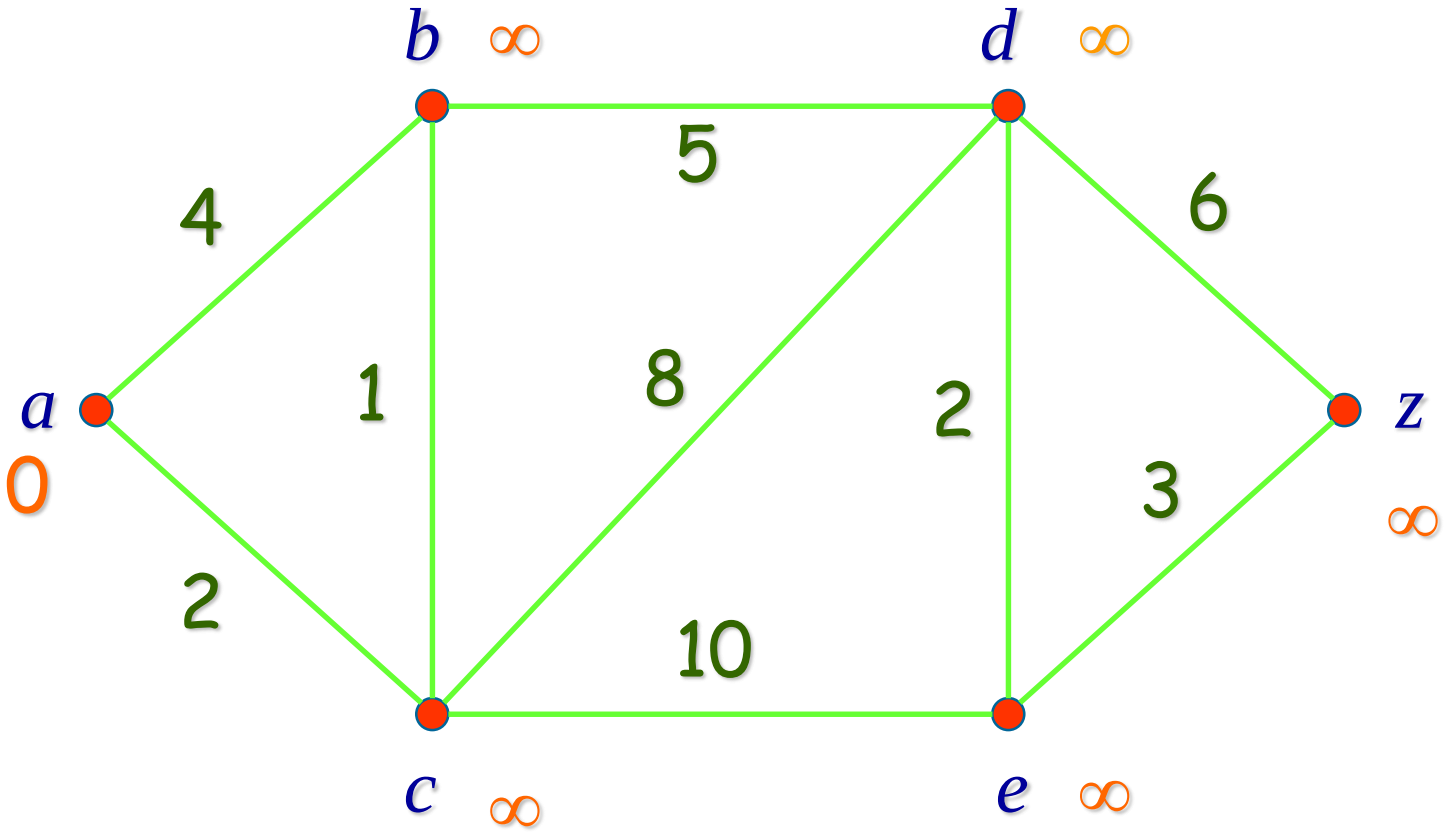
End {$L(z)$=length of shortest path from a to z }



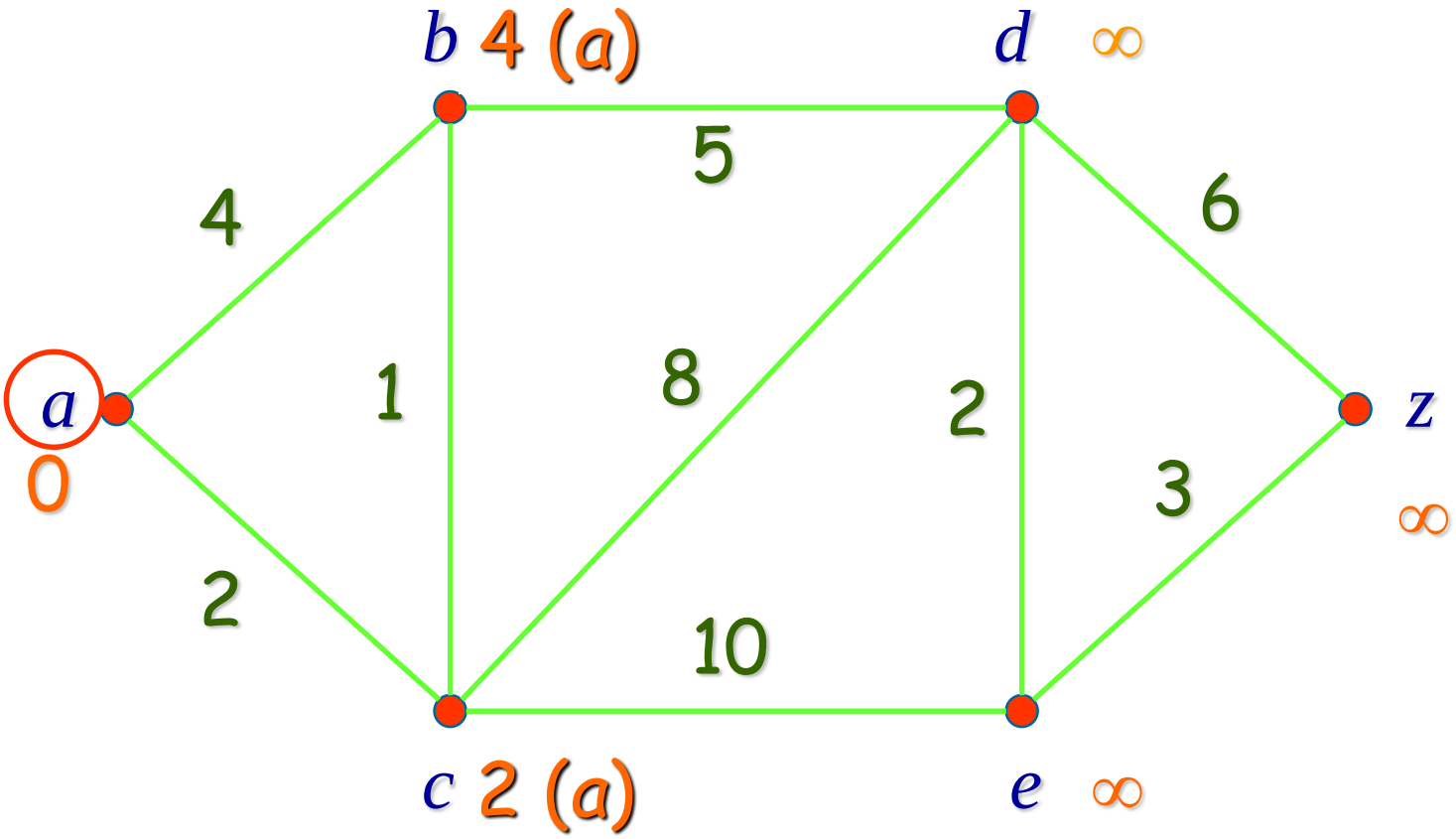
〖Example 2〗Find the length of the shortest path between *a* and *z* in the given weighted graph.



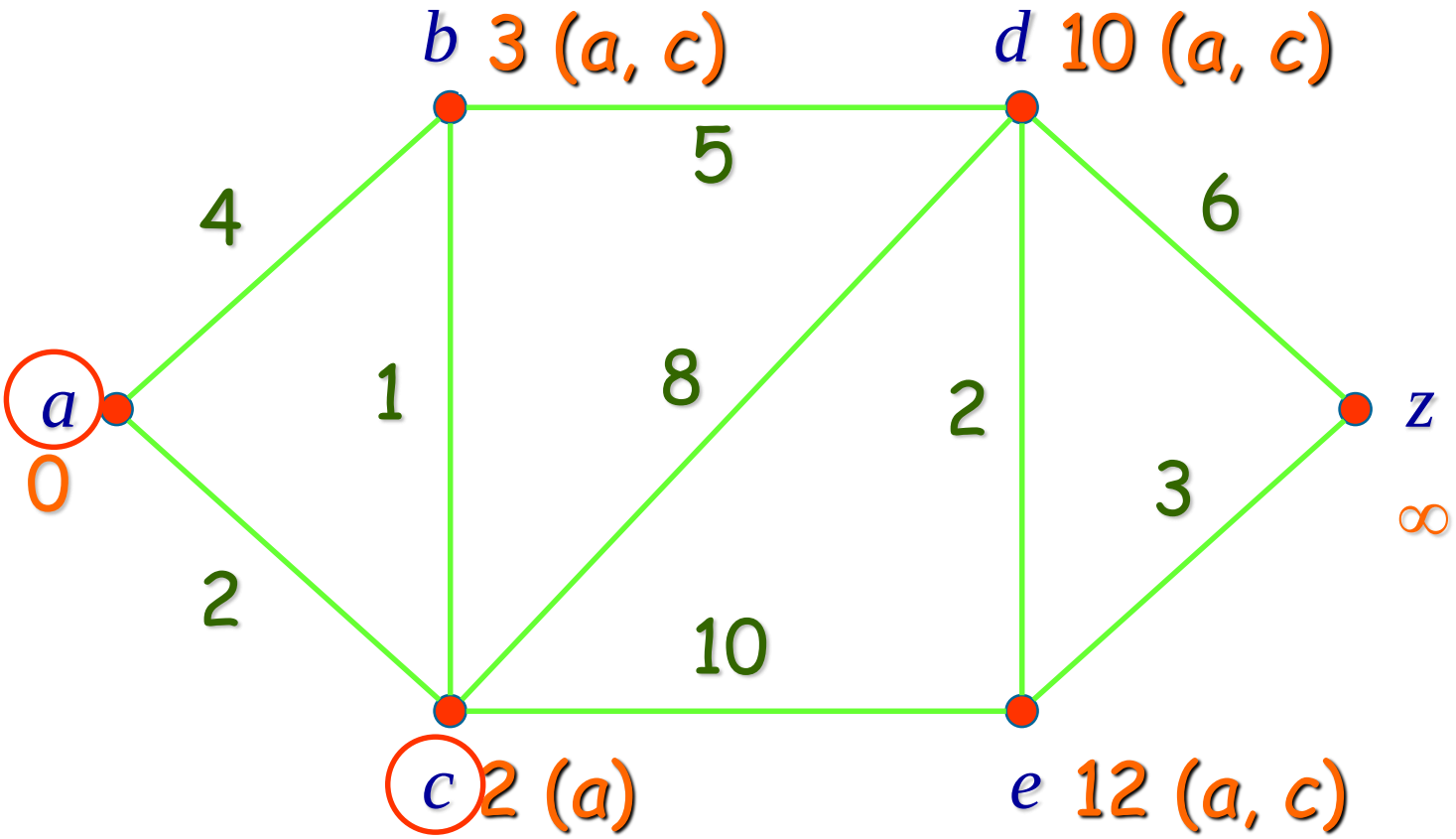
Step 0



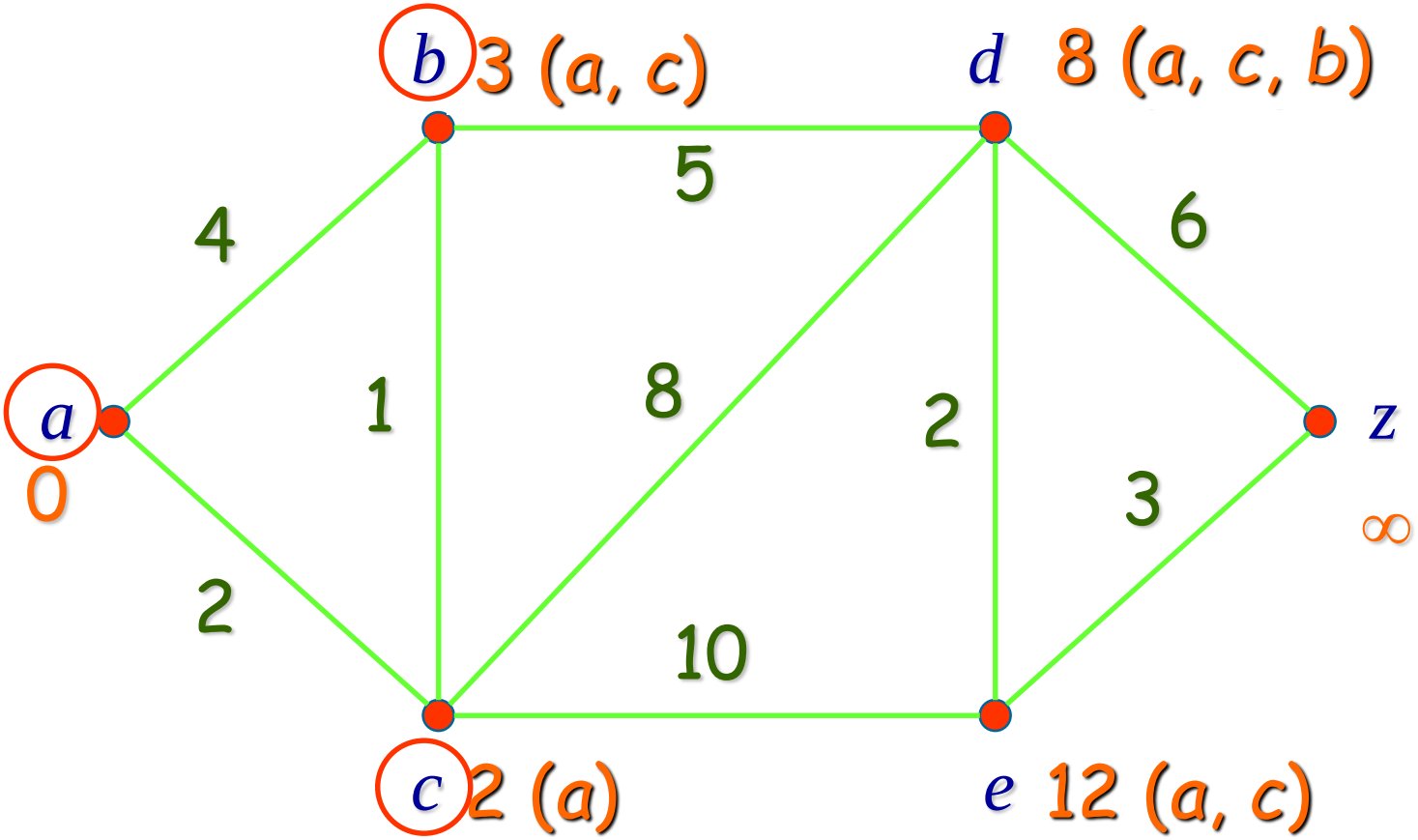
Step 1



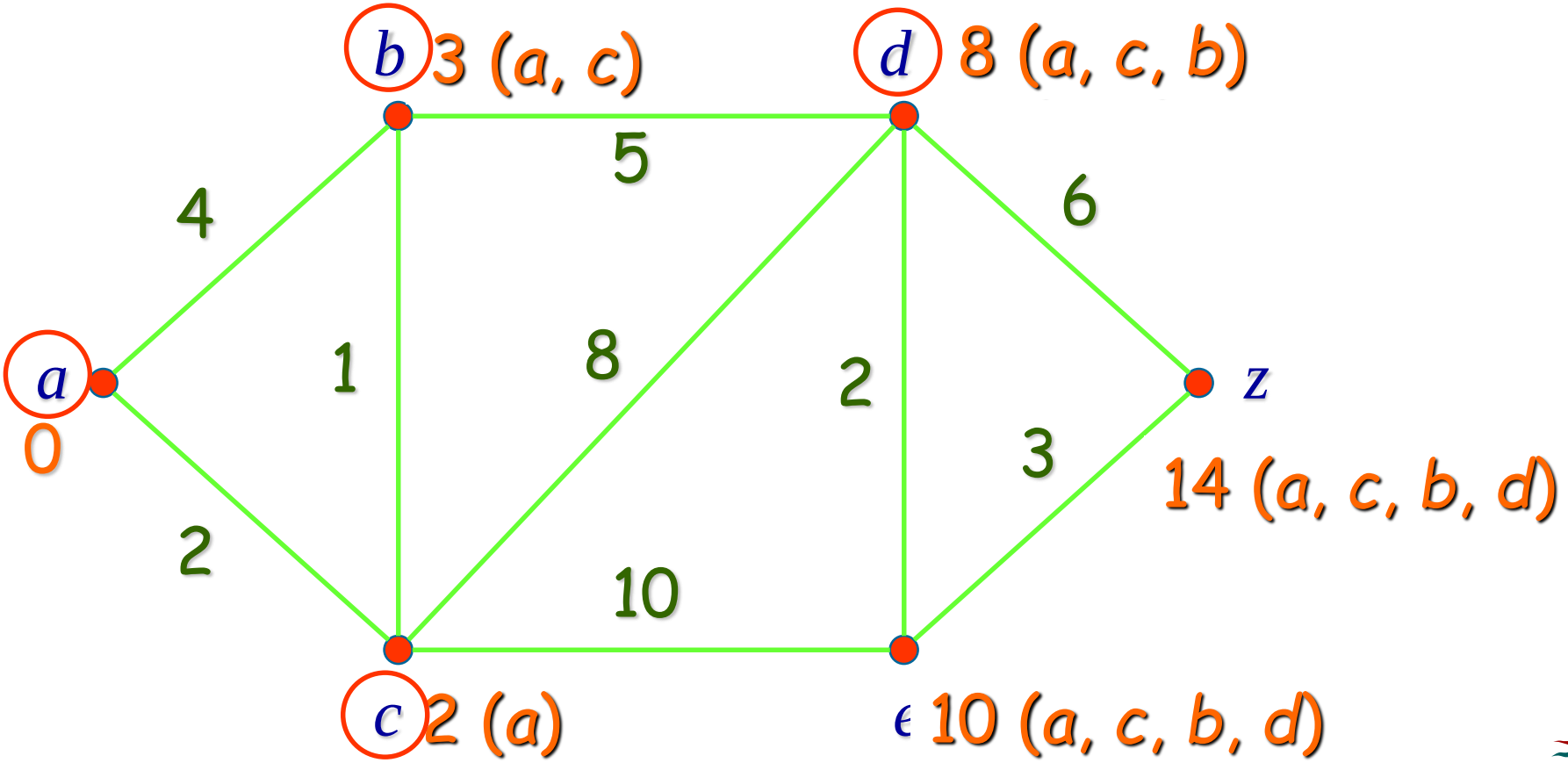
Step 2



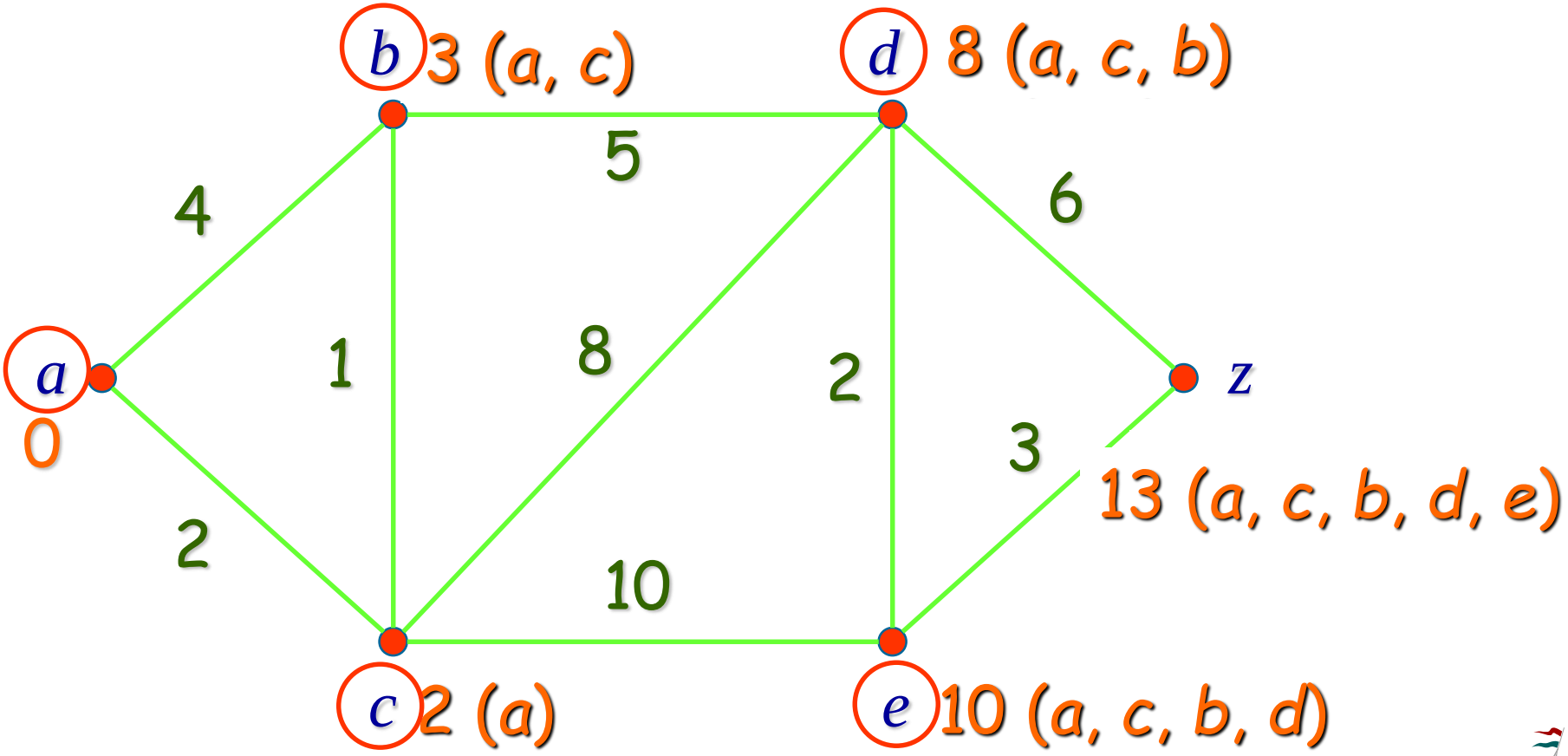
Step 3



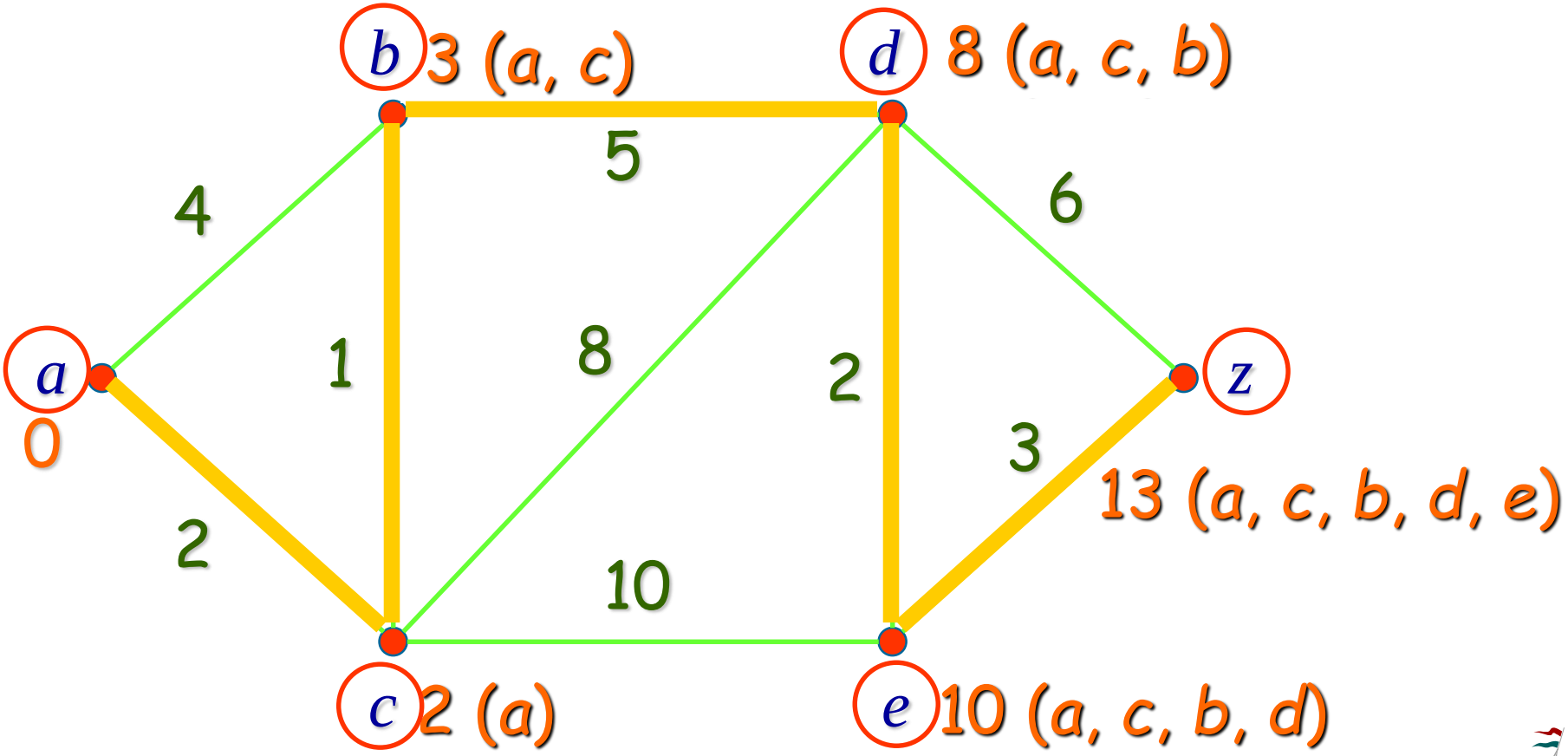
Step 4



Step 5



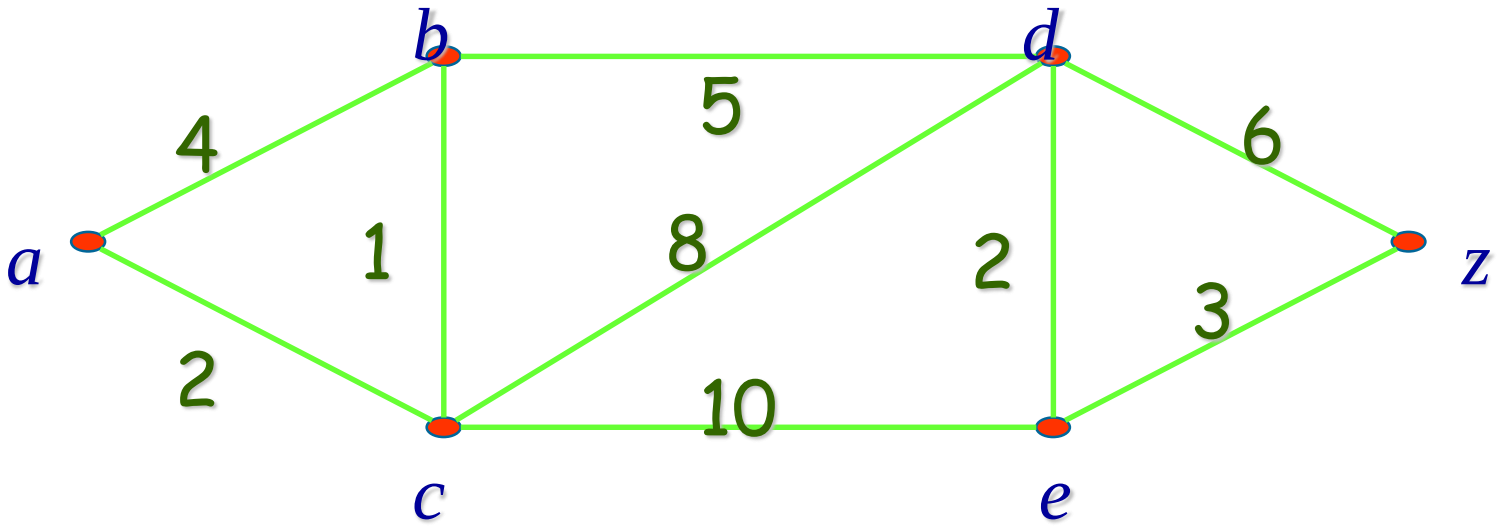
Step 6



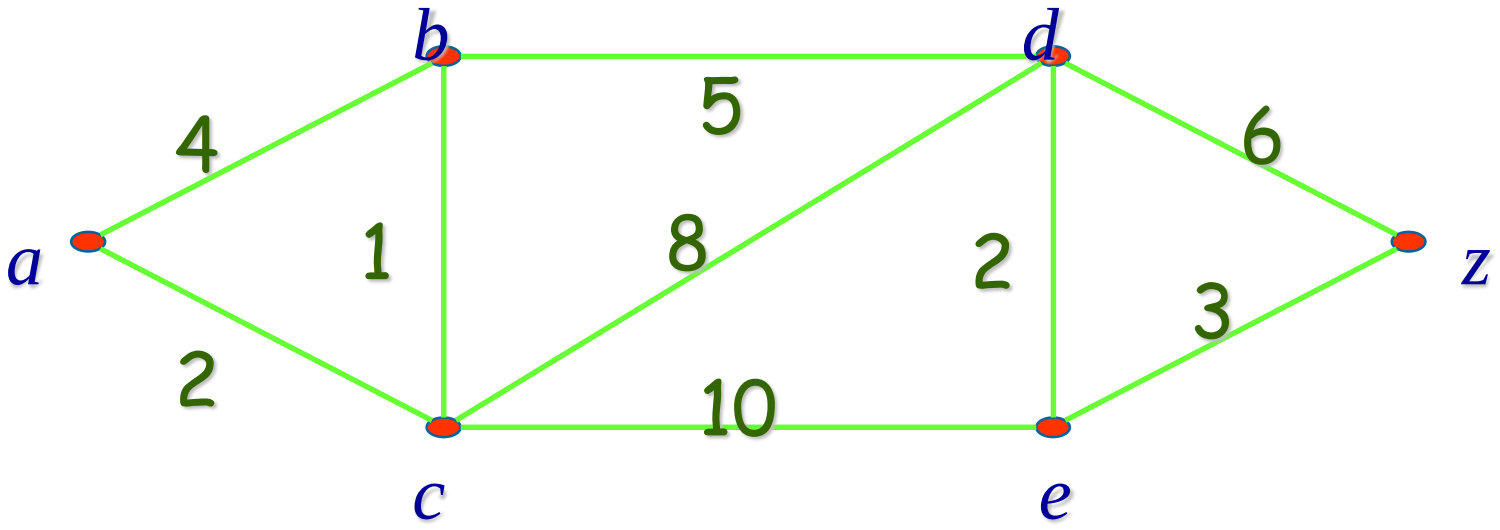
Remark:

In performing Dijkstra's algorithm it is sometimes more convenient to keep track of labels of vertices in each step using a table.

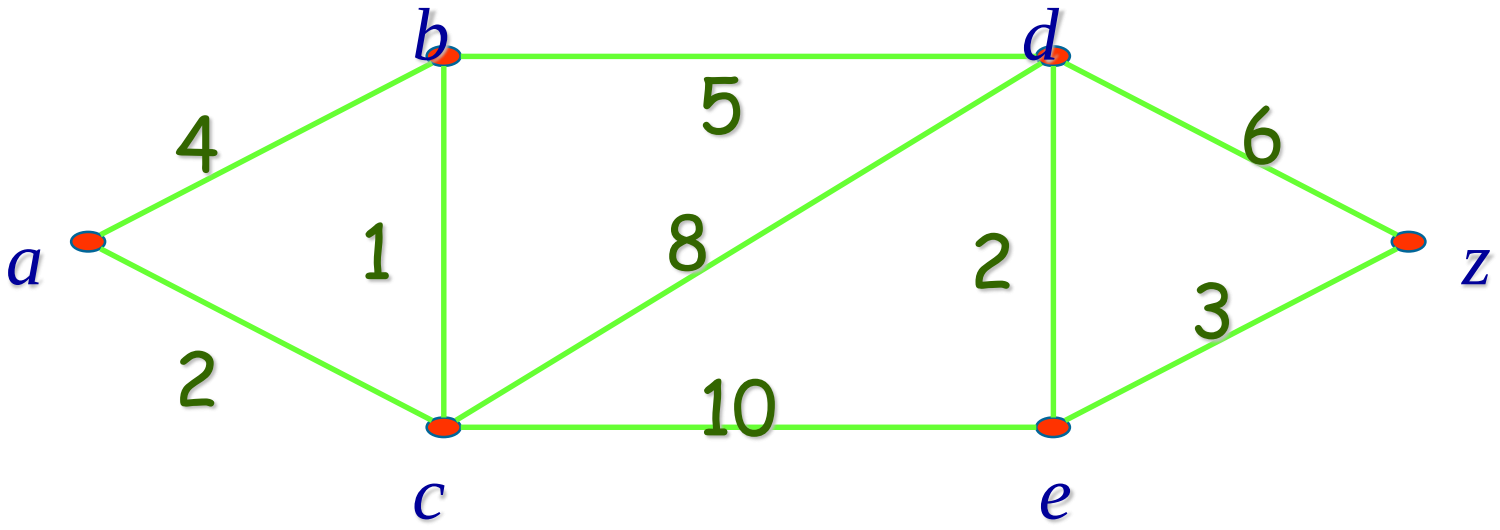




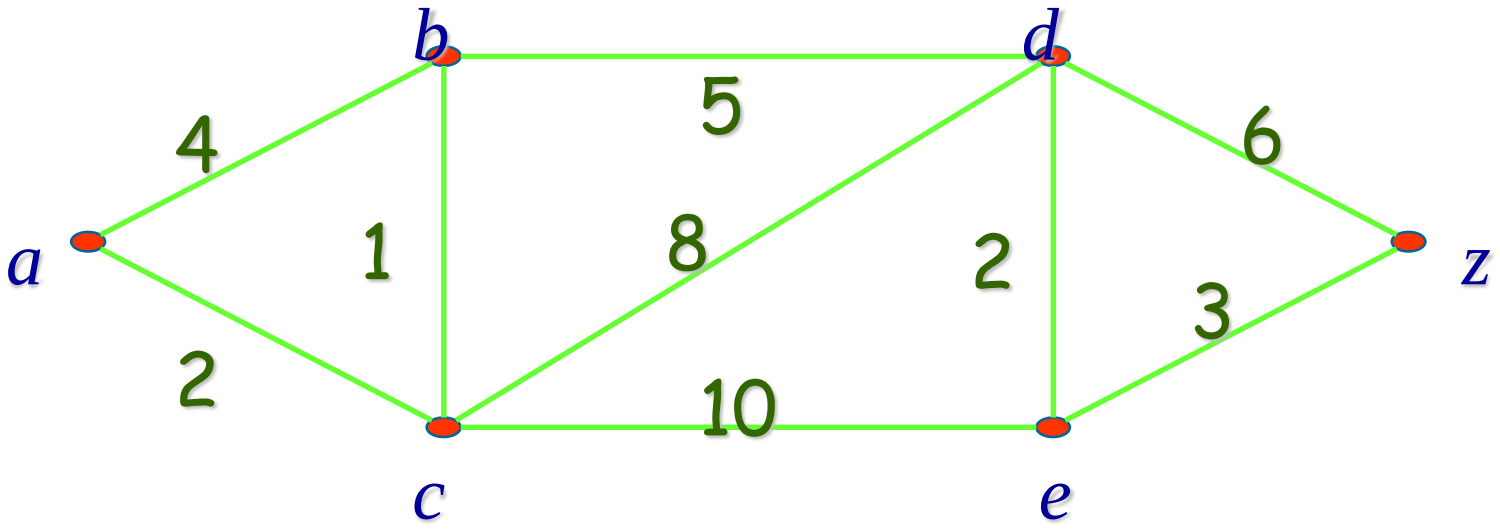
Vertex	S	Link						
a								
b								
c								
d								
e								
z								



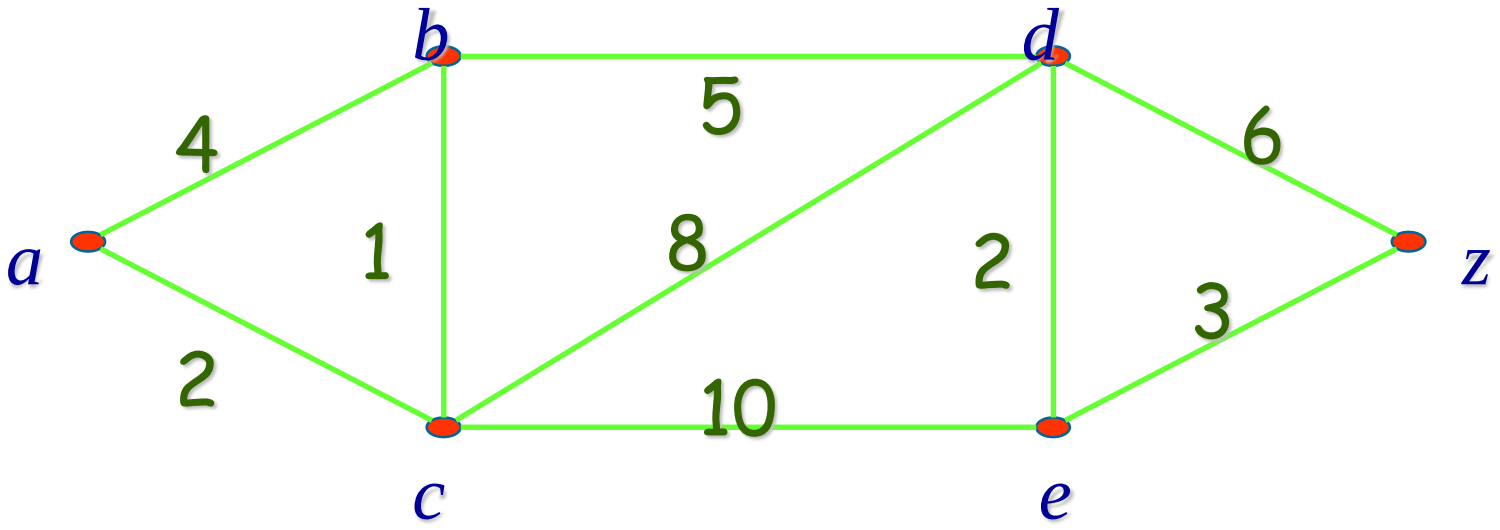
Vertex	S	Link	L_0				
a			0				
b			∞				
c			∞				
d			∞				
e			∞				
z			∞				



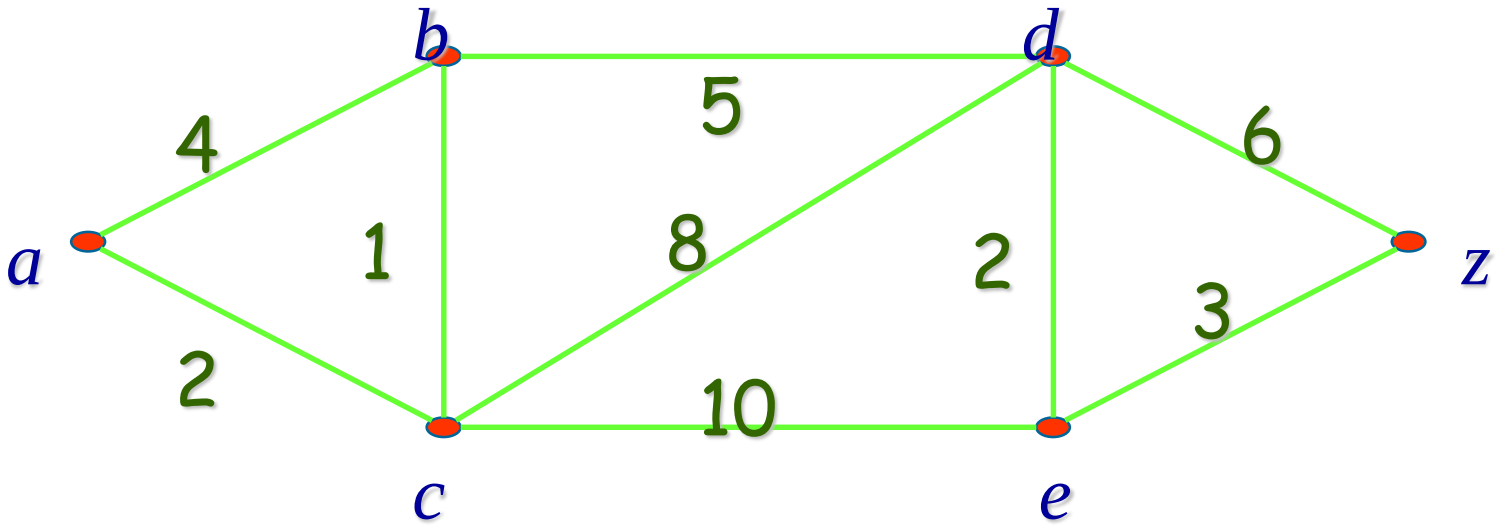
Vertex	S	Link	L_0	L_1				
a	1		0					
b		*a*	∞	4				
c		*a*	∞	2				
d			∞	∞				
e			∞	∞				
z			∞	∞				



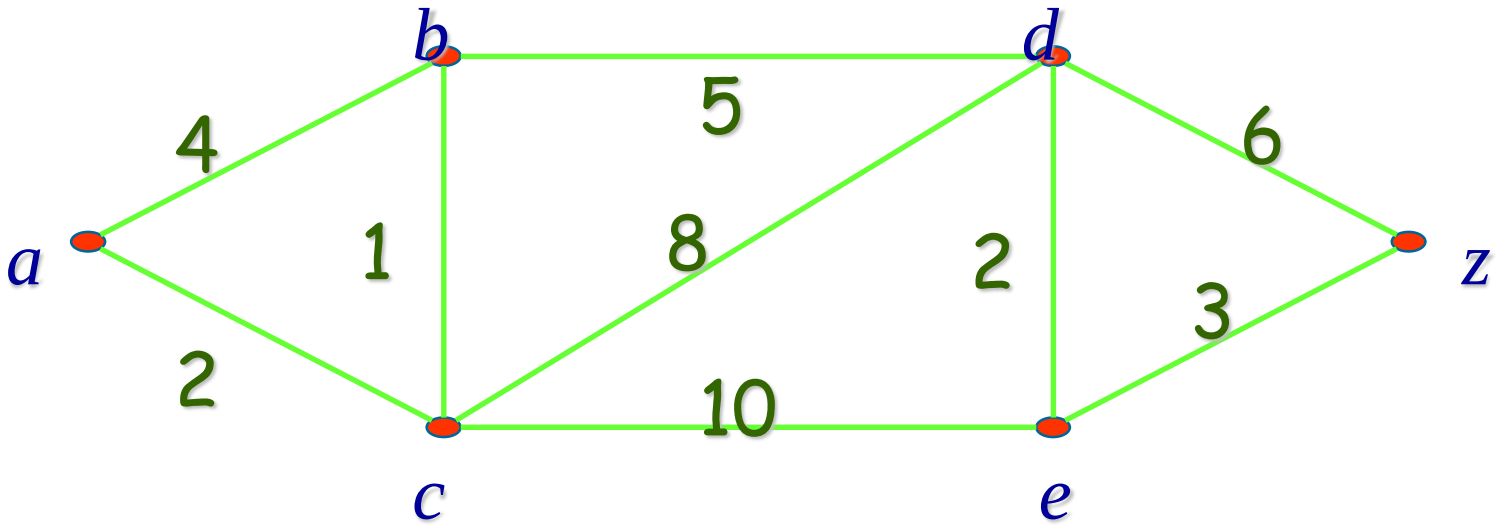
Vertex	S	Link	L_0	L_1	L_2			
a	1		0					
b		*a*→*c*	∞	4	3			
c	1	*a*	∞	2				
d		*a*→*c*	∞	∞	10			
e		*a*→*c*	∞	∞	12			
z			∞	∞	∞			



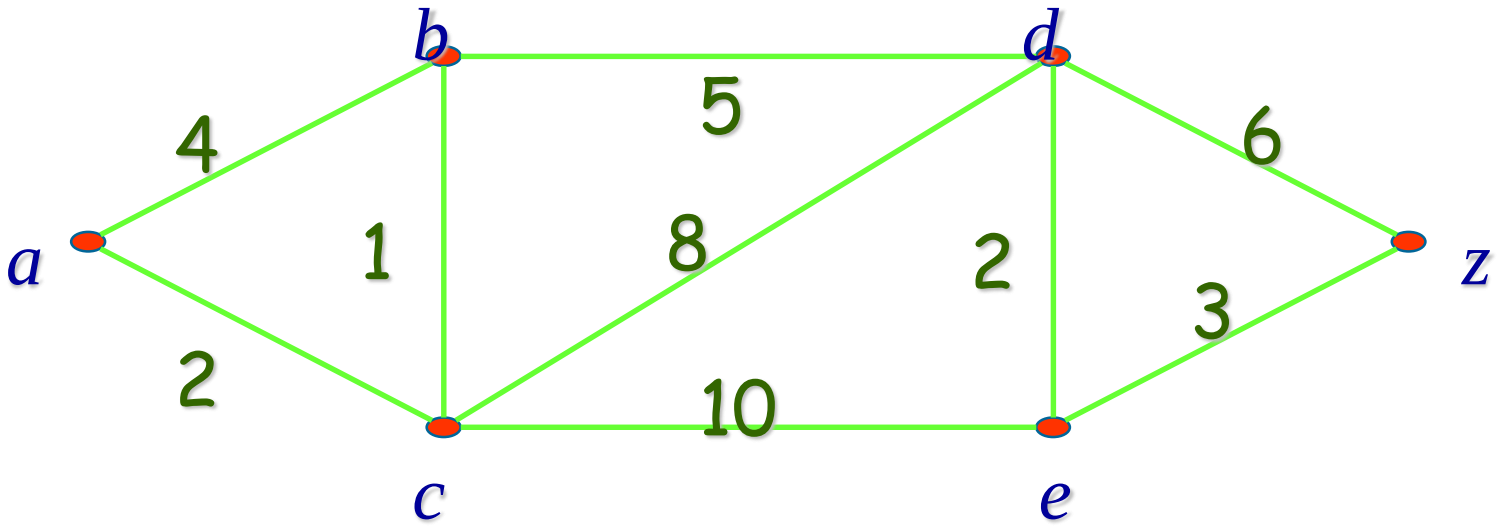
Vertex	S	Link	L_0	L_1	L_2	L_3		
a	1		0					
b	1	*a*→*c*	∞	4	3			
c	1	*a*	∞	2				
d		*a*→*c*→*b*	∞	∞	10	8		
e		*a*→*c*	∞	∞	12	12		
z			∞	∞	∞	∞		



Vertex	S	Link	L_0	L_1	L_2	L_3	L_4	
a	1		0					
b	1	*a*→*c*	∞	4	3			
c	1	*a*	∞	2				
d	1	*a*→*c*→*b*	∞	∞	10	8		
e		*a*→*c*→*b*→*d*	∞	∞	12	12	10	
z		*a*→*c*→*b*→*d*	∞	∞	∞	∞	14	



Vertex	S	Link	L_0	L_1	L_2	L_3	L_4	L_5
a	1		0					
b	1	*a*→*c*	∞	4	3			
c	1	*a*	∞	2				
d	1	*a*→*c*→*b*	∞	∞	10	8		
e	1	*a*→*c*→*b*→*d*	∞	∞	12	12	10	
z		*a*→*c*→*b*→*d* → *e*	∞	∞	∞	∞	14	13



Vertex	S	Link	L_0	L_1	L_2	L_3	L_4	L_5
a	1		0					
b	1	*a*→*c*	∞	4	3			
c	1	*a*	∞	2				
d	1	*a*→*c*→*b*	∞	∞	10	8		
e	1	*a*→*c*→*b*→*d*	∞	∞	12	12	10	
z	1	*a*→*c*→*b*→*d* → *e*	∞	∞	∞	∞	14	13

【Theorem 1】 Dijkstra's algorithm finds the length of a shortest path between two vertices in a connected simple undirected weighted graph.

Proof:

Take as the induction hypothesis the following assertion: At the *k*th iteration

I. the label of every vertex *v* in *S* is the length of the shortest path from *a* to this vertex, and

II. the label of every vertex not in *S* is the length of the shortest path from *a* to this vertex that contains only vertices in *S*.

(1) k=0

$L_0(a)=0$, $L_0(v)=\infty$, $S=\phi$



(2) Assume that the inductive hypothesis holds for the *k*th iteration.

Let *v* be the vertex added to *S* at the (*k*+1)st iteration so that *v* is a vertex not in *S* at the end of the *k*th iteration with the smallest label.

- **(I) holds at the end of the (*k*+1)st iteration**
  - ✓ **The vertices in *S* before the (*k*+1)st iteration are labeled with the length of the shortest path from *a*.**
  - ✓ ***v* must be labeled with the length of the shortest path to it from *a*.**

    If this were not the case, at the end of the *k*th iteration there would be a path of length less than $L_k(v)$ containing a vertex not in *S*.



Let u be the first vertex not in S in such a path. There is a path with length less than $L_k(v)$ from a to u containing only vertices of S. This contradicts the choice of v.

■ (II) is true.

Let u be a vertex not in S after k+1 iteration.

A shortest path from a to u containing only elements of S either contains v or it does not.

- If it does not contain v, then by the inductive hypothesis its length is $L_k(u)$.
- If it does contain v, then it must be made up of a path from a to v of the shortest possible length containing elements of S other than v, followed by the edge from v to u. In this case its length would be $L_k(v)+w(v,u)$.



【Theorem 2】 Dijkstra's algorithm uses $O(n^2)$ operations (additions and comparisons) to find the length of the shortest path between two vertices in a connected simple undirected weighted graph.

Analysis:

- Use no more than n-1 iteration
- Each iteration,

  using no more than n-1 comparisons to determine the vertex not in S_k with the smallest label

  no more than 2(n-1) operations are used to update no more than n-1 labels



3. The Traveling Salesman Problem

The traveling salesman problem

The traveling salesman problem asks for the circuit of minimum total weight in a weighted, complete, undirected graph that visits each vertex exactly once and returns to its starting point.

The equivalent problem:

Find a Hamilton circuit with minimum total weight in the complete graph.



✵ Solution of the traveling salesman problem

(1) A straightforward method

Examine all possible Hamilton circuits and select one of minimum total length.

The number of Hamilton circuits: $(n-1)!$

Since a Hamilton circuits can be traveled in reverse order, we need only examine $(n-1)!/2$ **circuits to find the answer.**

The time complexity:

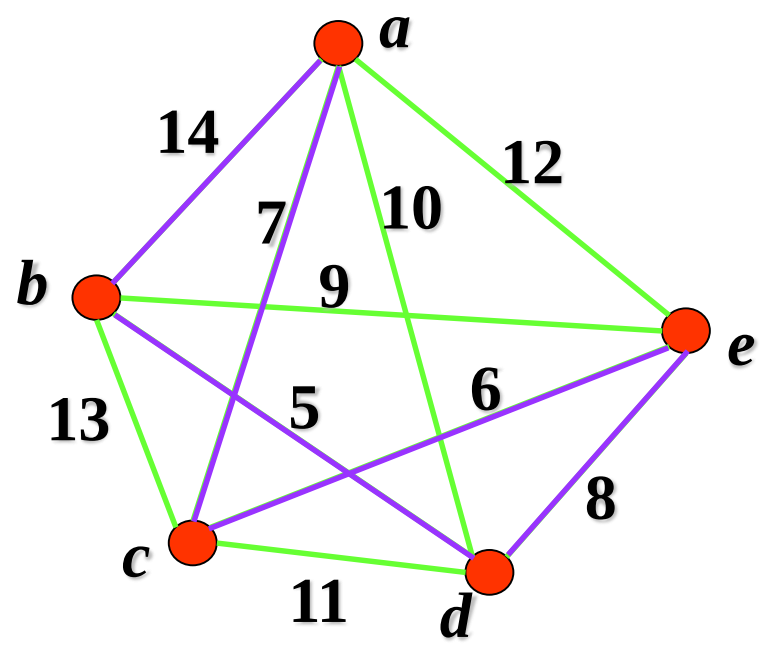
$$n!$$



(2) Approximation algorithm

These are algorithms that do not necessary produce the exact solution to the problem but instead are guaranteed to produce a solution that is close to an exact solution.

For example,



The length of this path: 40

The exact solution (the length is 37):*a*,*c*,*e*,*b*,*d*,*a*

The time complexity:

$$1+2+3+\cdots+(n-2)=\frac{1}{2}(n-1)(n-2)$$

Homework:

P. 751: 3, 17, 26



CHAPTER 10 Graphs

10.1 Graphs and Graph Models

10.2 Graph Terminology and Special Types of Graphs

10.3 Representing Graphs and Graph Isomorphism

10.4 Connectivity

10.5 Euler and Hamilton Paths

10.6 Shortest Path Problems

10.7 Planar Graphs

10.8 Graph Coloring



【Definition】A graph is called *planar* if it can be drawn in the plane without any edges crossing .

Such a drawing is called a *planar representation* of the graph.

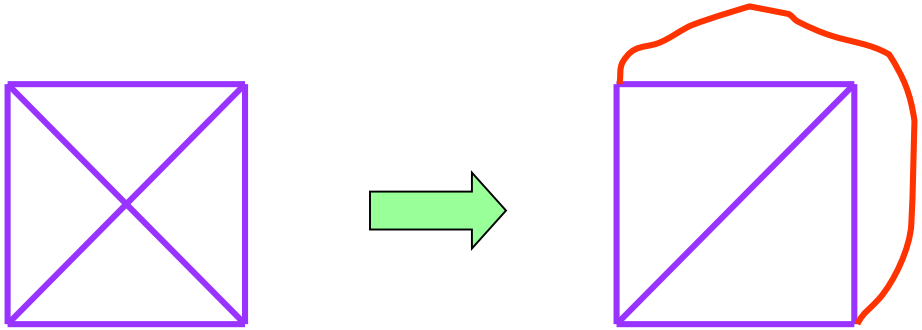
Understanding planar graph is important:

- **Any graph representation of maps is planar.**
- **Electronic circuits usually represented by planar graphs.**

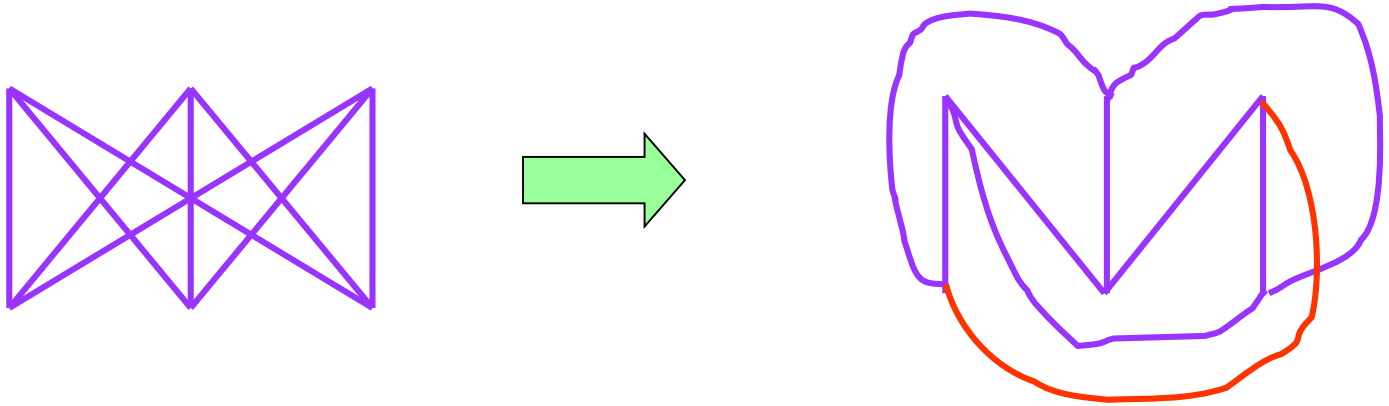


〖Example 1〗Are the following graphs planar?

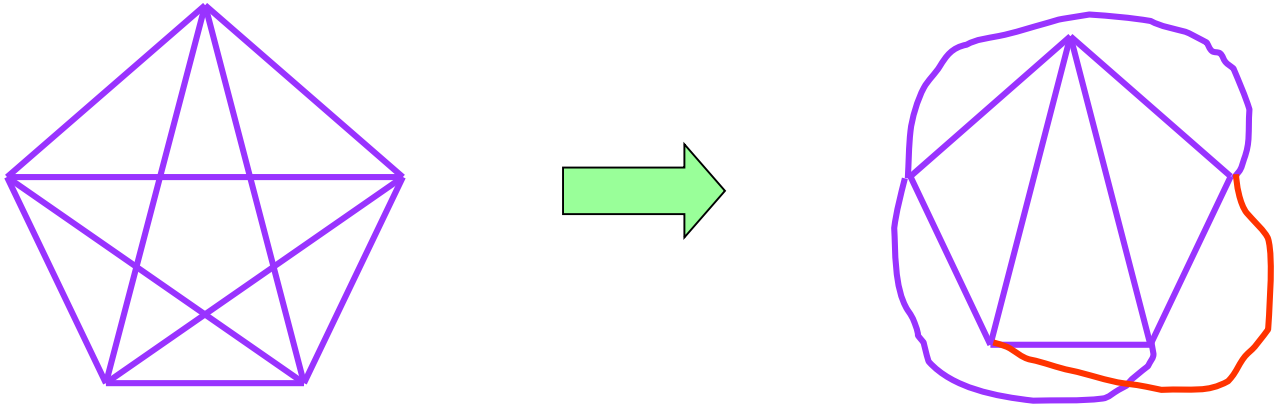
(1)



(2)



(3)



Note:

To prove that a graph is planar amounts to redrawing the edges in a way that no edges will cross. May need to move vertices around and the edges may have to be drawn in a very indirect fashion.

1. Euler's Formula

Region

-- **A region is a part of the plane completely disconnected off from other parts of the plane by the edges of the graph.**

- **Bounded region**
- **Unbounded region**

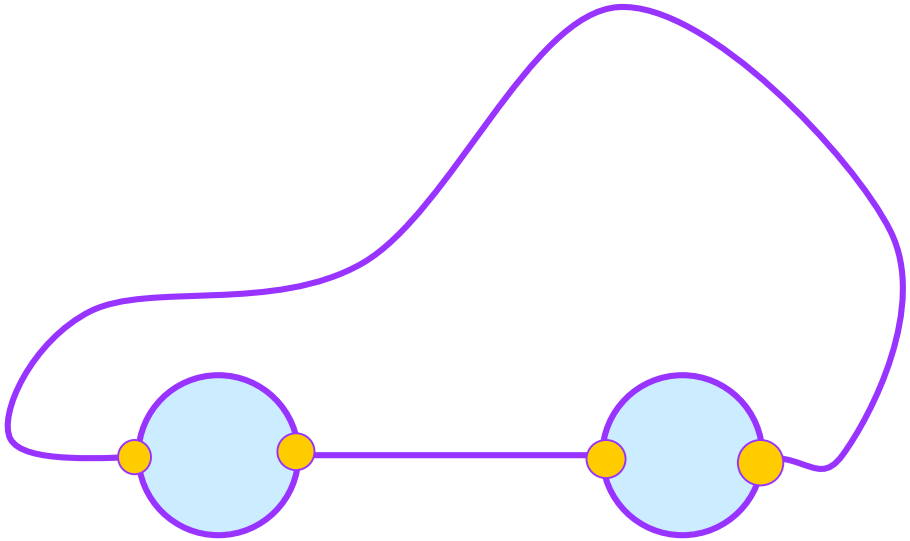
Note:

There is one unbounded region in a planar graph.



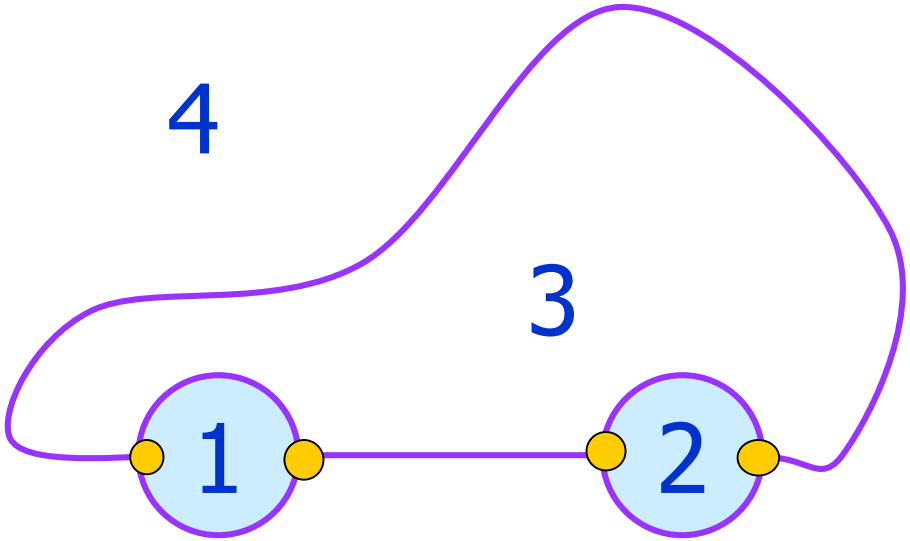
〖Example 2〗How many regions does the following graph have?

(1)

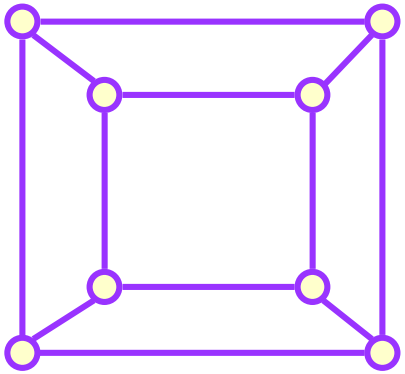


〖Example 2〗How many regions does the following graph have?

(1)

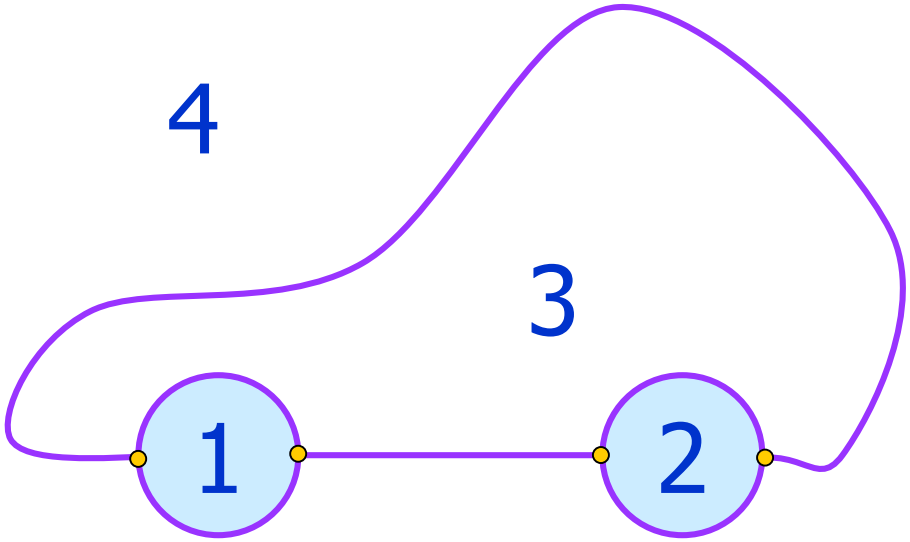


(2)

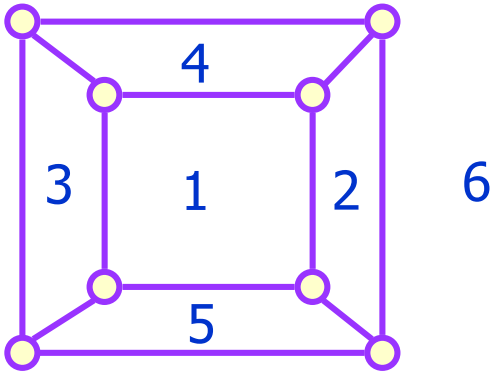


〖Example 2〗How many regions does the following graph have?

(1)



(2)



【Theorem 1】 Euler's formula

Let G be a *connected planar simple graph* with e edges and v vertices. Let r be the number of regions in a planar representation of G. Then $r=e-v+2$.

Proof:

First, we specify a planar representation of G. We will prove the theorem by constructing a sequence of subgraphs G_1, G_2, ⋯; $G_e = G$, successively adding an edge at each stage.

The constructing method: Arbitrarily pick one edge of G to obtain G_1. Obtain G_n from G_{n-1} by arbitrarily adding an edge that is, incident with a vertex already in G_{n-1}.

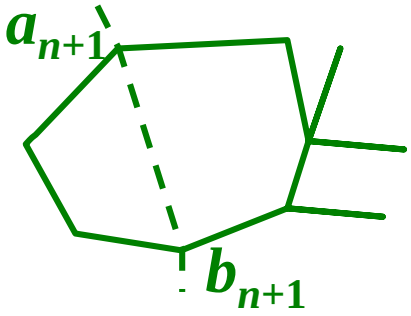
Let r_n, e_n, and v_n represent the number of regions, edges, and vertices of the planar representation of G_n induced by the planar representation of G, respectively.



(1) The relationship $r_1 = e_1 - v_1 + 2$ is true for G_1, since $e_1 = 1$, $v_1 = 2$, and $r_1 = 1$.

(2) Now assume that $r_n = e_n - v_n + 2$. Let $\{a_{n+1}, b_{n+1}\}$ be the edge that is added to G_n to obtain G_{n+1}.

◆ Both a_{n+1} and b_{n+1} are already in G_n.



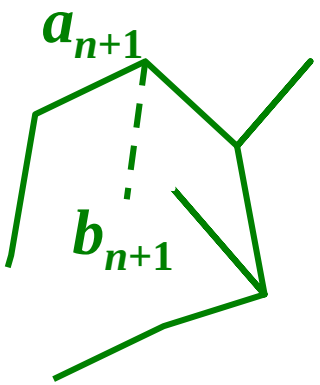
These two vertices must be on the boundary of a common region R, or else it would be impossible to add the edge $\{a_{n+1}, b_{n+1}\}$ to G_n without two edges crossing (and G_{n+1} is planar).

The addition of this new edge splits R into two regions.

Consequently, $r_{n+1} = r_n + 1$, $e_{n+1} = e_n + 1$, and $v_{n+1} = v_n$. Thus, $r_{n+1} = e_{n+1} - v_{n+1} + 2$.

◆ One of the two vertices of the new edge is not already in G_n .

Suppose that a_{n+1} is in G_n but that b_{n+1} is not.



Adding this new edge does not produce any new regions, since b_{n+1} must be in a region that has a_{n+1} on its boundary.

Consequently, $r_{n+1} = r_n$. Moreover, $e_{n+1} = e_n + 1$ and $v_{n+1} = v_n + 1$.

Hence, $r_{n+1} = e_{n+1} + 1 - v_{n+1} - 1 + 2$.

Note:

1) The Euler's formula is *necessary condition*.

2) How about unconnected simple planar graph?



〖Example 3〗 Suppose that a connected planar simple graph has 20 vertices, each of degree 3. How many regions does this planar graph have?

Solution:

By handshaking theorem,

$$3v = 2e \Rightarrow e = 30$$

From Euler's formula, the number of regions is

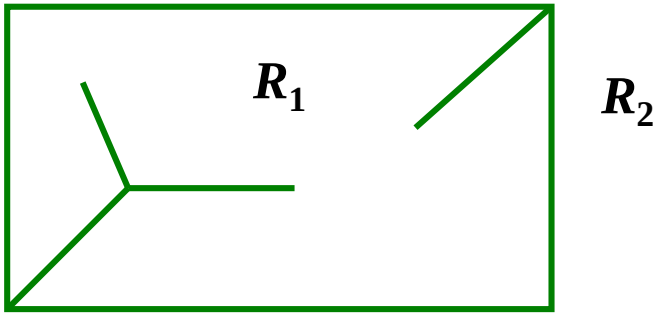
$$r = e - v + 2 = 30 - 20 + 2 = 12$$



【Definition】 Suppose R is a region of a connected planar simple graph, the number of the edges which surround R is called the *Degree of R* .

Notation: Deg(R)

For example,

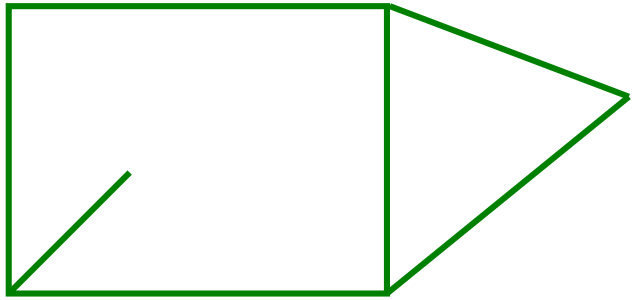


$\mathrm{Deg}(R_1) = 12$
$\mathrm{Deg}(R_2) = 4$

【 Corollary 1 】 If G is a connected planar simple graph with e edges and v vertices where $v \geq 3$, then $e \leq 3v-6$

Proof:

Suppose that a connected planar simple graph divides the plane into r regions, the degree of each region is at least 3.



Since $2e = \sum \deg(R_i) \geq 3r$,

it imply $r \leq (2/3)e$

Using Euler's formula $e-v+2=r$, we obtain

$e-v+2 \leq (2/3)e$, this shows that

$e \leq 3v-6$

Note:

- **The equality holds if and only if every region has exactly three edges.**
- **For unconnected planar simple graph，$e \le 3v-6$ is also holds.**

Since for a component, $e_i \le 3v_i - 6$

$$e = \sum e_i \le \sum (3v_i - 6) < 3\sum v_i - 6 = 3v - 6$$



【 Corollary 2 】 If *G* is a connected planar simple graph, then *G* has *a* vertex of degree not exceeding five.

Proof:

(1) *G* has one or two vertices

(2) G has at least three vertices

By Corollary 1 , we know that $e \leq 3v-6$, so $2e \leq 6v-12$

If the degree of every vertex were at least six, then

$2e \geq 6v$



【 Corollary 3 】 If a connected planar simple graph has *e* edges and *v* vertices with $v \geq 3$ and no circuits of length 3,then $e \leq 2v-4$.

Generally, if every region of a planar connected graph has at least *k* edges, then

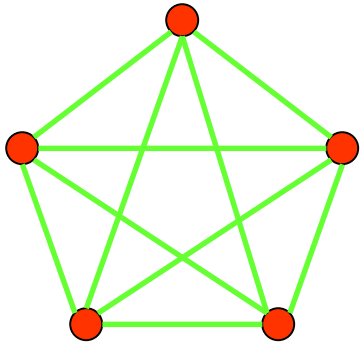
$$e \leq \frac{(v-2)k}{k-2}$$



〖Example 4〗 Show that k_5, $k_{3,3}$ are nonplanar.

Proof:

(1)

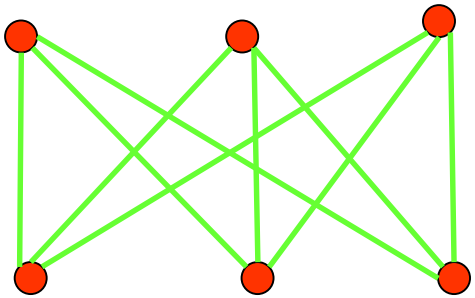


The graph k_5 has 5 vertices and 10 edges.

However, the inequality $e \leqq 3v-6$ is not satisfied for this graph since $e=10$ and $3v-6=9$.

Therefore, k_5 is not planar.

(2)



$k_{3,3}$ has 6 vertices and 9 edges.

Since $k_{3,3}$ has no circuits of length 3 (this is easy to see since it is bipartite), Corollary 2 can be used .

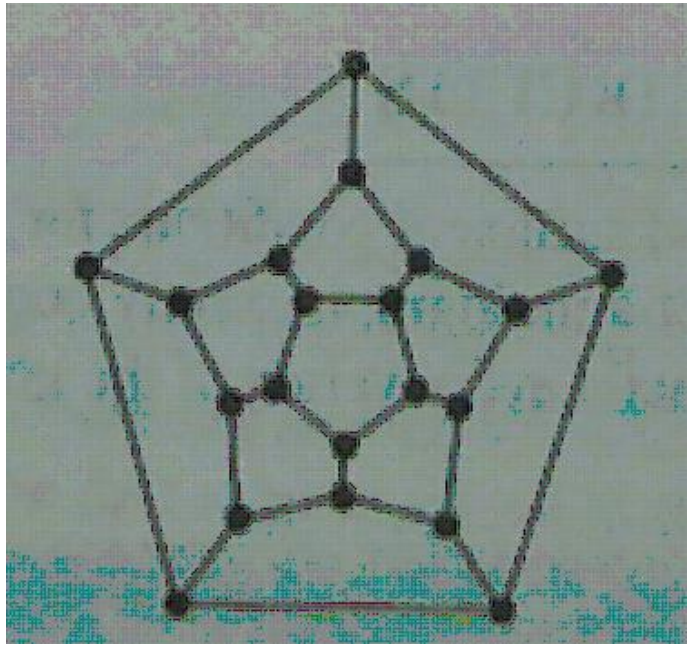
Since e=9 and 2v-4=8, corollary 2 shows that $k_{3,3}$ is nonplanar.

〖Example 5〗 The construction of Dodecahedron .

Solution:

Since the degree of every vertex is 3 and the degree of every region is 5. Then

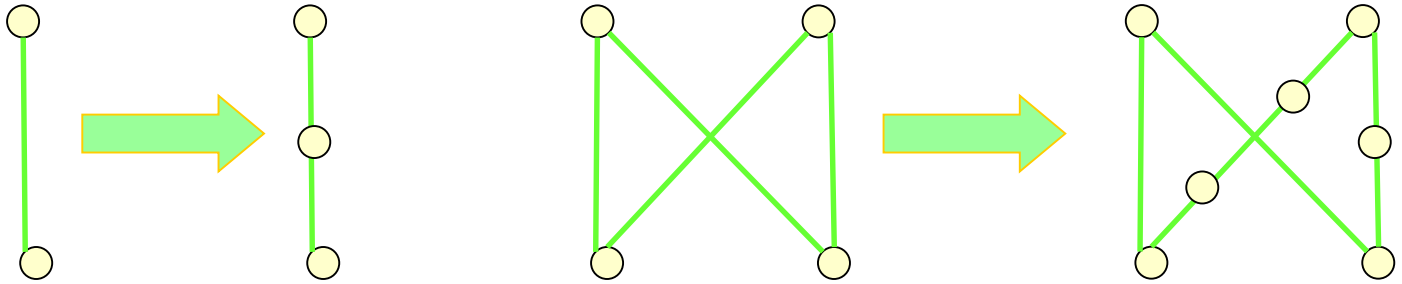
$$\begin{cases} 2e = 3v \\ 2e = 5r \\ r = e - v + 2 \end{cases}$$



It follows that *v*=20, *e*=30 and *r*=12.

2. KURATOWSKI'S THEOREM

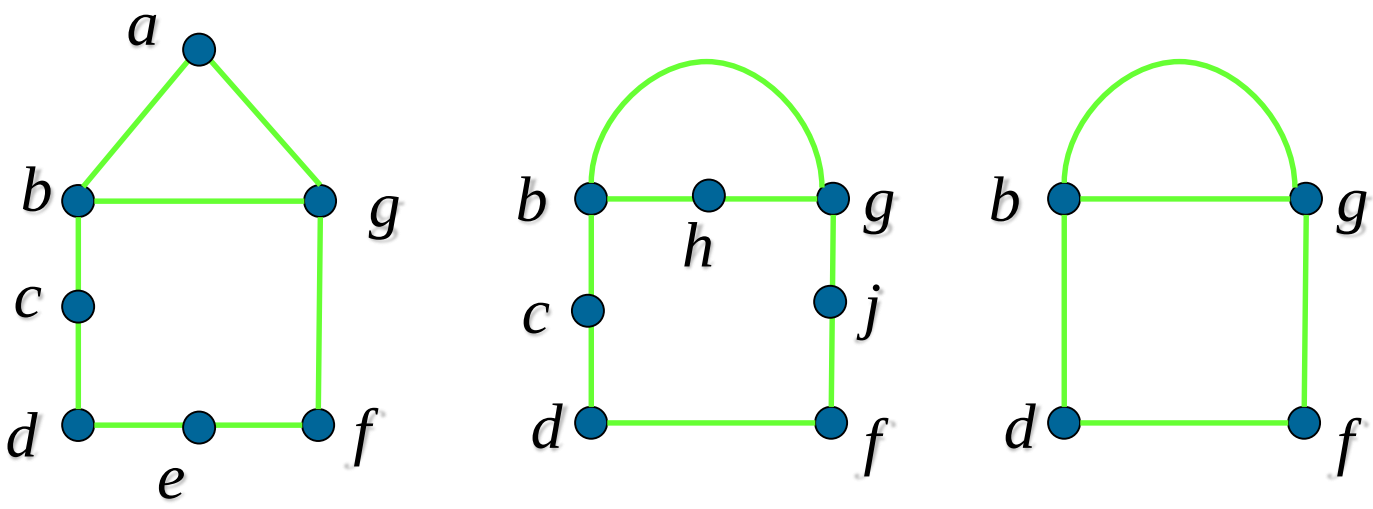
Elementary subdivision



homeomorphic

-- **The graph $G_1=(V_1,E_1)$ and $G_2=(V_2,E_2)$ are called homeomorphic if they can be obtained from the same graph by a sequence of elementary subdivision.**

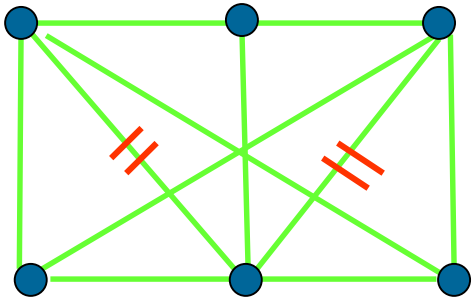
For example,

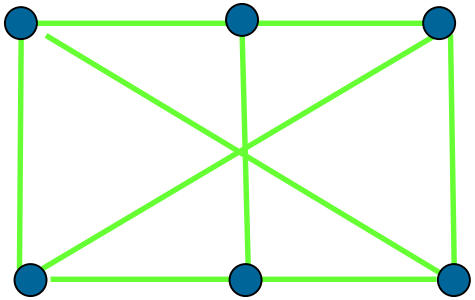


【 Theorem 2】 A graph is nonplanar if and only if it contains a subgraph homeomorphic to $K_{3,3}$ or K_5.

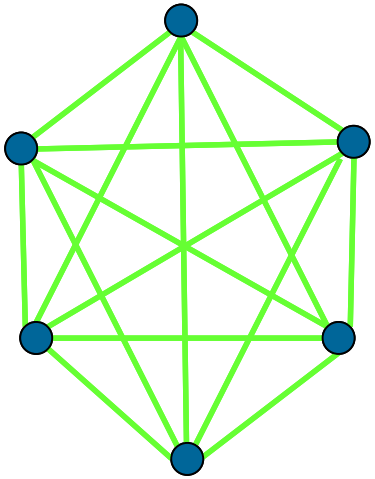
〖Example 6〗 Determine whether the following graphs are planar.

(1)



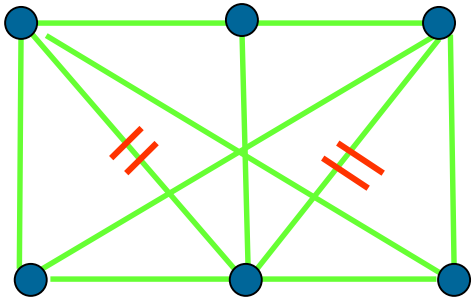


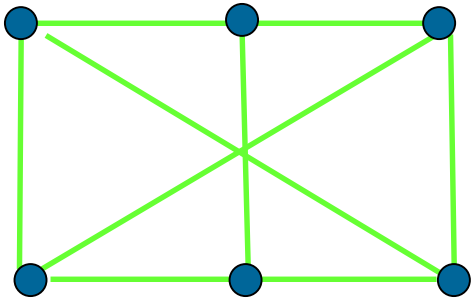
(2)



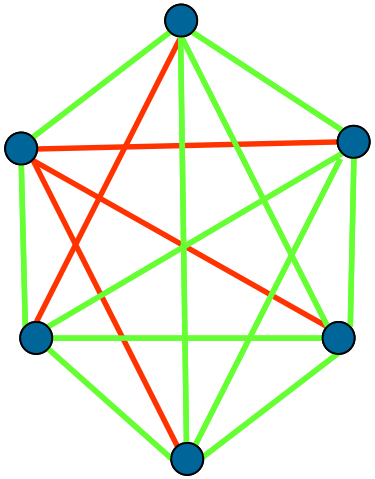
〖Example 6〗 Determine whether the following graphs are planar.

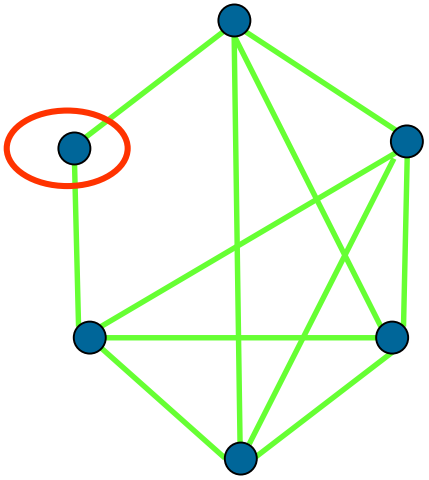
(1)





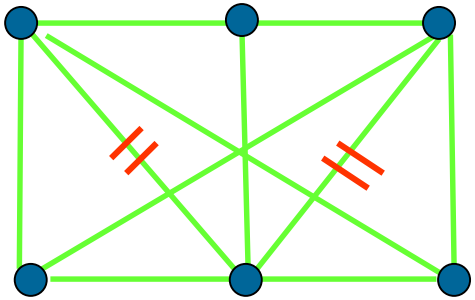
(2)

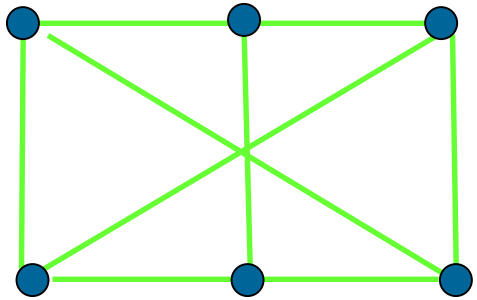




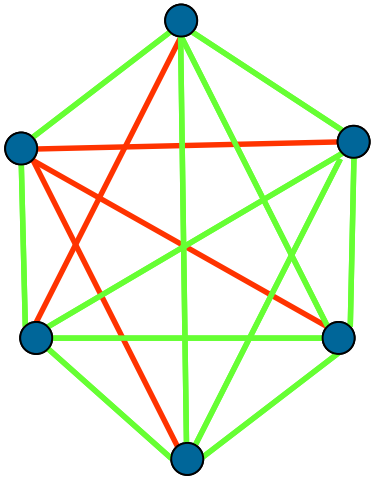
〖Example 6〗 Determine whether the following graphs are planar.

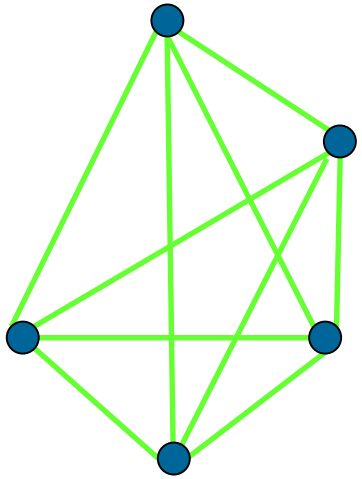
(1)





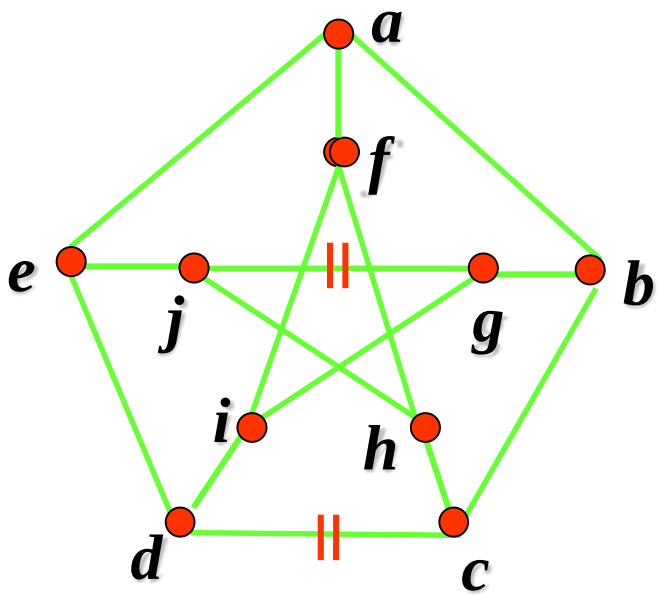
(2)

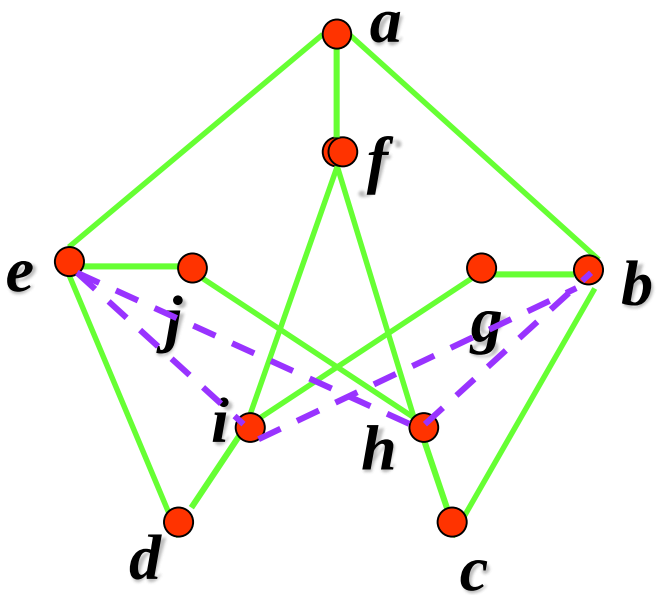


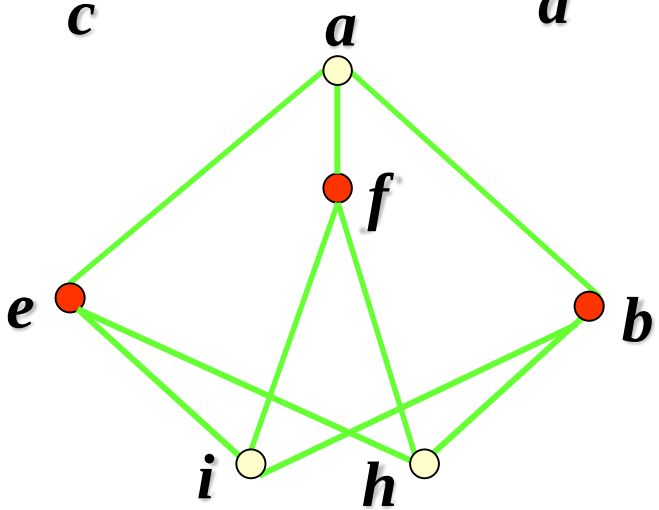


〖Example 6〗 Determine whether the following graphs are planar.

(3)







Homework:

P. 760: 7, 17, 20-25



CHAPTER 9 Graphs

9.1 Graphs and Graph Models

9.2 Graph Terminology and Special Types of Graphs

9.3 Representing Graphs and Graph Isomorphism

9.4 Connectivity

9.5 Euler and Hamilton Paths

9.6 Shortest Path Problems

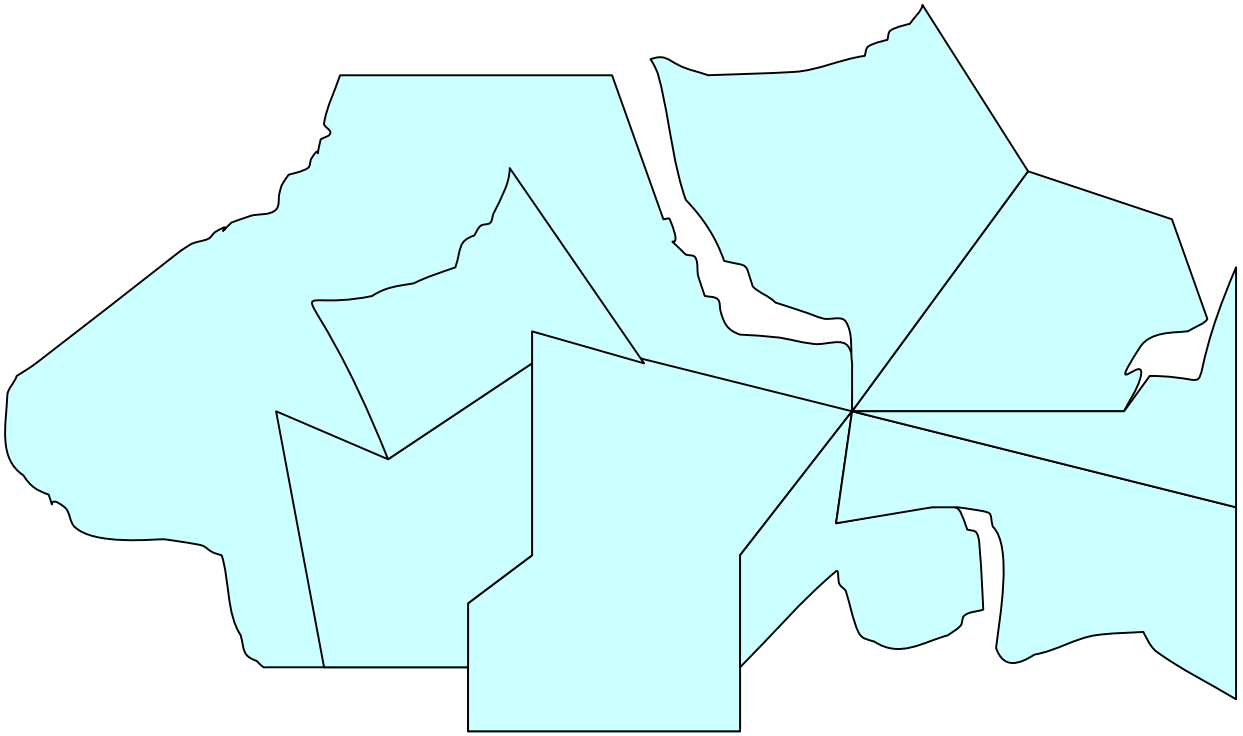
9.7 Planar Graphs

9.8 Graph Coloring



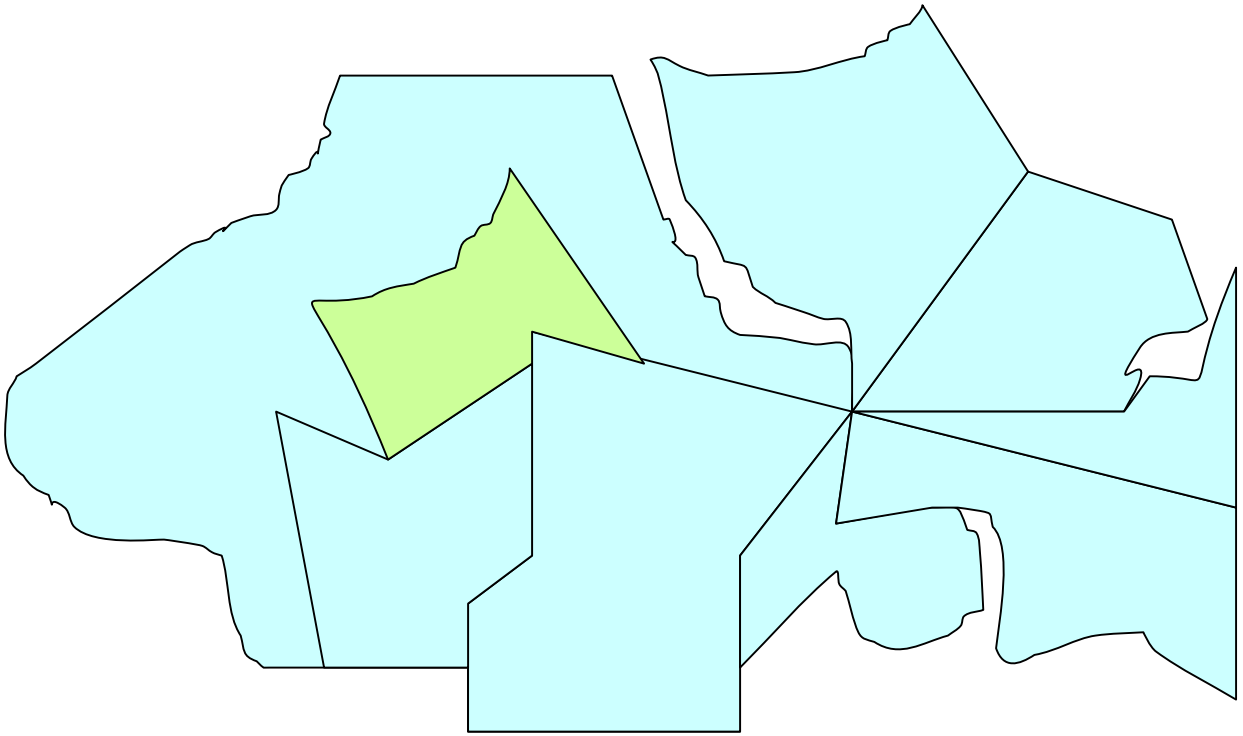
The problem related to the coloring of maps:

Determining the least number of colors that can be used to color a map so that adjacent regions never have the same color.



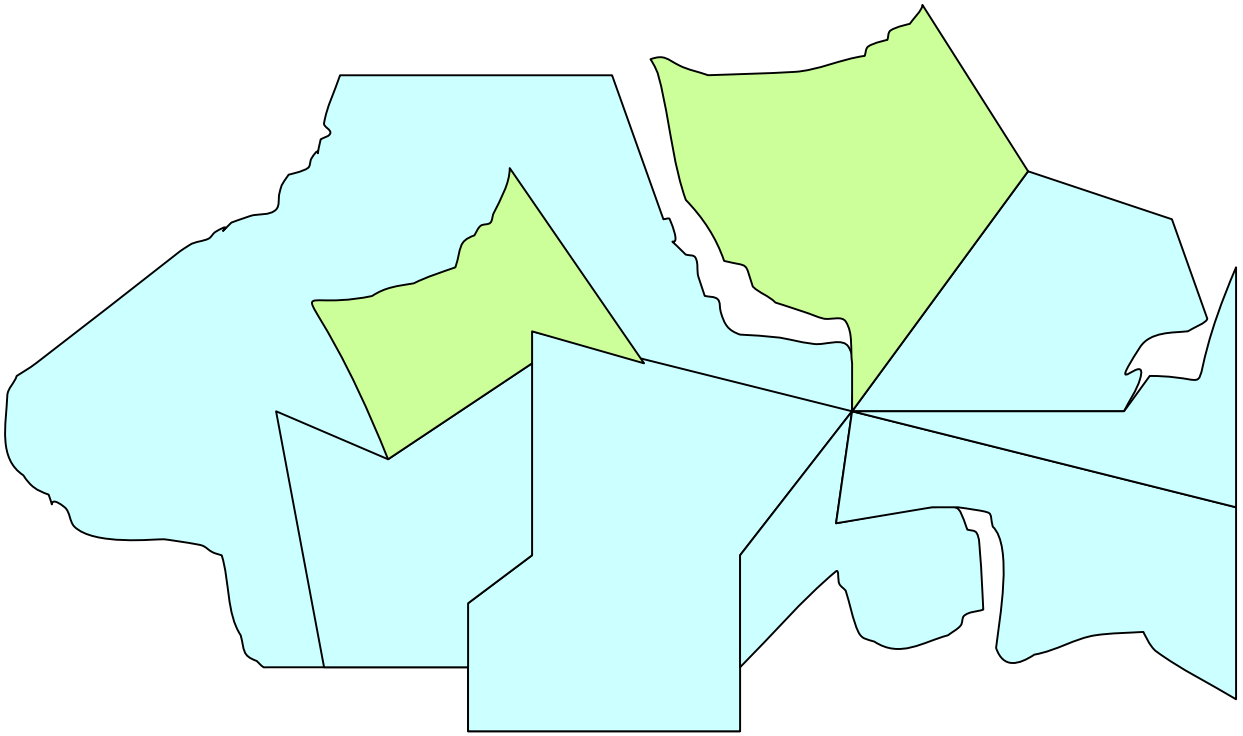
The problem related to the coloring of maps:

Determining the least number of colors that can be used to color a map so that adjacent regions never have the same color.



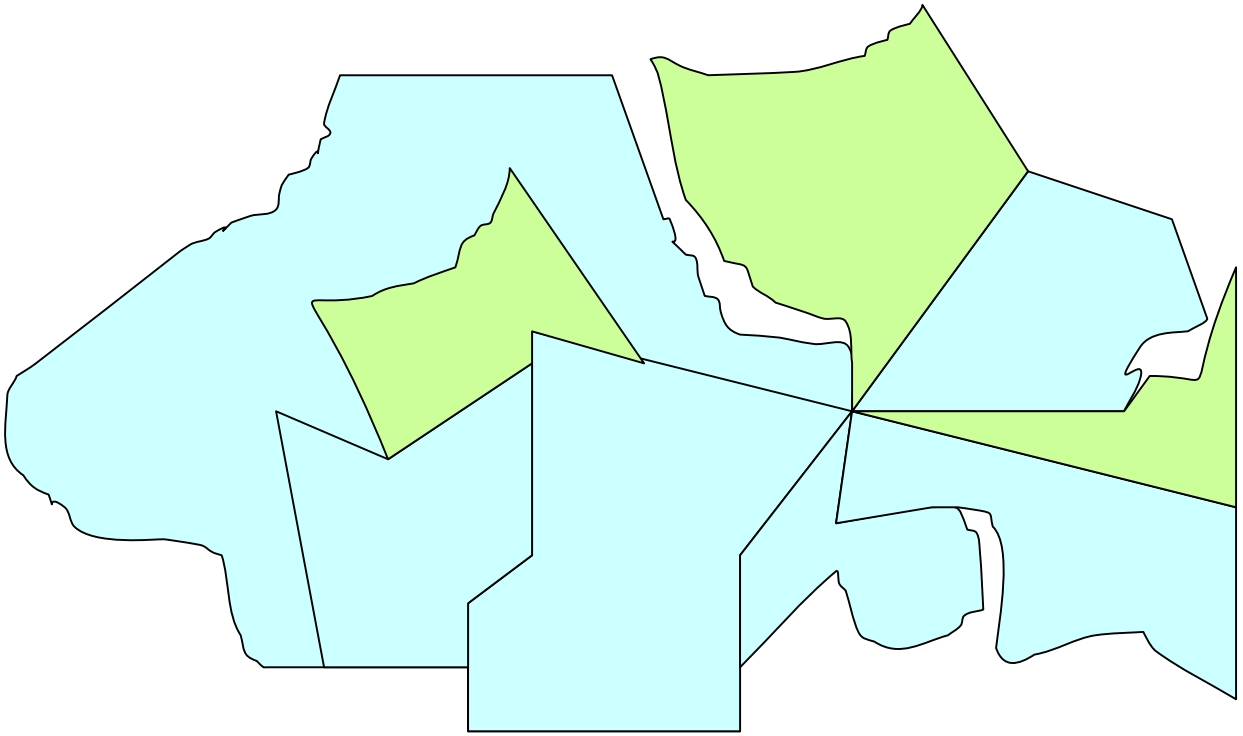
The problem related to the coloring of maps:

Determining the least number of colors that can be used to color a map so that adjacent regions never have the same color.



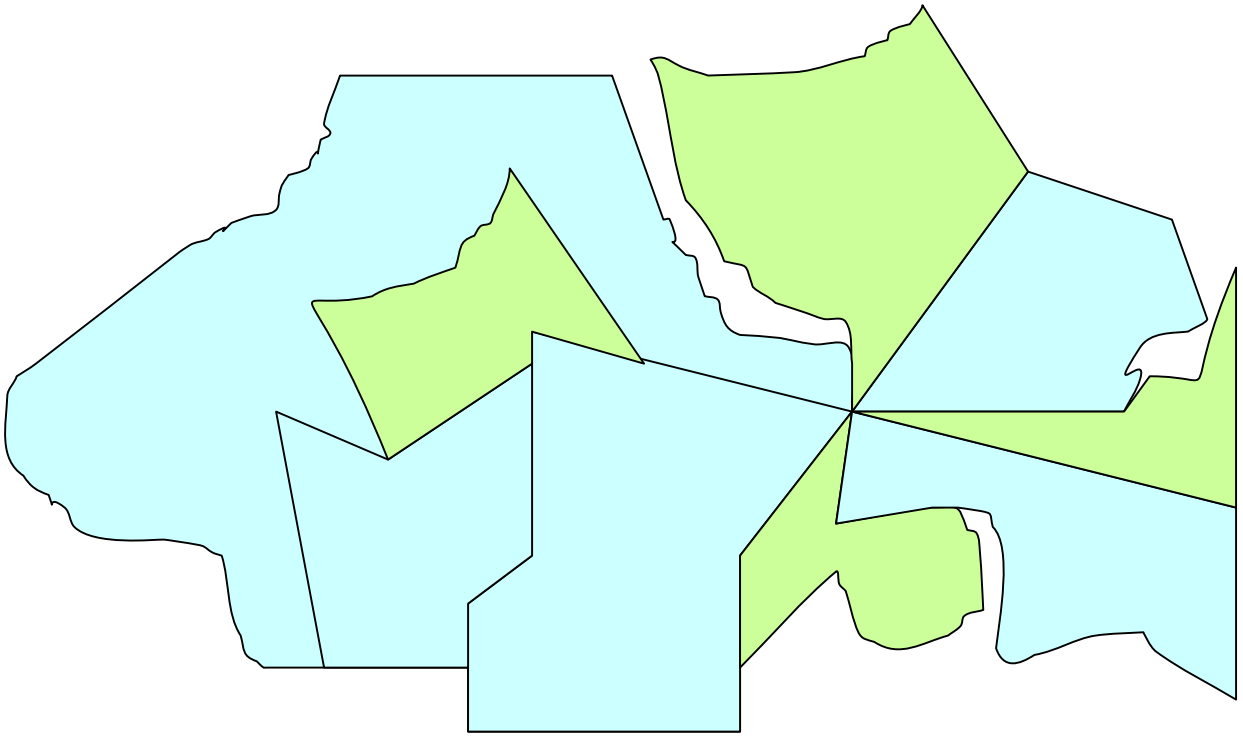
The problem related to the coloring of maps:

Determining the least number of colors that can be used to color a map so that adjacent regions never have the same color.



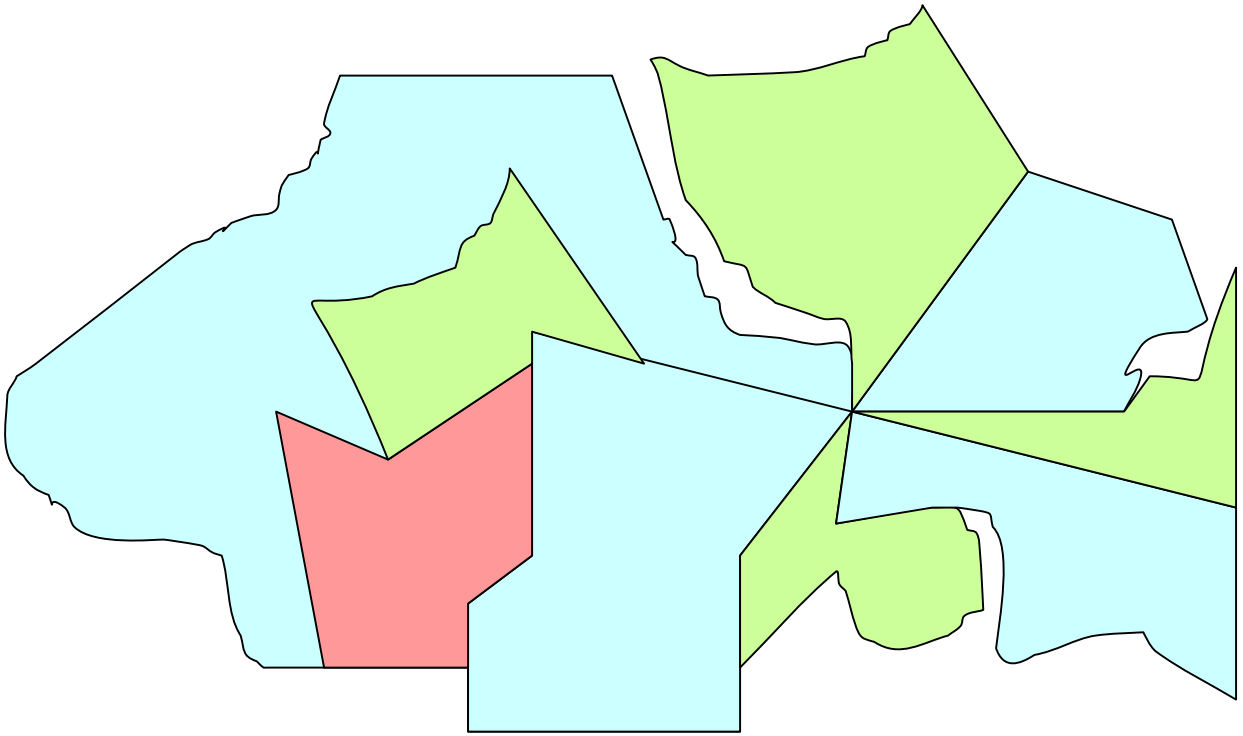
The problem related to the coloring of maps:

Determining the least number of colors that can be used to color a map so that adjacent regions never have the same color.



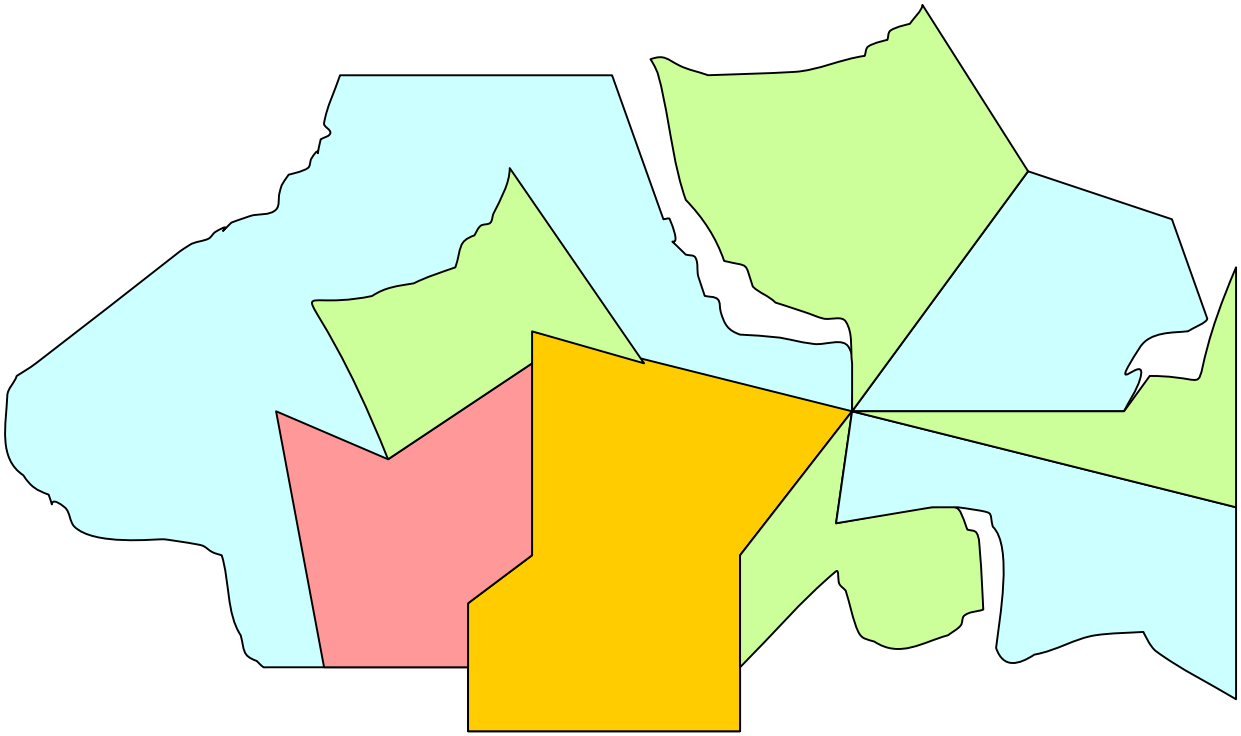
The problem related to the coloring of maps:

Determining the least number of colors that can be used to color a map so that adjacent regions never have the same color.



The problem related to the coloring of maps:

Determining the least number of colors that can be used to color a map so that adjacent regions never have the same color.

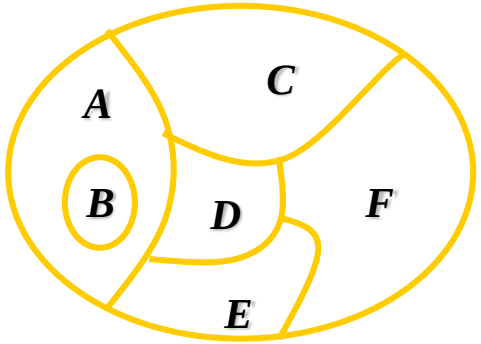


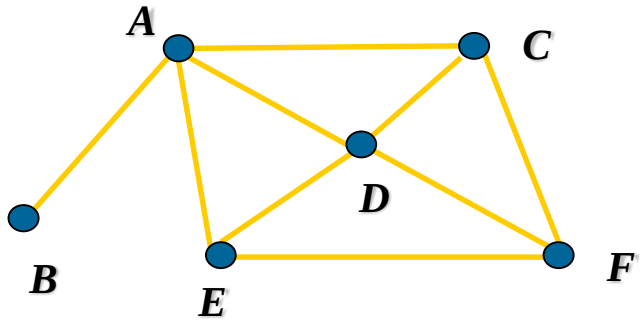
The problem of coloring a map, can be reduced to a graph-theoretic problem.

Each map in the plane can be represented by a graph, namely *the dual graph of the map*.

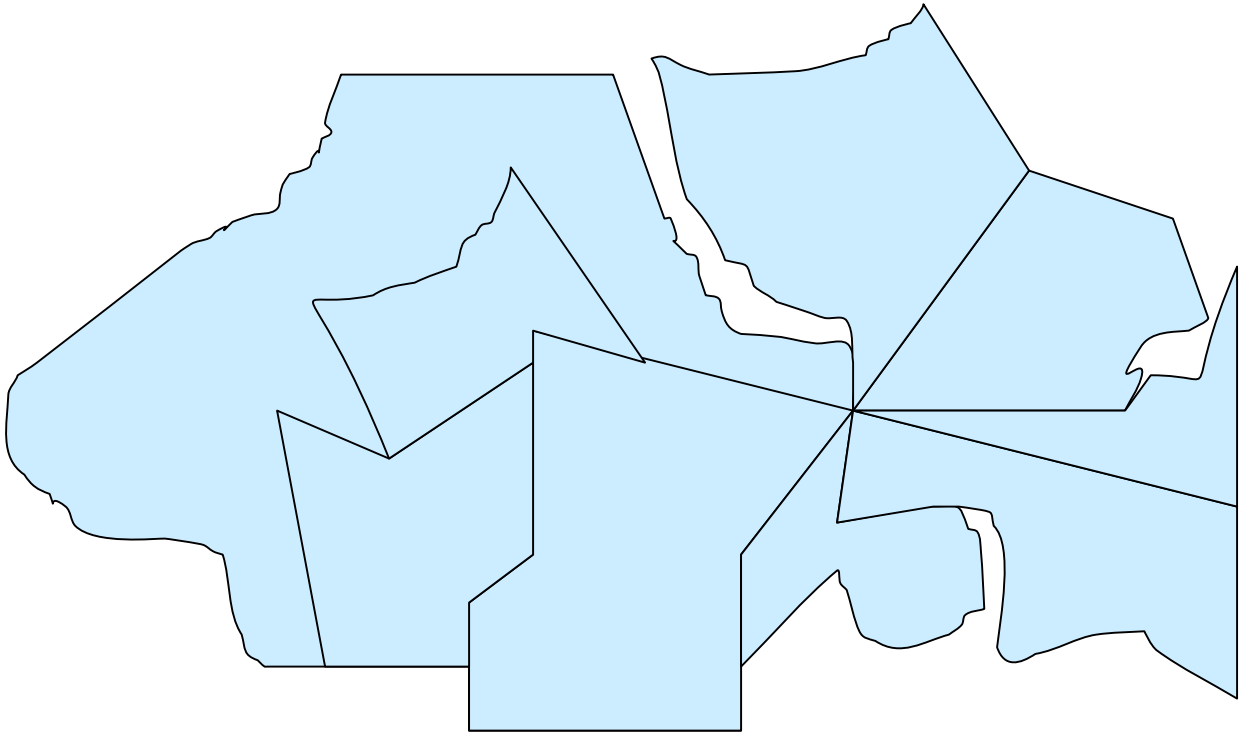
- **Each region of the map is represented by a vertex.**
- **Edge connect two vertices if the regions represented by these vertices have a common border.**
- **Two regions that touch at only one point are not considered adjacent.**

For example,

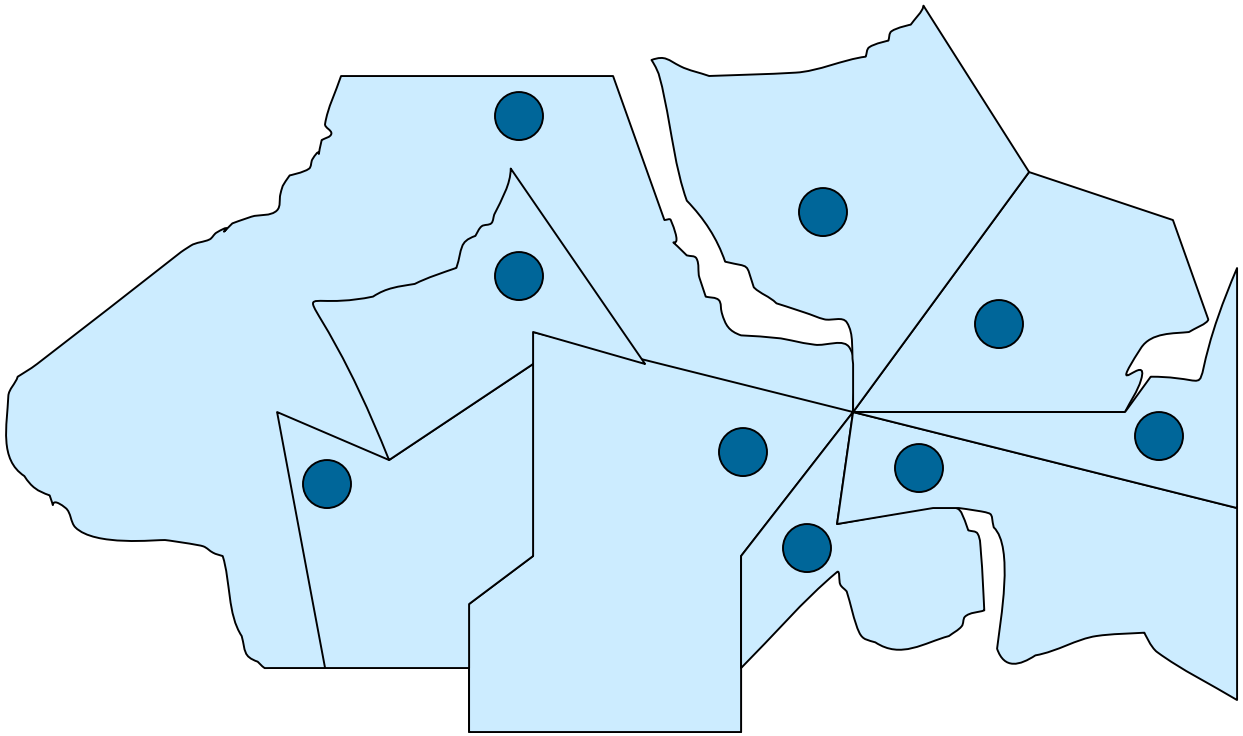




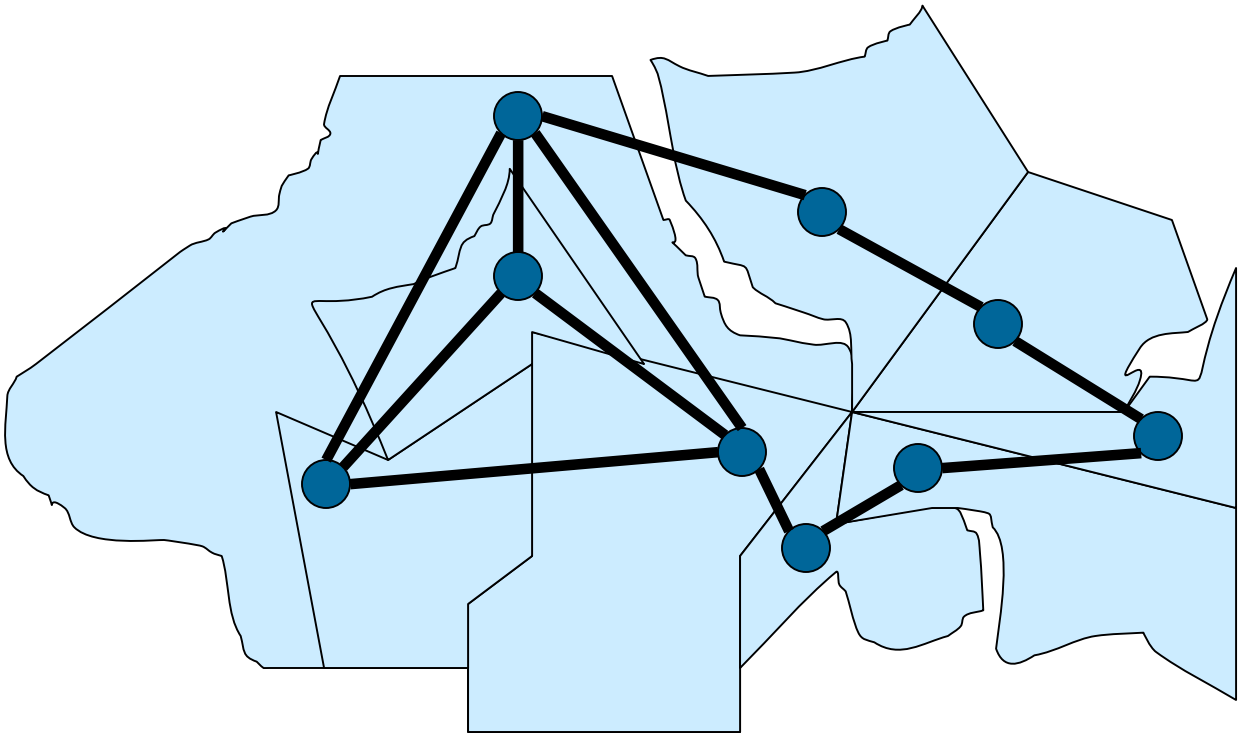
Coloring a map is equivalent to coloring the vertices of the dual graph so that no two adjacent vertices in this graph have the same color.



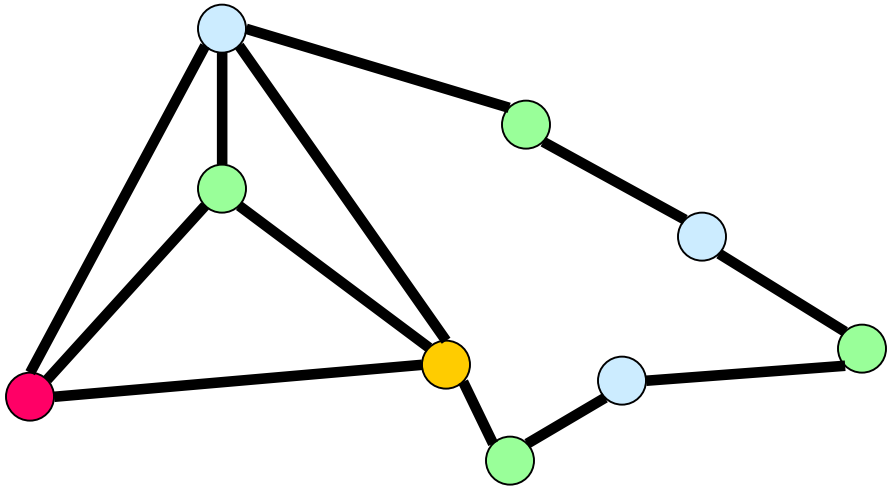
For each region introduce a vertex.



For each pair of regions with a positive-length common border introduce an edge.



Coloring regions is equivalent to coloring vertices of dual graph.





Terminologies:

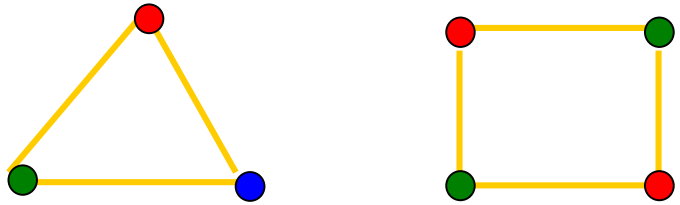
coloring

-- **A *coloring* of a simple graph is the assignment of a color to each vertex of the graph so that no two adjacent vertices are assigned the same color.**

The chromatic number of a graph

-- **is the least number of colors needed for a coloring of this graph, denoted by $x(G)$**

For example,



【Theorem 1】 The Four Color Theorem

The chromatic number of a planar graph is no greater than four.

- **Any planar map of regions can be depicted using 4 colors so that no two regions that share a positive-length border have the same color.**
- **The four color theorem was originally proposed as a conjecture in 1850s.**
- **Proof by Haken and Appel used exhaustive computer search in 1976.**
- **The four color theorem applies only to planar graphs. Nonplanar graphs can have arbitrarily large chromatic numbers.**



1. The chromatic numbers of some simple graphs

Show that the chromatic numbers of a graph is *n*.

- **Show that the graph can be colored with *n* colors.**

  Method: constructing such a coloring.

- **Show that the graph cannot be colored using fewer than *n* colors.**

The chromatic numbers of some simple graphs:

(1) The graph *G* contains only some isolated vertices.

$$x(G)=1$$

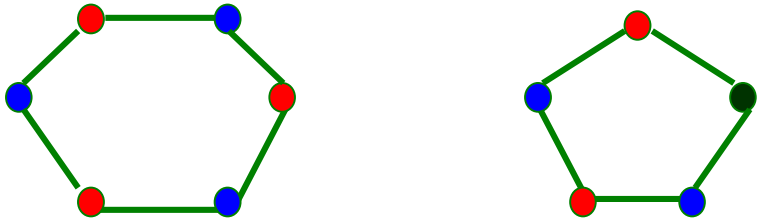


(2) The graph *G* is a path containing no circuit.



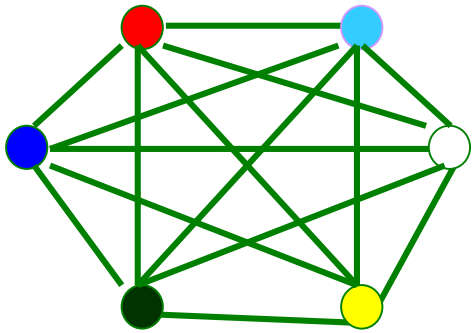
$$x(G) = 2$$

(3) C_n



$$\begin{cases} x(G) = 2 & \textbf{if } n \textbf{ is even} \\ x(G) = 3 & \textbf{if } n \textbf{ is odd} \end{cases}$$

(4) K_n



$$x(G) = n$$

(5) A simple graph with a chromatic number of 2 is bipartite.

A connected bipartite graph has a chromatic number of 2.

2. Applications of graph colorings

(1) Scheduling Exams

How can the exams at a university be scheduled so that no student has two exams at the same time?

Solution:

This scheduling problem can be solved using a graph model, with vertices representing courses and with an edge between two vertices if there is a common student in the courses they represent.

Each time slot for a final exam is represented by a different color.

A scheduling of the exams corresponds to a coloring of the associated graph.



For example, Suppose want to schedule some final exams for CS courses with following call numbers:

1007, 3137, 3157, 3203, 3261, 4115, 4118, 4156

Suppose also that there are no common students in the following pairs of courses because of prerequisites:

1007-3137

1007-3157, 3137-3157

1007-3203

1007-3261, 3137-3261, 3203-3261

1007-4115, 3137-4115, 3203-4115, 3261-4115

1007-4118, 3137-4118

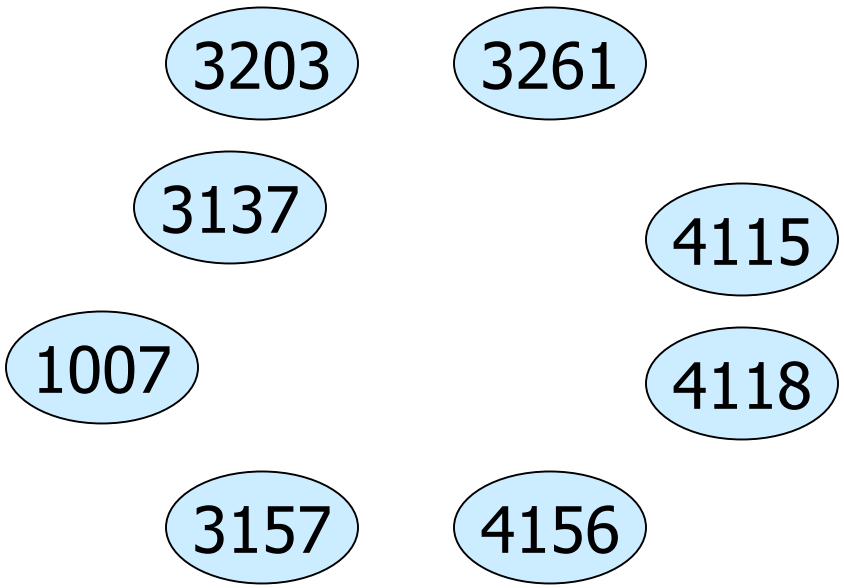
1007-4156, 3137-4156, 3157-4156

How many exam slots are necessary to schedule exams?

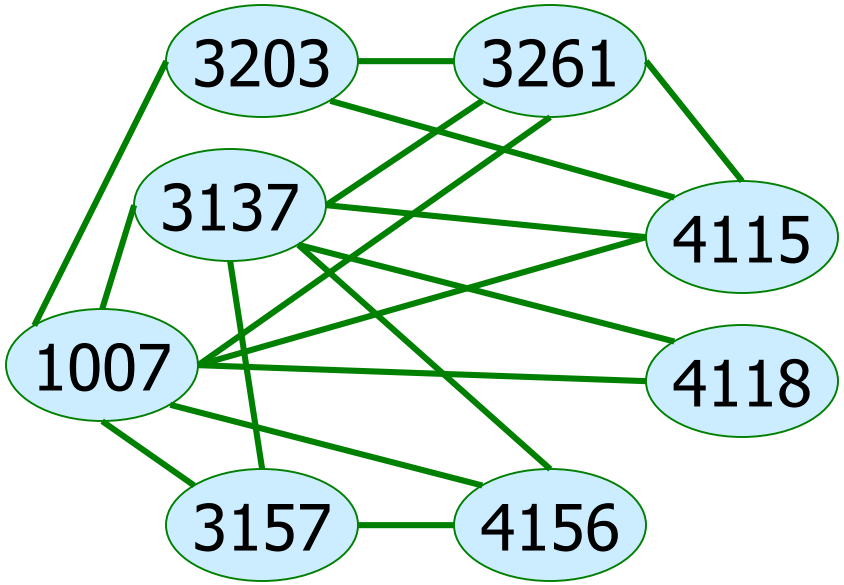


Turn this into a graph coloring problem.

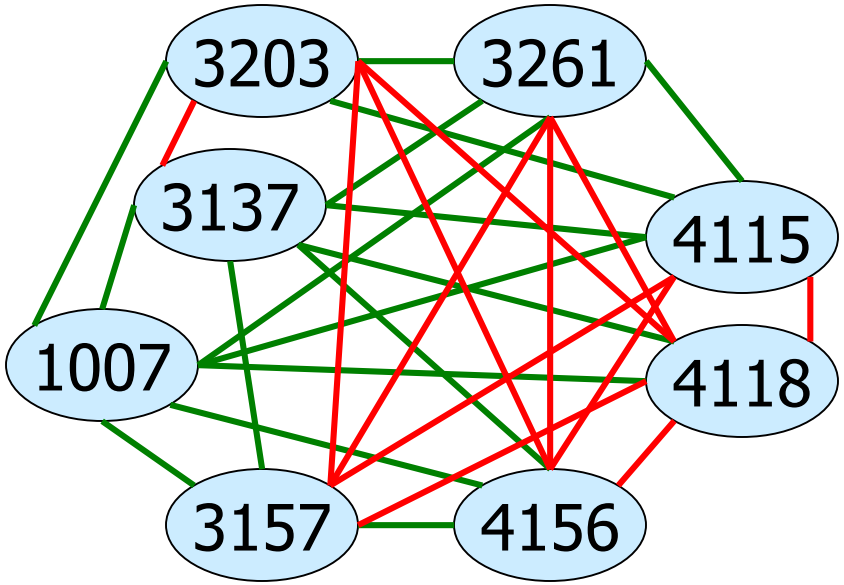
-- Vertices are courses, and edges are courses which *cannot* be scheduled simultaneously because of possible students in common:



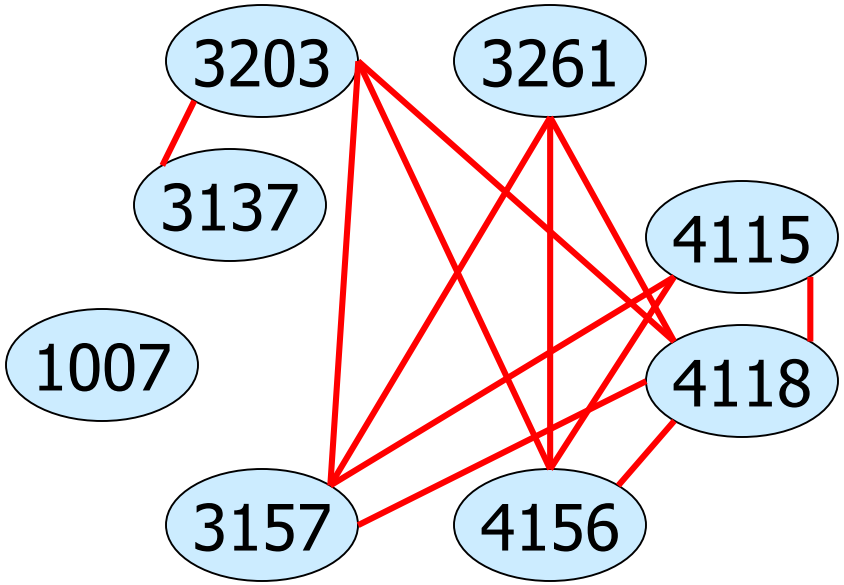
One way to do this is to put edges down where students mutually excluded…



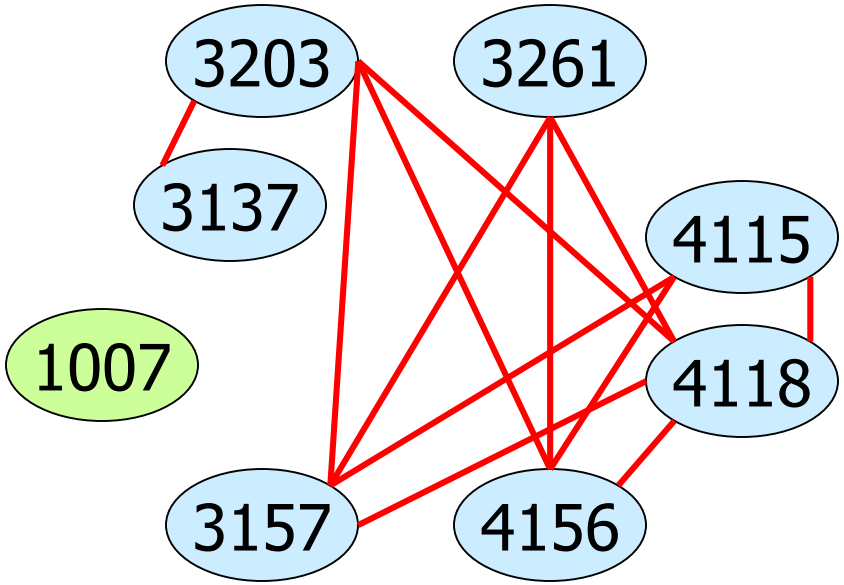
…and then compute the complementary graph:



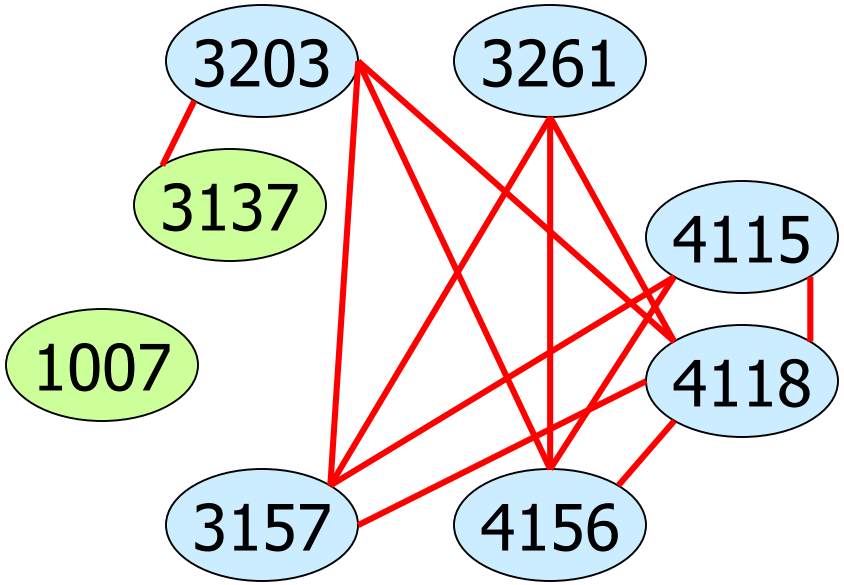
…and then compute the complementary graph:



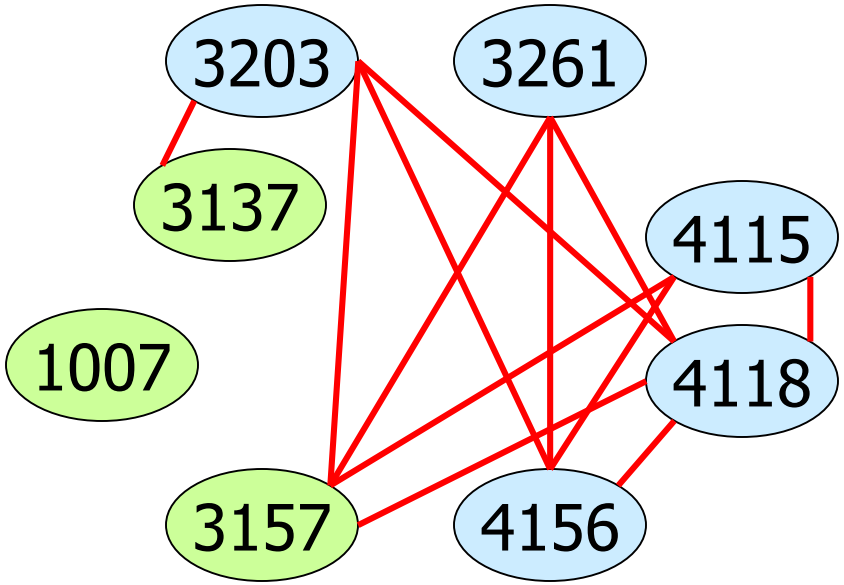
Coloring …



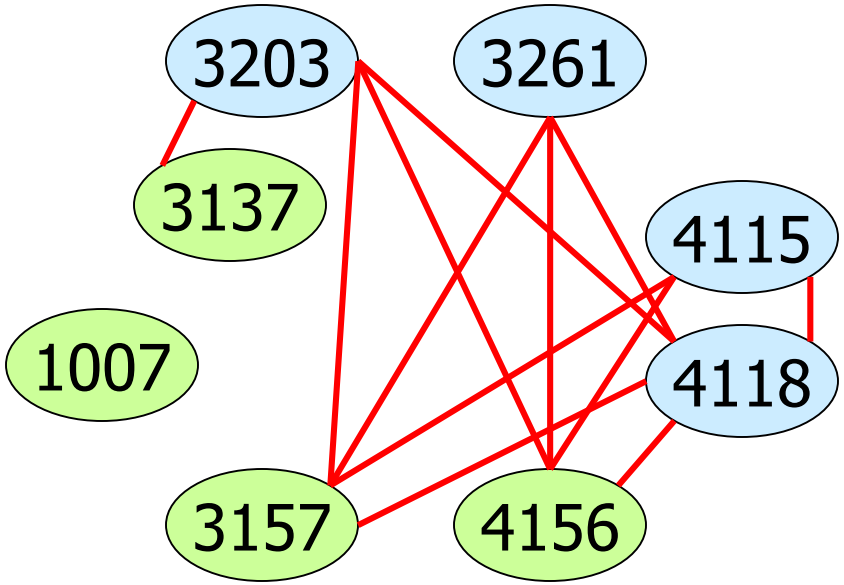
Coloring …



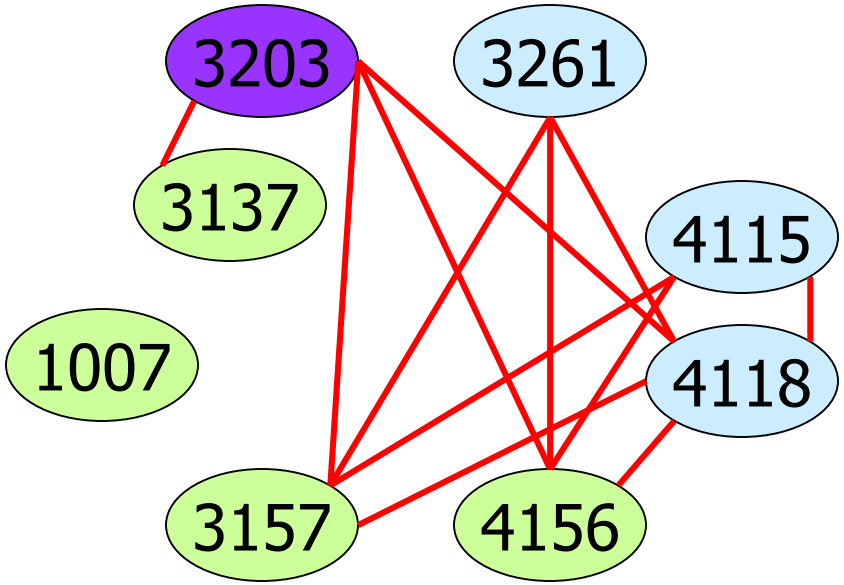
Coloring …



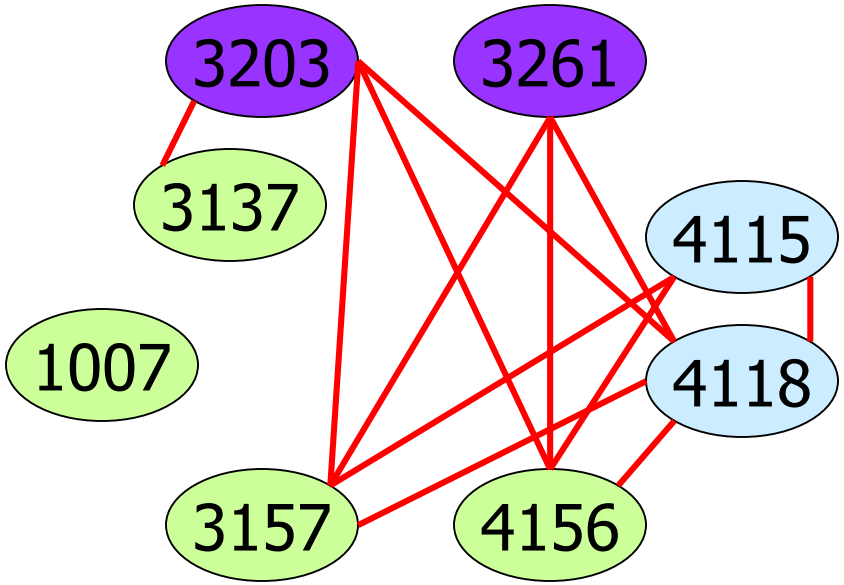
Coloring …



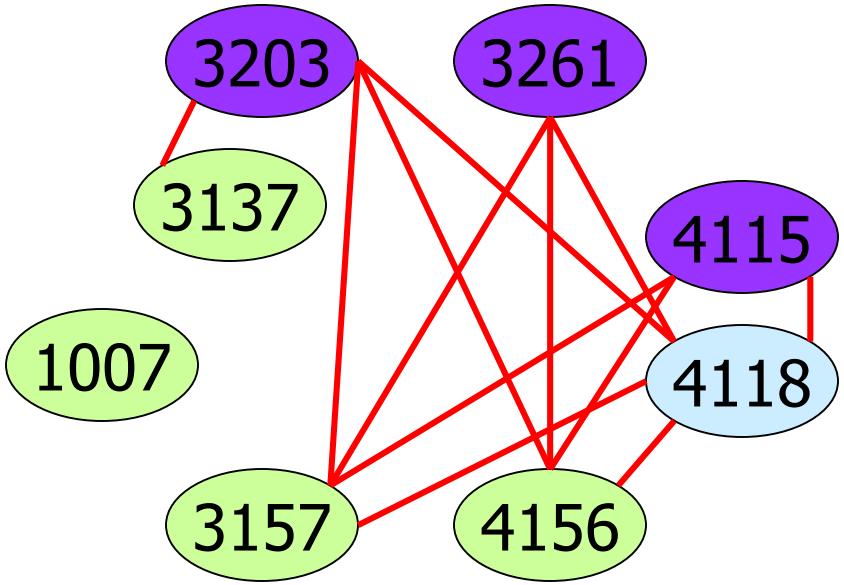
Coloring …



Coloring …



Coloring …



2. Applications of graph colorings

(2) Set up natural habitats of animal in a zoo

Solution:

Let the vertices of a graph be the animals.

Draw an edge between two vertices if the animals they represent cannot be in the same habitat because of their eating habits.

A coloring of this graph gives an assignment of habitats.



Homework:

P. 768: 3, 8-11, 17

